

UNIT-1

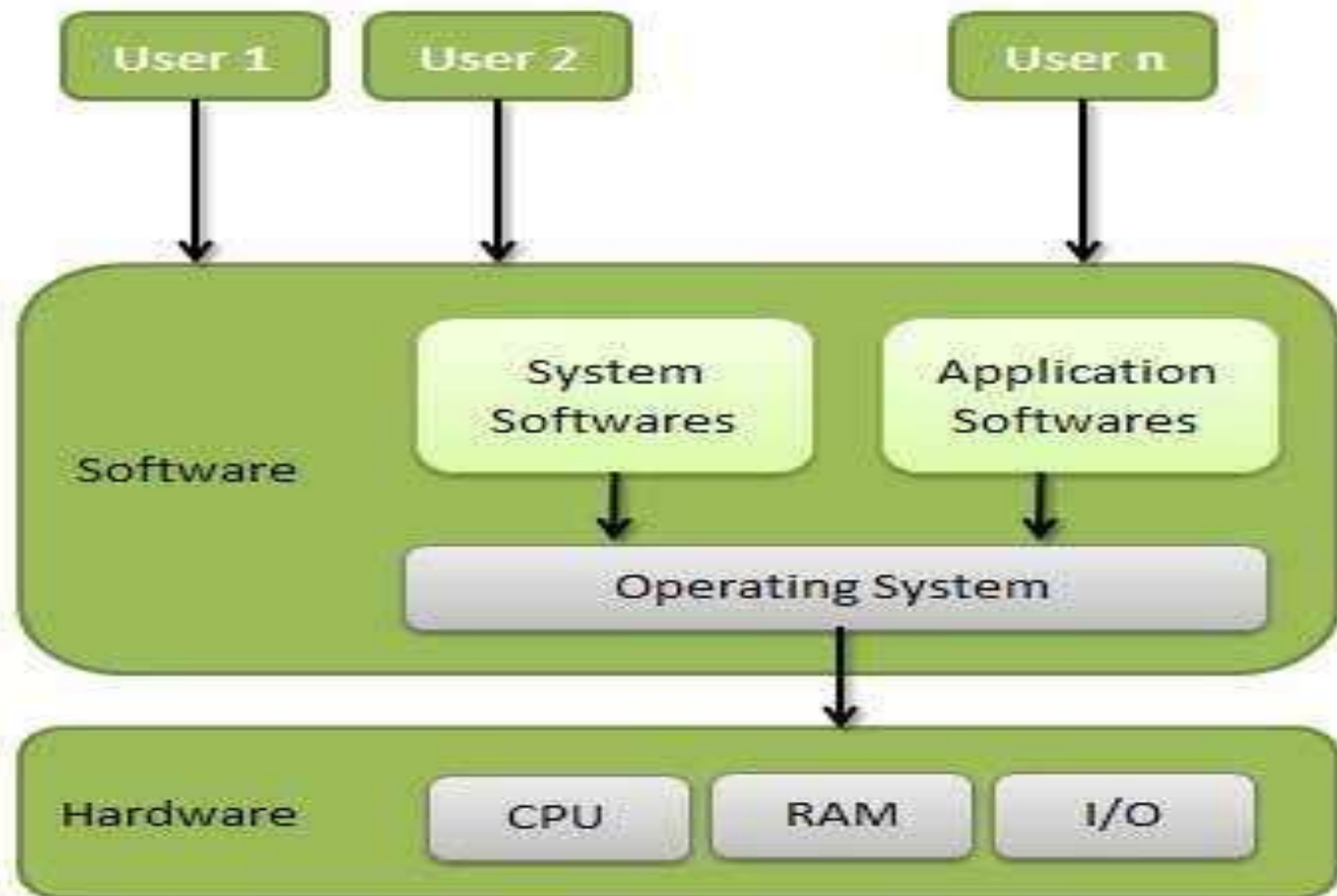
Operating Systems & System Programming

(23ACA06)

Operating System

- An operating system is an interface **between a computer user and computer hardware.**
- An operating system is a software which performs all the basic tasks **such as file management, memory mgmt, process mgmt, handling input and output, and controlling peripheral devices such as disk drives and printers.**
- The software that contains the core components of the operating system is called the **kernel.**

Architecture of OS



Computer System Components

1. **Hardware** – provides basic computing resources (CPU, memory, I/O devices).
2. **Operating system** – controls and coordinates the use of the hardware among the various application programs for the various users.
3. **Applications programs** – The system resources are used to solve the computing problems of the users.
4. **Users** - (people, machines, other computers). Abstract View of System Components

Operating system goals

- Execute user programs and make solving user problems easier.
- Make the computer system convenient to use.
- Use the computer hardware in an efficient manner.

Functions of Operating System

i. Process Management

- A process is a program in execution.
- A process needs certain resources, including CPU time, memory, files, and I/O devices, to accomplish its task.

The Following activities are,

- ◆ Process creation and deletion.
- ◆ process suspension and resumption.
- ◆ Provision of mechanisms for:
 - • process synchronization
 - • process communication

ii. Main-Memory Management

- Memory is a large **array of words or bytes**, each with its **own address**.
- It is a repository of quickly accessible data shared by the CPU and I/O devices.
- Main memory is a volatile storage device.
- It loses its contents in the case of system failure.
- Following activities are,
 - ◆ Keep track of which parts of memory are currently being used and by whom.
 - ◆ Decide which processes to load when memory space becomes available.
 - ◆ Allocate and de-allocate memory space as needed.

iii. File Management

- A file is a collection of related information defined by its creator. Commonly, files represent programs and data.

Following activities are,

- ◆ File creation and deletion.
- ◆ Directory creation and deletion.
- ◆ Support of primitives for manipulating files and directories.
- ◆ Mapping files onto secondary storage.
- ◆ File backup on stable (non-volatile) storage media.

iv. I/O System Management

The I/O system consists of:

- ◆ A buffer-caching system
- ◆ A general device-driver interface
- ◆ Drivers for specific hardware devices

v. Secondary-Storage Management

- Main memory (primary storage) is volatile and too small to accommodate all data and programs permanently the computer system must provide secondary storage to back up main memory.
- the following activities are,
 - ✦ Free space management
 - ✦ Storage allocation
 - ✦ Disk scheduling

Types of Operating Systems

Operating systems can be classified into several types based on their functionality and usage.

- Batch, Time-Sharing, Distributed,
- Network,
- Real-time, Mobile,
- Embedded operating systems.

Additionally, there are based on user count

- single-user,
- multi-user,
- processing capabilities (multiprogramming, multiprocessing, multitasking), and other factors.

1. Batch Operating System:

- Processes **jobs in batches** without direct user interaction.
- Suitable for **tasks that can be completed offline**, like payroll processing or generating reports.

2. Time-Sharing Operating System:

- **Allows multiple users** to access the system simultaneously, sharing resources like CPU time.
- Enables interactive computing, **where users can run programs and receive feedback** in real-time.

3. Distributed Operating System:

- Manages a **collection of interconnected computers**, presenting them as a **single system**.
- Facilitates **resource sharing and communication** between different machines.

4. Network Operating System:

- **Runs on a server and manages network** resources like files, printers, and users.
- Provides services for **network communication, security, and administration**.

5. Real-Time Operating System:

- Designed for systems **where timing is critical**, like industrial control or medical devices.
- Responds to events within a strict time constraint.

6. Mobile Operating System:

- Specifically designed for mobile devices like **smartphones and tablets**.
- Optimized for touch-based interfaces and mobile-specific features.

7. Embedded Operating System:

- Used in specialized devices like **appliances, vehicles, and industrial equipment.**
- Typically **lightweight and tailored** to the specific hardware and functionality of the device.

Other important distinctions:

Single-user vs. Multi-user:

- Single-user OSes support **only one user at a time**, while multi-user OSes **allow multiple users** to access the system concurrently.

MultiProgramming vs. MultiTasking vs. MultiProcessing:

- **Multiprogramming:** Allows multiple programs to reside in memory and be executed concurrently, improving CPU utilization.
- **Multitasking:** Allows a single user to run multiple programs seemingly at the same time, giving the illusion of parallel execution.
- **Multiprocessing:** Uses multiple processors to execute tasks, significantly increasing processing power.

Generation of Operating System

- **First Generation (1945-1955):**
Vacuum Tubes and Plugboards
- Computers in this era were **large and expensive, relying on vacuum tubes** for computation.
- Operating systems were rudimentary or non-existent, with **manual loading of program code**.
- **Users interacted directly** with the hardware using plugboards to input instructions.

2. Second Generation (1955-1965): Transistors and Batch Systems

- The invention of the **transistor** led to smaller, faster, and more reliable computers.
- Batch processing systems emerged, **allowing users to submit jobs** (programs and data) to be **executed sequentially**.
- **Magnetic tapes** were used for storage and input/output operations.
- Examples include early **IBM systems** and the introduction of OS/360.

3. Third Generation (1965-1980): Integrated Circuits and Multiprogramming

- The use of **integrated circuits** (ICs) resulted in further miniaturization and increased computing power.
- Multiprogramming **allowed multiple programs** to reside in memory and **share the CPU, improving efficiency.**
- **Time-sharing systems** allowed multiple users to interact with the computer simultaneously.
- Examples include the **development of Unix** and the rise of **personal computers.**
- **Graphical User Interfaces (GUIs)** began to emerge.

4. Fourth Generation (1980-Present): Personal Computers and Networking

- The invention of the **microprocessor** led to the development of personal computers.
- **GUIs became more sophisticated**, making computers more **user-friendly**.
- **Networking technologies** advanced, leading to the development of network operating systems.
- **Examples:** Windows, macOS, Linux, and Android.
- The focus shifted towards **mobility and cloud computing**.

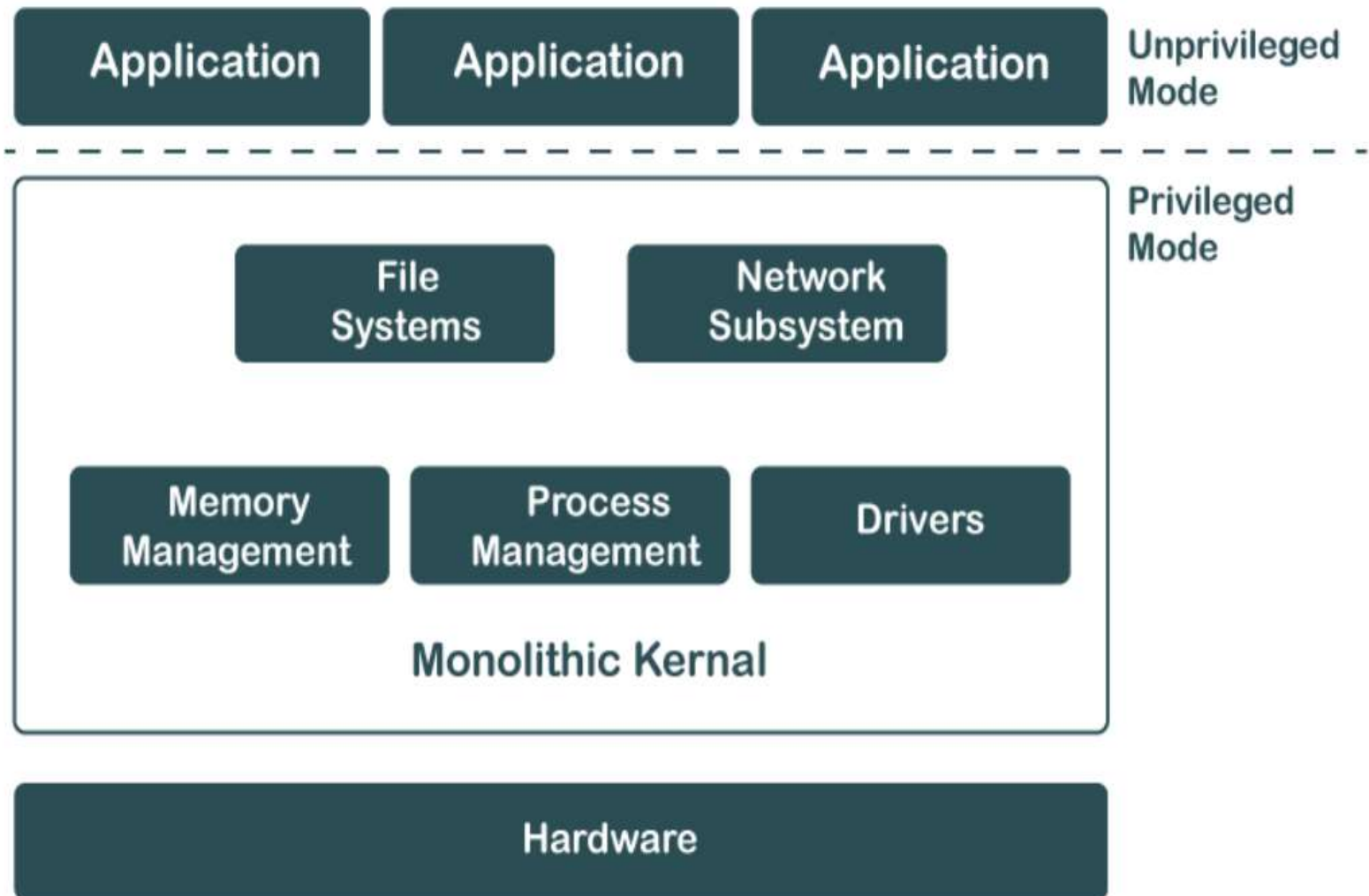
OS Structure Methods

- OS Structure Methods is a design that **enables user application programs** (MSO, WB, MP, Comm.Player. Whatsapp, gmail etc..) **to communicate with the hardware of the machine.**
- Monolithic Structure
- Layered Approach Structure
- Modular Structure
- Micro-Kernel Structure

1. MONOLITHIC STRUCTURE

- The monolithic operating system **controls all aspects of the operating system's operation**, including file management, memory management, device management, and operational operations.
- The core part of OS is called the kernel (OS).
- All other System components are **provided with fundamental services by the kernel**.
- The OS and H/W use it as their main interface.

Monolithic Kernal System

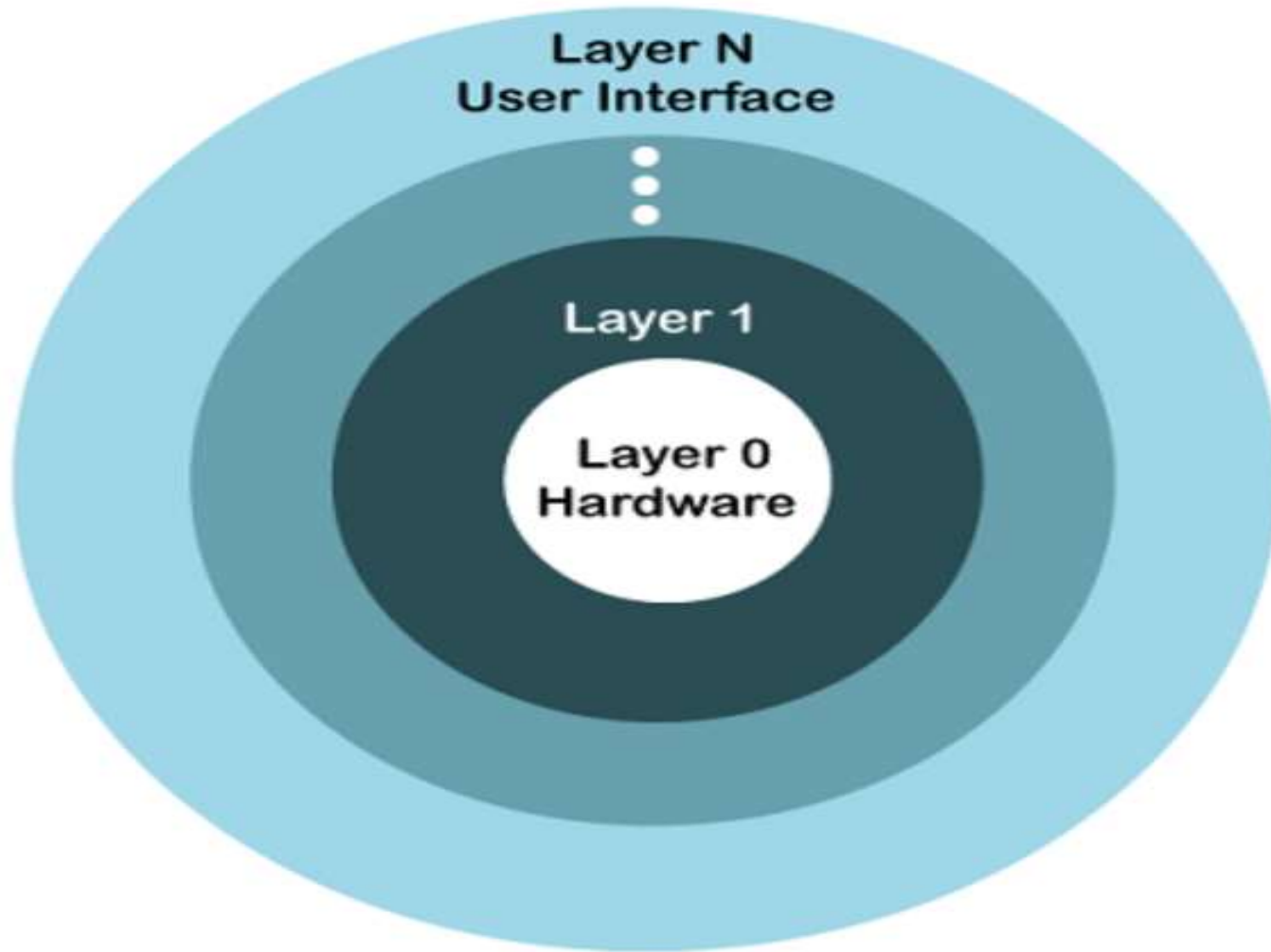


- **Monolithic kernel** performs the role of a virtual computer.
- **Monolithic kernel** is an old operating system that was used in banks to carry out simple tasks like batch processing and time-sharing, which allows numerous users at different terminals to access the Operating System.

2. LAYERED STRUCTURE

- The OS is **separated into layers or levels** in this kind of arrangement.
- **Layer 0** (the lowest layer) contains the hardware.
- **layer 1** (the highest layer) contains the user interface (layer N).
- These layers are organized view of hierarchically, with the **top-level layers making use of the capabilities of the lower-level ones.**
- The layered structures are hierarchical, debugging is simpler, so that, **all lower-level layers are debugged before the upper layer is examined.**

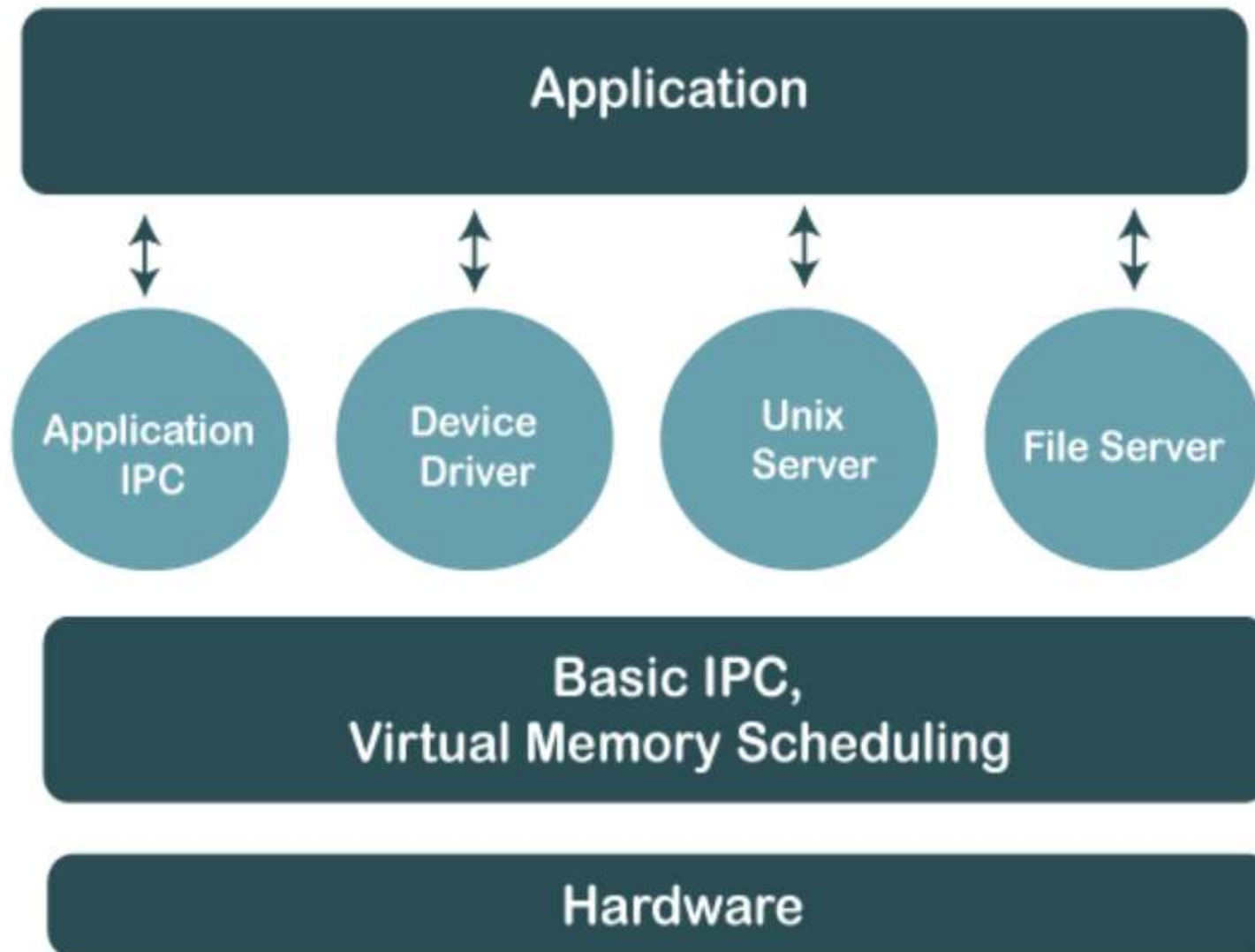
LAYERED STRUCTURE



3. MICRO-KERNEL STRUCTURE

- **Operating System** using a micro-kernel framework that strips the kernel of any unnecessary parts.
- Each Micro-Kernel is created separately and is kept apart from the others.
- **If one Micro-Kernel effect by malfunctions,** the remaining operating system is unaffected and continues to function normally.
- As result, the system is now more trustworthy and secure.

Microkernal Operating System



OS Services

There are five services,

i. 1. Program Execution

- The computer systems is to allow the user to execute programs.
- So the operating system provides an environment where the user can conveniently run programs.
- **Running a program involves** the allocating and de-allocating memory, CPU scheduling in case of multiprocessing.

Cont...

2. I/O Operations

- Each program requires an input and produces output.
- So, the operating systems are providing I/O makes it convenient for the users to run programs.

3. File System Manipulation

- The output of a program may need to be written into new files or input taken from some files. The operating system provides this service.

4. Communications

- The processes need to communicate with each other to exchange information during execution.
- Communications may be between processes running on the same computer or running on the different computers.
- Communications occur in two ways:
 - (i) shared memory
 - (ii) message passing

Cont...

5. Error Detection

- An error is one part of the **system may cause malfunctioning** of the complete system.
- **To avoid such a situation** operating system constantly monitors the system for detecting the errors.
- **There are three services** provided by operating systems **for ensuring the efficient operation** of the system itself.
 - i. Resource allocation
 - ii. Accounting
 - iii. Protection

Cont...

i. Resource allocation

- When multiple users are logged on the system **or** multiple jobs are running at the same time, resources must be allocated to each of them.
- Many different types of resources are managed by the operating system.

ii. Accounting

- The OS keep track of which **users use how many and which kinds of computer resources.**
- This record keeping may be used for accounting (so that users can be billed) **or** simply for accumulating usage statistics.

Cont...

iii. Protection

- When **several disjointed processes execute concurrently**, it should not be possible for one process to interfere with the others.
- Protection involves ensuring that all access to system resources is controlled.
- **Security of the system from outsiders** is also important. Such as username and password, to be allowed access to the resources.

1. System Call

- ❑ System calls provide an interface **between the process and the operating system.**
- ❑ System calls **allow user-level processes** to request some services from OS which process itself is not allowed to do.

For example, I/O process involves a **system call telling the OS** to read or write particular area and this request is satisfied by the operating system.

Different types of system calls

1. Process control

- end, abort
- load, execute
- create process, terminate process
- get process attributes, set process attributes
- wait for time
- wait event, signal event
- allocate and free memory

2. File management

- create file, delete file
- open, close
- read, write, reposition
- get file attributes, set file attributes

3. Device management

- request device, release device
- read, write, reposition
- get device attributes, set device attributes
- logically attach or detach devices

4. Information maintenance

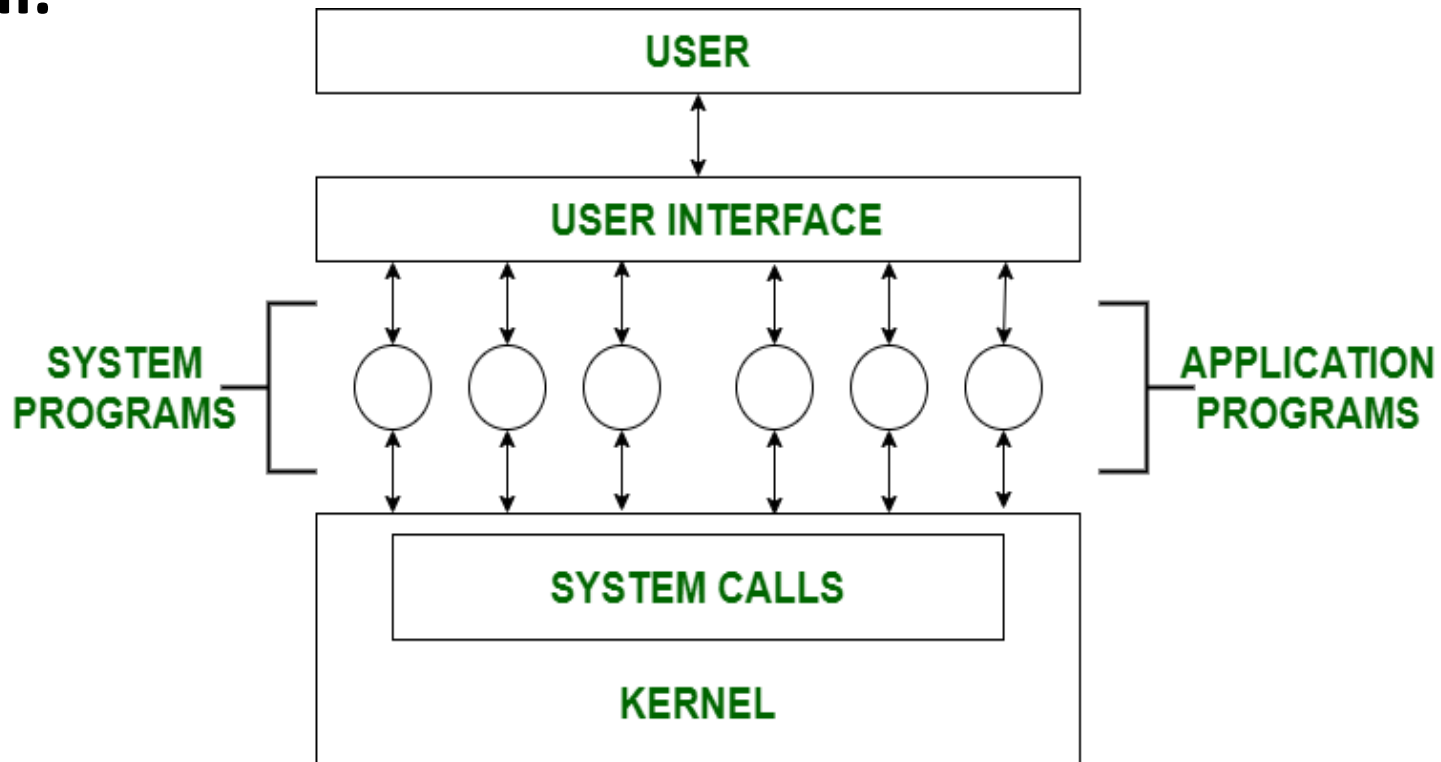
- get time **or** date, set time **or** date
- get system data, set system data
- get process, file, **or** device attributes
- set process, file, **or** device attributes

5. Communications

- create, delete communication connection
- send, receive messages
- transfer status information
- attach or detach remote devices

2. System Programs

- System programs **provide convenient environment** for program development and execution.
- Its acting as a **bridge between the UI and system call.**



Key Functions

- 1. File Management:** A file is a collection of specific information stored in the memory of system.
 - Its defined as the process of manipulating files in the computer system.
 - Its includes the process of creating, modifying and deleting files.
- 2. Command Line Interface(CLI's) :** provide user facility to **write commands directly to the system** for performing any operation.
 - CLIs can perform many tasks like file manipulation, system configuration and etc.

Key Functions

3. Device Drivers:

- it act **as an intermediary** between the OS and devices.
- Its provide facility **to both** OS and devices to understand each other's language **so that** they can work together efficiently without interrupt.

4. Status Information:

- Information like date, time amount of available memory, **or** disk space is asked by some users.
- All this information is formatted and displayed on output devices **or** printed.

Key Functions

5. File Modification:

- This is used for modifying the content of files.
- The files stored on disks **or** other storage devices, we **use different types of editors.**
- For searching contents of files **or** perform transformations of files we **use special commands.**

6. Programming-Language support:

- For common programming languages, we **use Compilers, Assemblers, Debuggers, and interpreters** which are already provided to users.

Key Functions

7. Program Loading and Execution:

- When the program is ready after Assembling and compilation, it must be loaded into memory for execution.
- **A loader** is part of an operating system that is responsible for loading programs and libraries.

Key Functions

8. Communications:

- Programs provides connections among processes, users, and computer systems.
- Users can send messages to another user on their screen.
- User can send e-mail, browsing on web pages, remote login, the transformation of files from one user to another.

3. System boot



3. System boot

- System boot is the process of starting a computer or embedded device by loading **the OS from disk into RAM.**
- It's the **sequence of events that occurs** when you turn on a computer and it becomes ready for use.
- **The boot process can be triggered** by a power button press or a software command



Breakdown Of The Key Stages:

1. **Power-on Self-Test (POST):**
2. **Bootloader Activation:**
3. **Operating System Loading:**
4. **System Initialization:**
5. **User Mode Initialization:**

There are two main types of booting:

- **Cold Booting:** Starting the computer from a completely powered-off state.
- **Warm Booting:** Restarting the computer while it's already powered on.

Booting is crucial for:

- **Transitioning from dormant to active state:** Loading the OS allows the system to become usable.
- **Applying software updates:** The boot process allows the system to check for and install updates.
- **Clearing memory:** Some systems use booting to reset RAM.
- **Recovery:** Booting can help restore a system after a crash.
- **Optimal resource allocation:** Booting ensures hardware and software are properly initialized

4. Virtual Machines

- A virtual machine (VM) in an operating system provides an **isolated environment that behaves like a separate computer.**
- VM allowing **multiple OS** and applications to run concurrently on a **single physical machine.**
- This is achieved through a **software layer called a hypervisor**, which manages and allocates resources to each VM.

Two type of Virtual Machine,

- 1. System Virtual Machine:** gives us complete system platform and gives the execution of the complete virtual OS.
- 2. Process Virtual Machine :** While process virtual machines, unlike system virtual machine, **does not provide us with the facility to install the virtual operating system completely.**

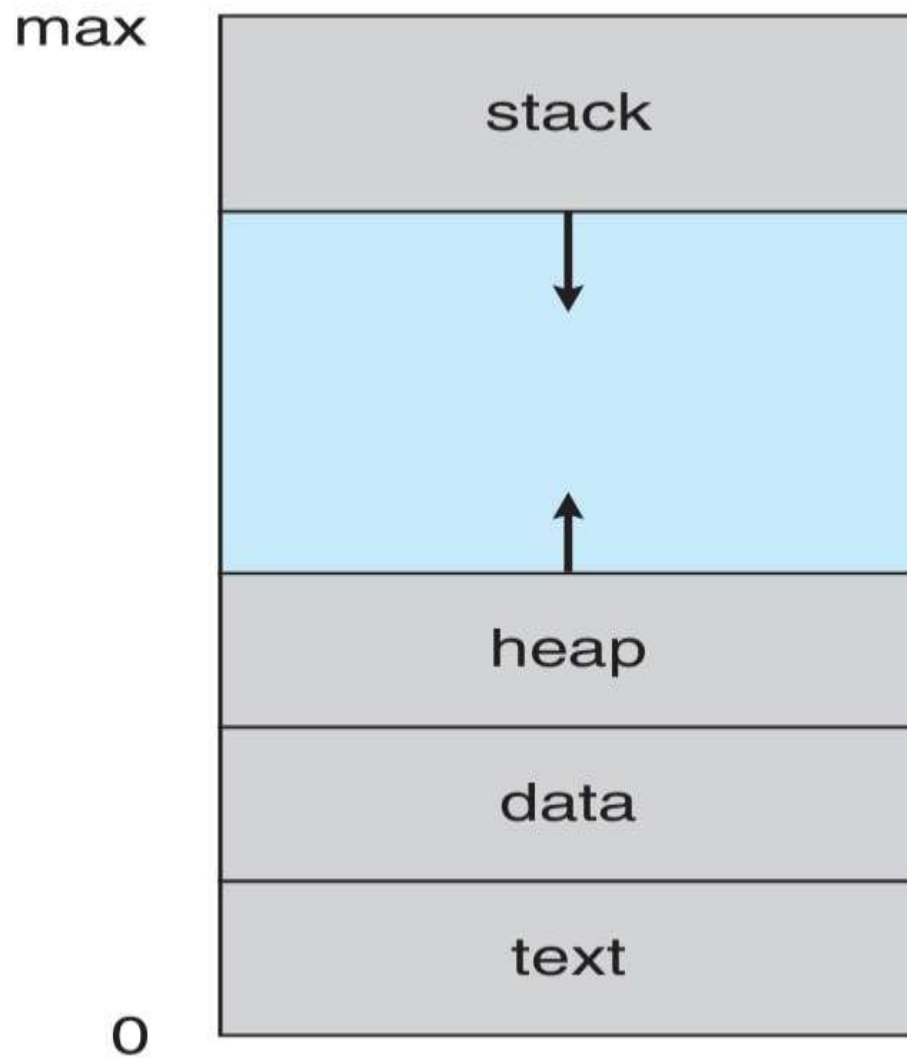
Process Concept

1. **The Process:** An OS executes a variety of programs that run as a process.
 - **Process** – a program in execution;
 - **process execution** must progress in sequential fashion.
 - No parallel execution of instructions of a single process
 - **Multiple parts**
 - The program code, also called **text section**
 - Current activity including **program counter**, processor registers
 - **Stack** containing temporary data
 - Function parameters, return addresses, local variables

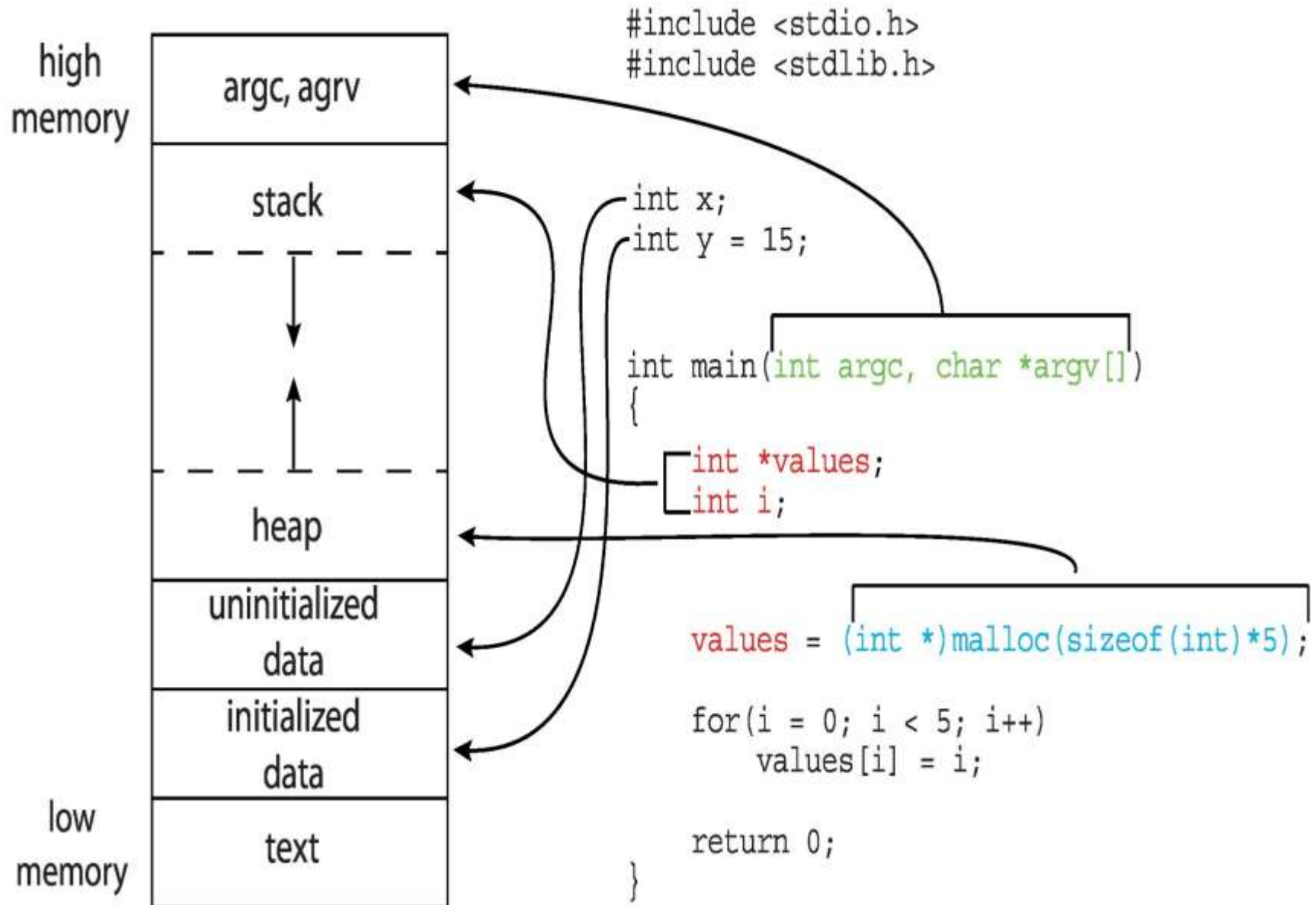
1. Process Concept(cont...)

- **Data section** containing global variables
- **Heap** containing memory dynamically allocated during run time.
- Program is **passive** entity stored on disk (**executable file**); process is **active**
 - Program becomes process when an executable file is loaded into memory
- Execution of program started **via GUI mouse** clicks **or** command line entry of its name, etc.
- One program can be several processes
 - Consider multiple users executing the same program

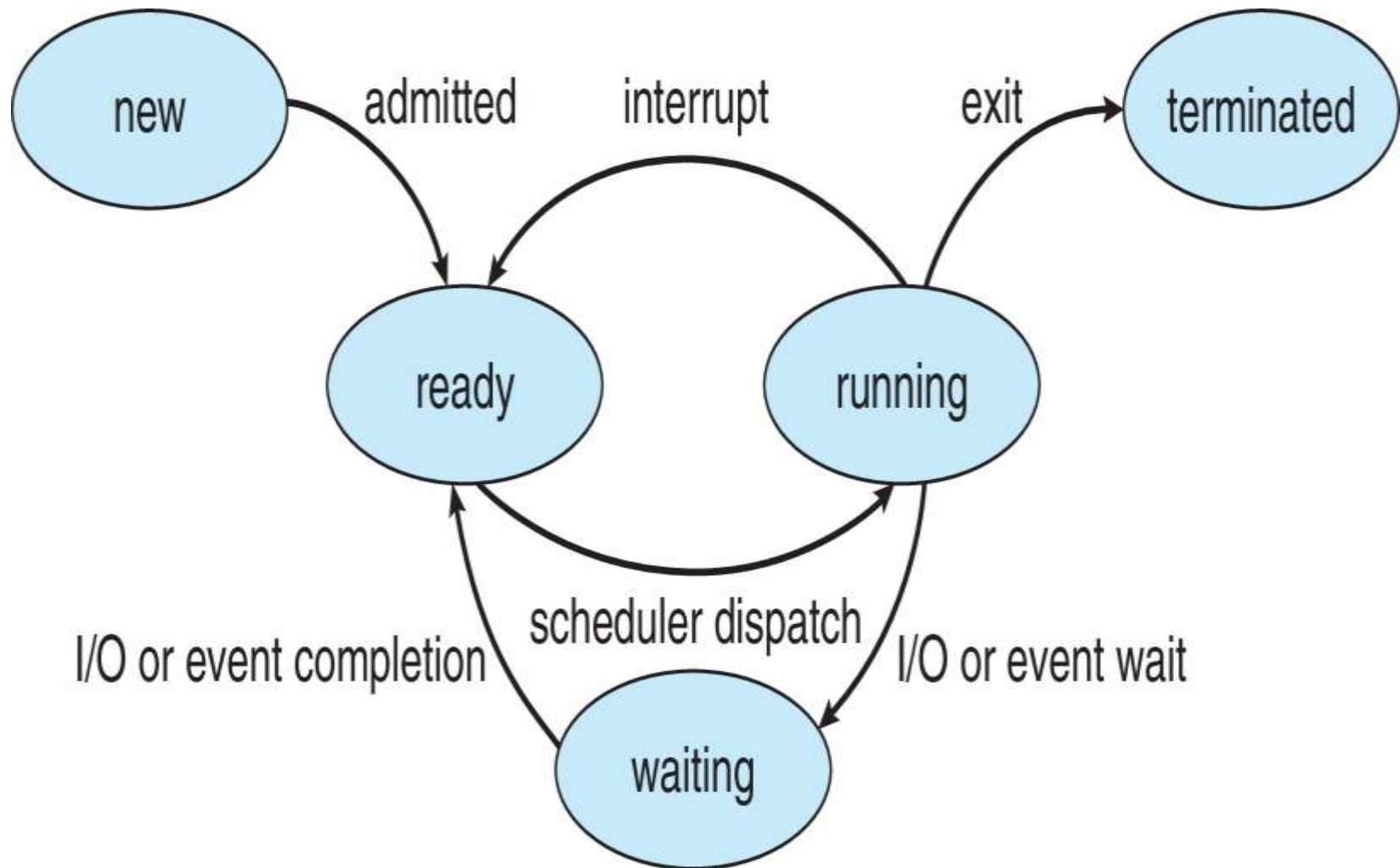
Process in Memory



Memory Layout of a C Program



2. Process State



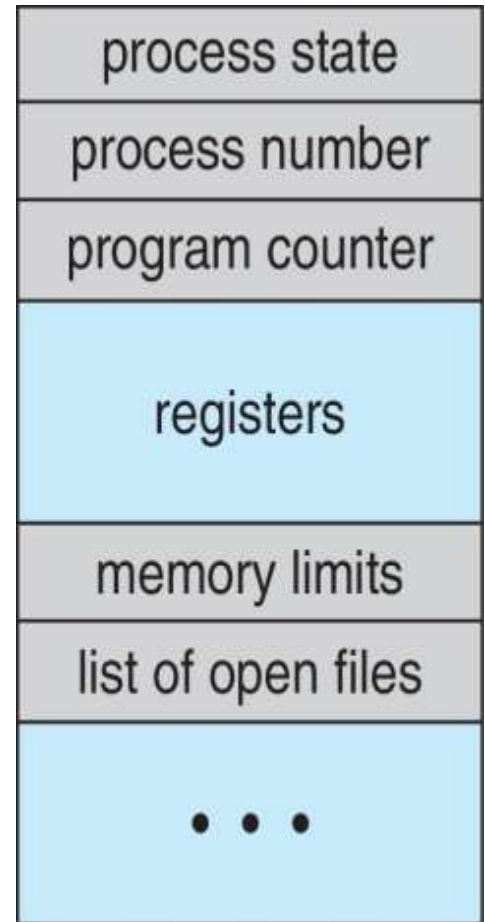
Process State

- As a process executes, it changes **state**
 - **New**: The process is being created
 - **Running**: Instructions are being executed
 - **Waiting**: The process is waiting for some event to occur
 - **Ready**: The process is waiting to be assigned to a processor
 - **Terminated**: The process has finished execution

3. Process Control Block (PCB)

Information associated with each process(also called **task control block**)

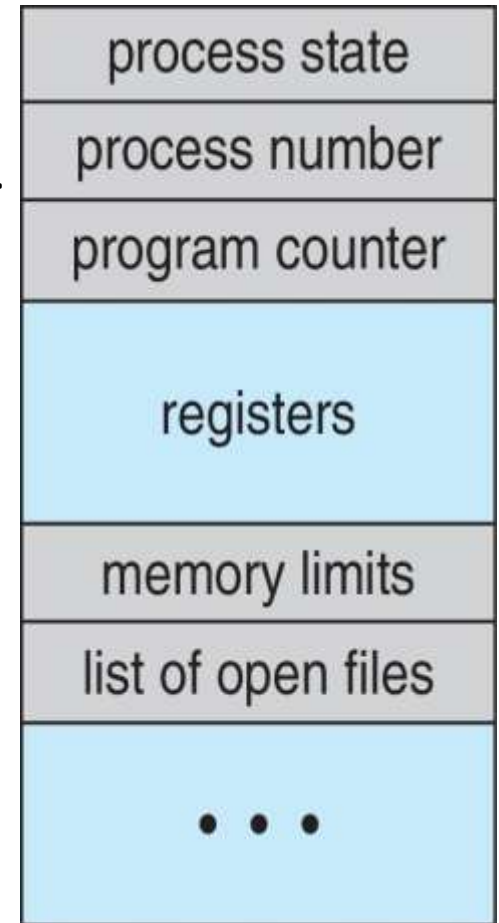
- **Process state** – new, ready, running, waiting, etc.
- **Program counter** – location of instruction to next execute
- **CPU registers** – contents of all process-centric registers
- **CPU scheduling information**- priorities, scheduling queue pointers



Cont...

Information associated with each process(also called **task control block**)

- **Memory-management information** – memory allocated to the process
- **Accounting information** – CPU used, clock time elapsed since start, time limits.
- **I/O status information** – I/O devices allocated to process, list of open files



Cont ...

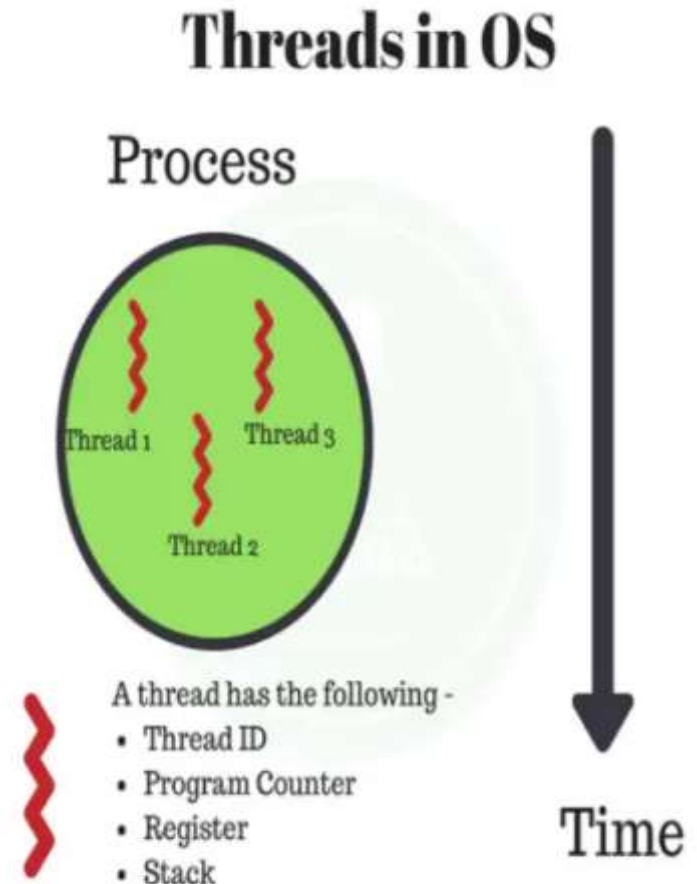
- **PCB** is data structure used to store all the information about a process.
- When a process is created (**initialized or installed**), the **OS creates a corresponding process control block**.
- **PCB** includes CPU scheduling, I/O resource management, file management information etc.
- **By this technique**, the hardware state can be **restored** so that, the process can be **scheduled to run again**.
- **If that process gets suspended**, the contents of the registers are saved on a stack and the particular stack frame is stored in the PCB.

Threads

- A thread is a single sequence stream within a process.
- Threads called **lightweight processes**.
- Each thread belongs to exactly one process.
- OS supports multithreading, the process can consist of many threads.
- **But** threads can be effective only **if the CPU is more than 1** otherwise **two threads have to context switch** for that single CPU.

Threads(cont...)

- A thread comprises of its own
 - **Thread ID** – Unique ID for a thread in execution
 - **Program counter** – Keeps track of instruction to execute
 - **System Register set** – Active Variables of thread
 - **Stack** – All execution history (Can be used for debugging)



Multithreading Models

- Threads may be provided either user level (user threads), or kernel level (kernel threads).

1. User threads

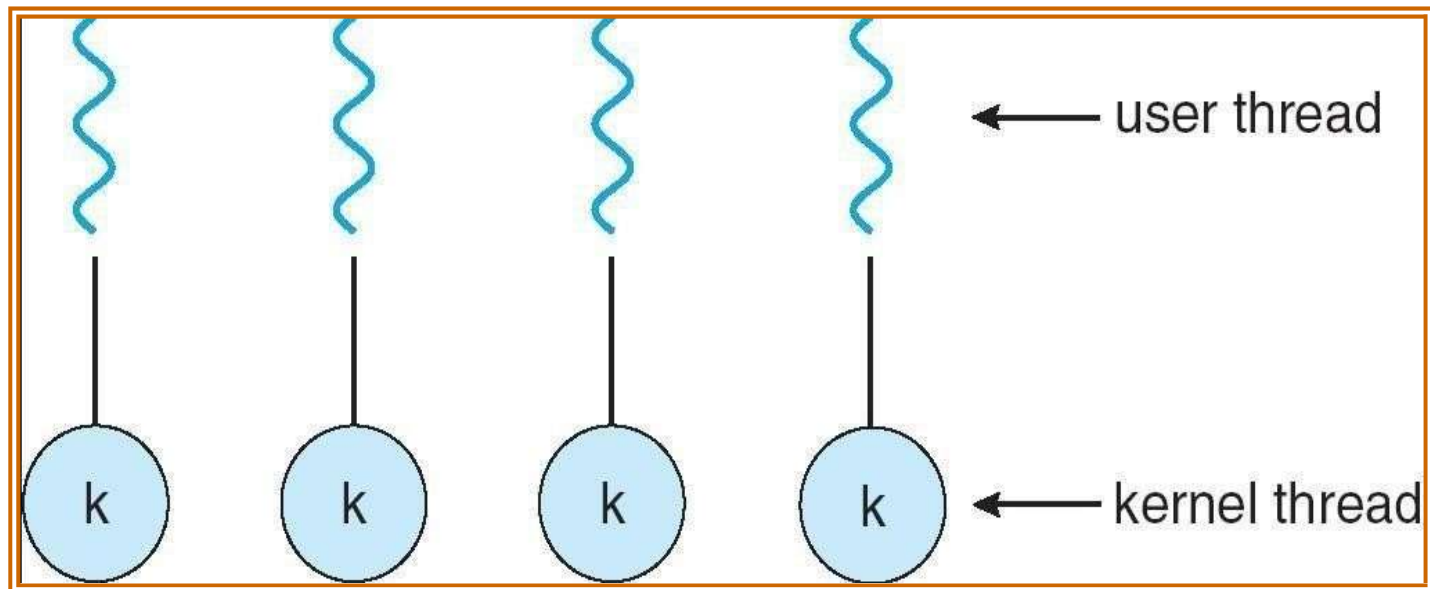
2. Kernel threads

- **User Threads** are supporting the kernel and manage without kernel support.
- **Kernel Threads** are support and manage directly by the operating system.

Multithreading Models(cont..)

- Three common ways of establishing such a relationship:
 1. one-to one model
 2. many-to-one model
 3. many-to-many model.

1.one-to one model

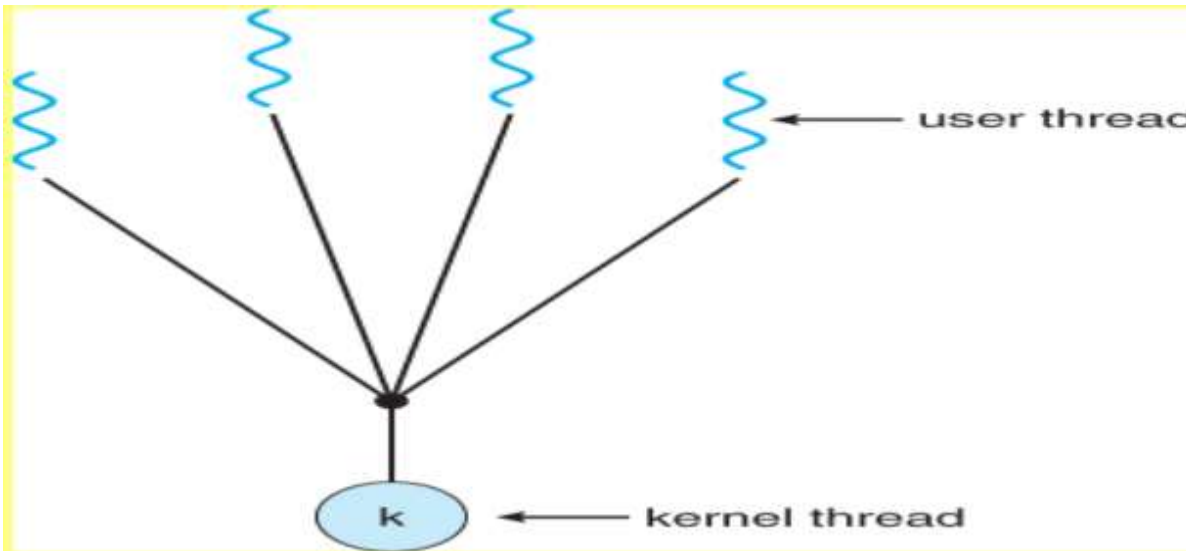


1. One-to-One Model

- This model maps each user thread to a kernel thread.
- It provides more concurrency than many-to-one model by allowing another thread to run when a thread makes a blocking system call.
- It allows multiple threads to run in parallel on multiprocessors.
- Drawback: kernel threads may burden the performance of a system.

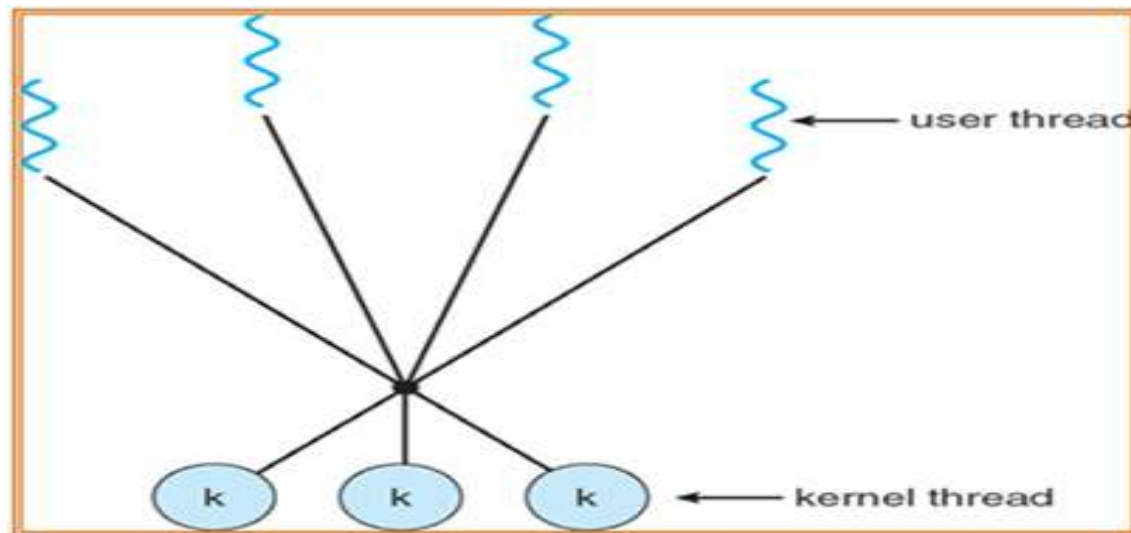
2. Many-to-One Model

- This model maps many user-level threads to one kernel thread.
- Thread management is done by the thread library in user space, so it is efficient.
- However, the entire process will block if a thread makes a blocking system call.



3. Many-to-Many Model

- The many-to-many model (**many user-level threads to many kernel-level threads**) avoids many of the limitations of the one-to-one model while extending multithreading capabilities even further.
- In this model, a user-level threads library provides sophisticated scheduling of user-level threads above kernel threads.



Scheduling Criteria

CPU utilization –

- How efficiently the CPU is being used.
- The goal is to keep the CPU as busy as possible.

Throughput –

- Number of processes complete their execution per time unit.
- Higher throughput means better performance.
-

Scheduling Criteria

Turnaround Time –

- Total time taken from process submission to completion (waiting + execution + I/O time).

Waiting Time –

- Time a process spends waiting in the ready queue.
- Lower waiting time is desirable.

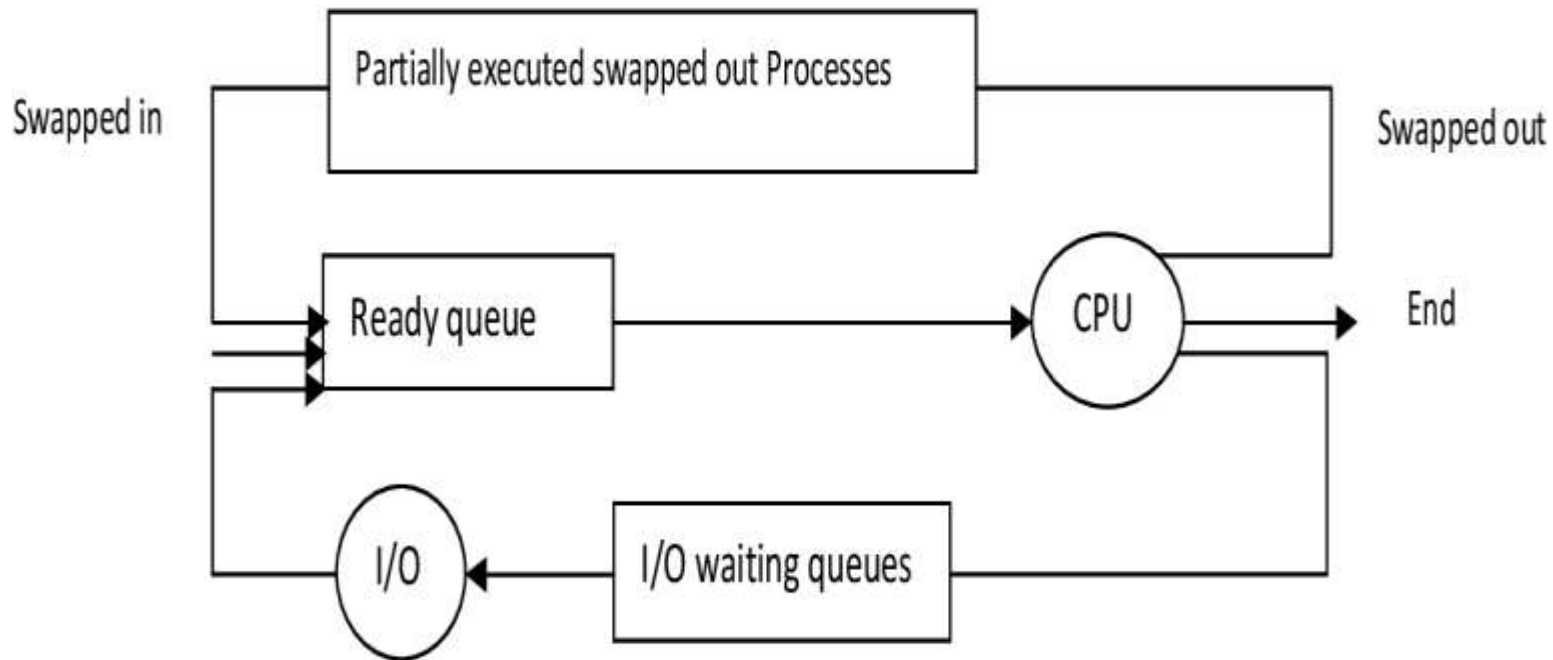
Response Time –

- Amount of time it takes from when a request was submitted until the first response is produced.

Scheduling Algorithms

- First- Come, First-Served (FCFS) Scheduling
- Shortest-Job-First (SJF) Scheduling
- Round Robin (RR)
- Priority Scheduling
- Multilevel Queue
- Multilevel Feedback Queue

Scheduling Algorithms (cont...)



1. First- Come, First-Served (FCFS) Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1, P_2, P_3
- **Gantt Chart** for the schedule is:



- **Waiting time** for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- **Average waiting time:** $(0 + 24 + 27)/3 = 17$

FCFS Scheduling (Cont.)

Suppose that the processes arrive in the order:

$$P_2, P_3, P_1$$

- **Gantt chart** for the schedule is:



- Waiting time for $P_1 = 6$; $P_2 = 0$; $P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- **Much better than previous case**

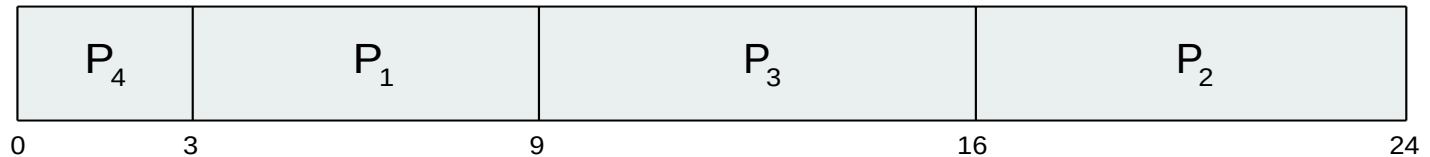
2. Shortest-Job-First (SJF) Scheduling

- In SJF scheduling, the process with the lowest burst time, among the list of available processes in the ready queue, is going to be scheduled next.
- SJF is optimal – gives minimum average waiting time for a given set of processes
- Preemptive version called **shortest-remaining-time-first**

Example of SJF

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

- SJF scheduling chart



- Waiting time: $p_4=0$, $p_1=3$, $p_3=9$, $p_2=16$
- Average waiting time = $(3 + 16 + 9 + 0) / 4 = \mathbf{7_{m/s}}$

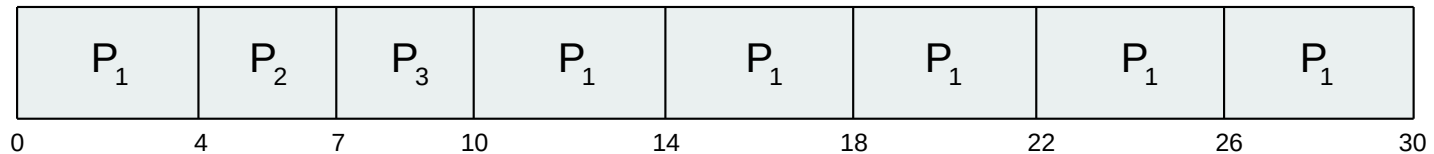
3. Round Robin (RR)

- **Round Robin:** where each process is cyclically assigned a fixed time slot.
- Each process gets a small unit of CPU time (**is called “time quantum q ”**).
- Each process in the ready queue is assigned the CPU for that time quantum, **if the execution of the process is completed** then the process will **end** else the process will go back to the **waiting table** and wait for its next turn to complete the execution.
- Its preemptive.

Example of RR with Time Quantum = 4

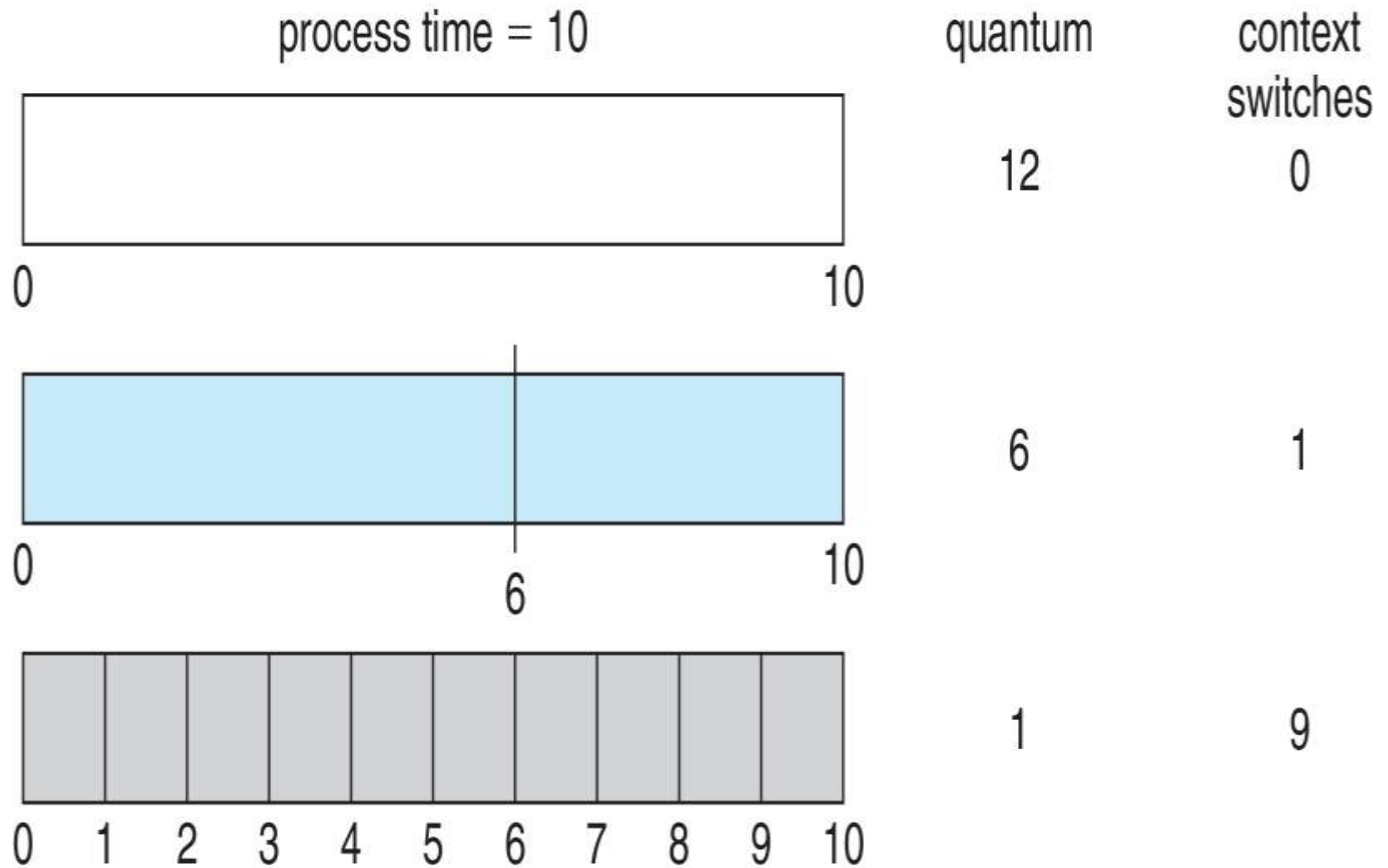
<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- The Gantt chart is:



- Typically, higher average turnaround than SJF, but better **response q** should be large compared to context switch time
 - q usually 10 milliseconds to 100 milliseconds,
 - Context switch < 10 microseconds

Time Quantum and Context Switch Time



4. Priority Scheduling

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer $\equiv$ highest priority)
 - Preemptive
 - Nonpreemptive
- SJF is priority scheduling where priority is the inverse of predicted next CPU burst time
- Problem $\equiv$ **Starvation** – low priority processes may never execute
- Solution $\equiv$ **Aging** – as time progresses increase the priority of the process

Example of Priority Scheduling

<u>Process</u>	<u>Burst Time</u>	<u>Priority</u>
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

- Priority scheduling Gantt Chart

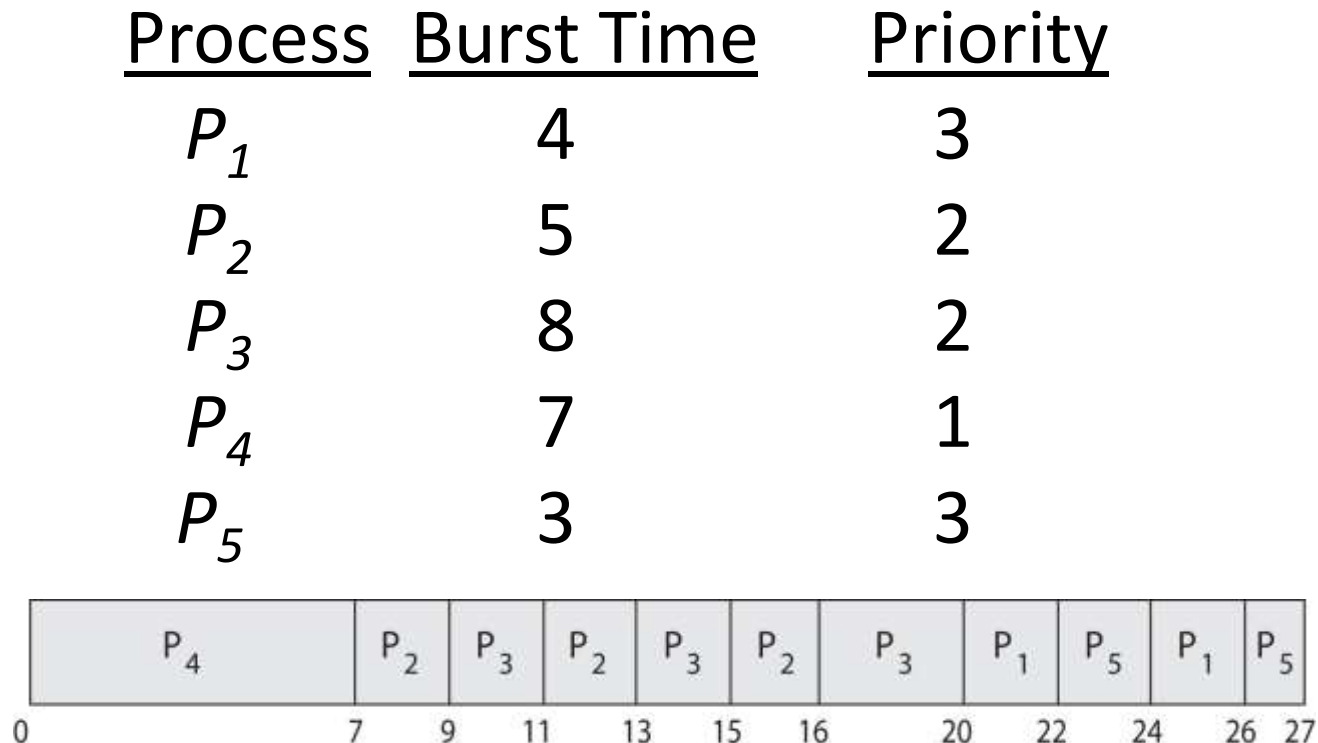


- Waiting time = $p_2=0$, $p_5=1$, $p_1=6$, $p_3=16$, $p_4=18$
- Average waiting time = $(0+1+6+16+18)/5=8.2$

Priority Scheduling with Round-Robin

- Run the process with the highest priority.
- Processes with the same priority run round-robin

Example:



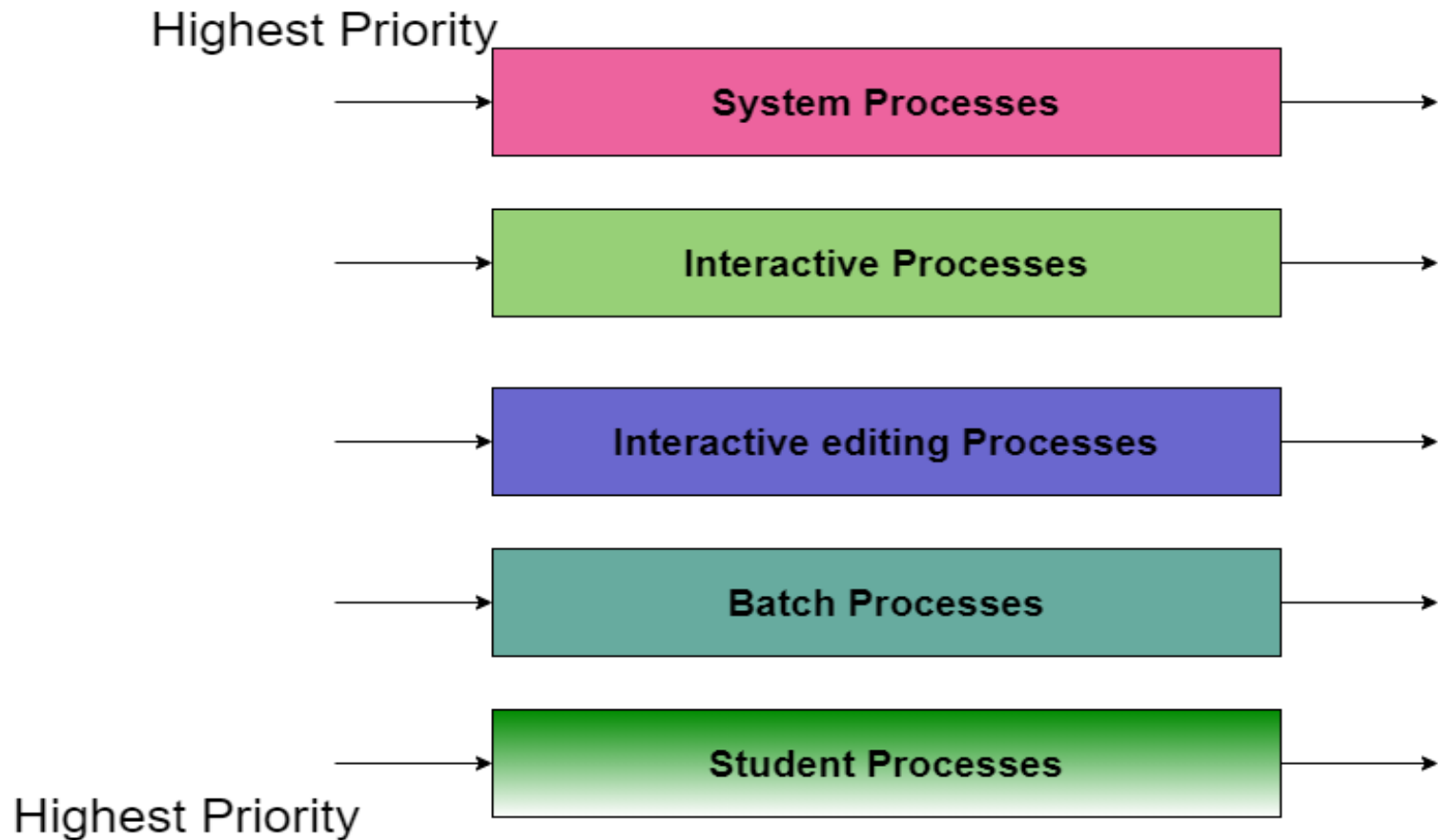
- Gantt Chart with time quantum = 2

5. Multilevel Queue

- The Ready Queue **consists of multiple queues**
- Multilevel queue scheduler defined by the following parameters:
 - Number of queues
 - Scheduling algorithms for each queue
 - Method used to determine which queue a process will enter when that process needs service
 - Scheduling among the queues

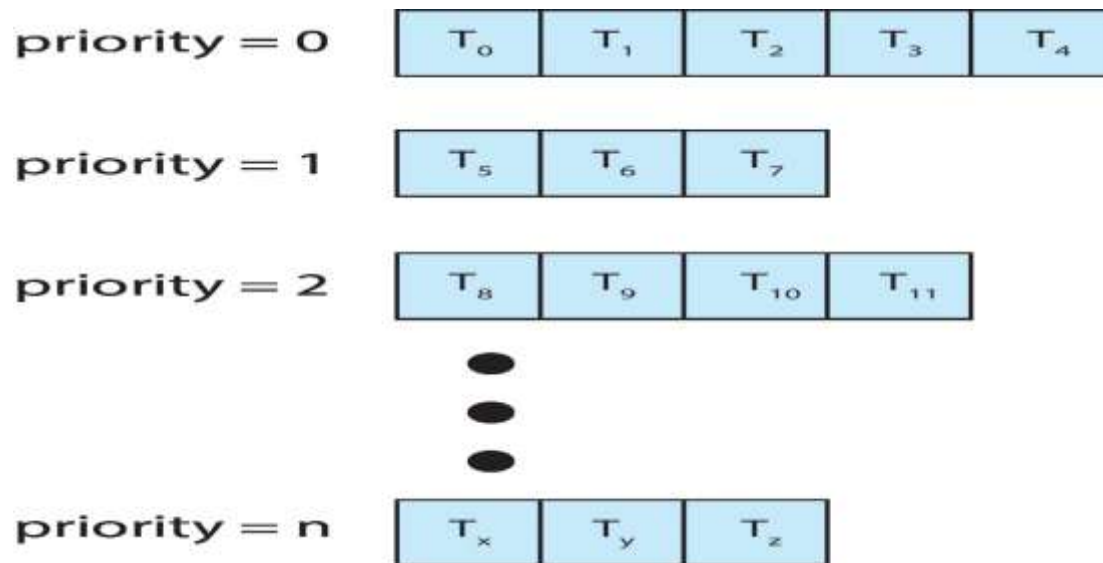
Multilevel Queue(cont...)

- Prioritization based upon process type



Multilevel Queue(cont...)

- With priority scheduling, have separate queues for each priority.
- Schedule the process in the highest-priority queue!



6. Multilevel Feedback Queue



6.Multilevel Feedback Queue

Processes are assigned to different queues with varying priorities and scheduling algorithms.

Example: Multiple queues can be used, with the **highest priority queue using FCFS** and **lower priority queues using Round Robin** with different time slices.

- Processes can move between queues based on their behavior (e.g., CPU-bound or I/O-bound).

Key Feature: Highly flexible and adaptable to different workloads.

Process Synchronization

- Synchronization refers to **the coordination of multiple processes or threads** to ensure they access shared resources in a **controlled manner, preventing conflicts and maintaining data consistency.**
- Synchronization mechanisms used to synchronize the processes,

When two or more process cooperates with each other, their order of execution must be preserved **otherwise conflicts** in their execution and **inappropriate** outputs can be produced.

Process Synchronization (cont...)

- A cooperative process can affect the execution of other process **or** can be affected by the execution of other process.
- Such processes need to be synchronized so that their order of execution can be guaranteed.
- The procedure involved in preserving the appropriate order of execution of cooperative processes is **known as Process Synchronization.**

Critical Section Problem

- Consider system of n processes $\{p_0, p_1, \dots p_{n-1}\}$
- Each process has critical section is code segment.
- Code segment is shared variables..
- Eg: P1, p2, p3 all the process wants to use the common variable... is called as CSP.
 - Process may be changing common variables, updating table, writing file, etc.
 - When one process in critical section, no other may be in its critical section

Critical Section Problem

- **Critical section problem** is to design protocol to solve this..
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**.
- **Entry Section:** Controls the entry into CS and gets LOCK on required resources...
- **Critical section:** code segments
- **Exit section:** removes LOCK an resources and others know that its CS is over.

Critical Section

- General structure of process $P_i(P1, P2, P3...)$

```
while (true) {
```

```
    entry section
```

```
        critical section
```

```
    exit section
```

```
        remainder section
```

```
}
```

Critical Section Problem(cont...)

solution to critical-section problem

- 1. Mutual Exclusion** - If process P_i is executing in its critical section, **then no other processes can be executing in their critical sections**
- 2. Progress** - If no process is executing in its critical section and wish to enter their critical section, **then the selection of the process** that will enter the critical section next cannot be postponed indefinitely.

Critical Section Problem(cont...)

solution to critical-section problem

- 3. Bounded Waiting** - A bound must exist on the number of times that **other processes are allowed to enter their critical sections after a process has made a request to enter its critical section** and before that request is granted.

Issues of critical section

1. Deadlock:

- When two **or** more threads **or** processes wait for each other to release a critical section, It can result in a deadlock situation.
- Deadlocks can be **difficult to detect and resolve**, and its significant impact on a program's performance and reliability.

Issues of critical section

2. Starvation:

- When a thread or process is **repeatedly prevented from entering a critical section**, So, thread or process is unable to progress.
- This can happen **if the critical section is held for an unusually long period of time**, or if a high-priority thread or process is **always given priority when entering the critical section**.

Issues of critical section

3. Overhead:

- Overhead may **reduce the program's overall performance.**
- When using critical sections, **threads or processes must acquire and release locks or semaphores, which can take time and resources.**

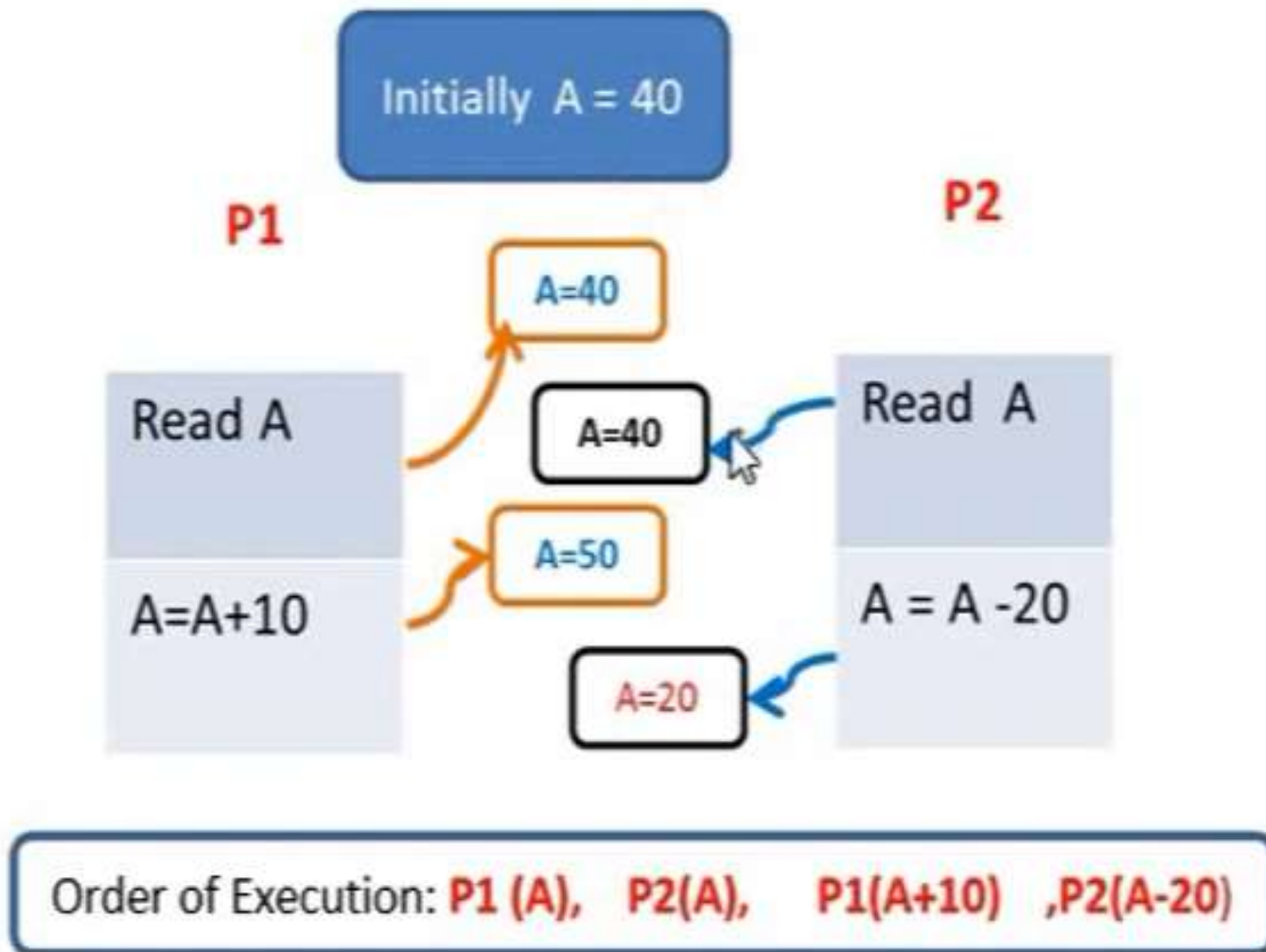
Race Condition

Race Condition is a situation in which **two or more processes are reading or writing shared variable (shared resource)**, and the result depends on the timing of how the processes are scheduled.

AIM: the results of a **process** should never be affected by other process **“Winning a Race”..**

- If both the processes can potentially write a variable at a given time, that situation is **called deadlock**.
- One process can potentially write a variable while another reads the variable.

Race Condition cont...



Race Condition cont...

Race Condition: unpredictable results, (prevent by locking method- synchronized method).

Race conditions in software: Air Traffic Signal, Railway Signaling, Multithreaded environment in JAVA etc..

Race Condition may occur in Critical Section..

Peterson's Solution

- Peterson's solution is **software-based solution to the critical section problem..**
- It provides a good algorithm to give solution for two process...
- Assume that ML instructions **LOAD** and **STORE** are atomic; that is cannot be interrupted..
- The two processes **P0** and **P1** share two variables:
 - **int** **turn**;
 - **boolean** **flag[2]**;

Peterson's Problem

A software solution to Critical Section Problem

```
while (true) {
```

entry section

critical section

exit section

remainder section

```
}
```

Process P_i, P_j

int turn;

boolean flag[2];

Structure of process P_i

do

{

flag[i] = TRUE;

turn = j;

While(flag[j] && turn == j);

critical section

flag[i] = FALSE;

Remainder section

}while (TRUE);

Peterson's Solution

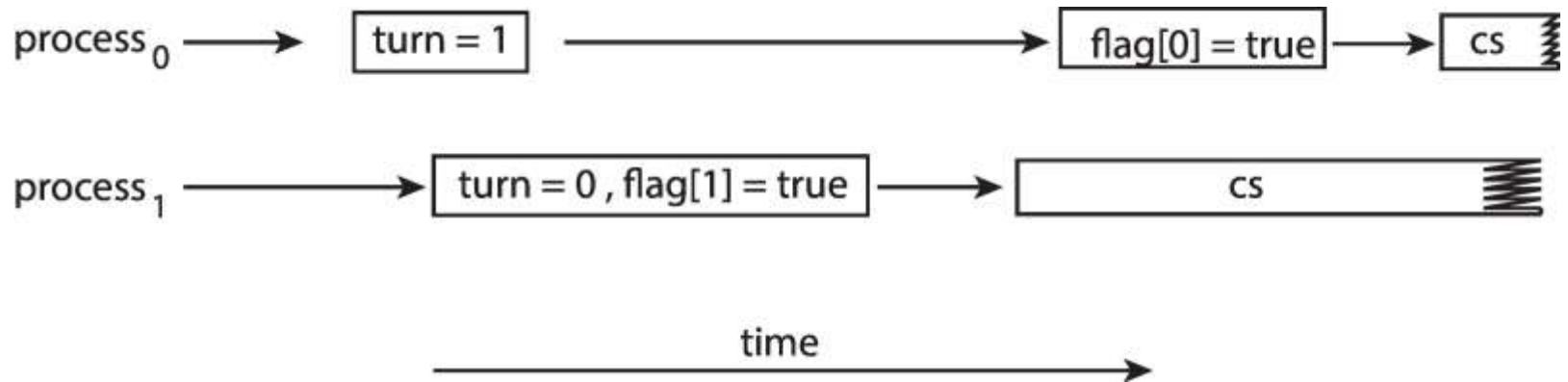
1. The variable **turn** indicates **whose turn** is to enter the critical section. **(if $\text{turn}=i$, then P_i will enter in the critical section)**
2. The **flag** array is used to indicate **if ready to enter the critical section**.
 - **Flag[i]= true** implies that process **P_i** is ready!
 - (The peterson's algorithm is an humble algorithm, **P_i** will set **$\text{turn}=j$** and **P_j** will set **$\text{turn}=i$**)

Peterson's Solution

- Peterson's solution preserves all the three conditions:
- **Mutual Exclusion:** is assured only one process can access the critical section at any time.
- **Progress:** is also assured, as process outside the CS does not blocks others processes from entering the CS.
- **Bounded Waiting:** is preserved as every gets a fair chance.

Peterson's Solution Revisited

- The effects of instruction reordering in Peterson's Solution



- This allows both processes to be in their critical section at the same time!
- To ensure that Peterson's solution will work correctly on modern computer architecture we must use **Memory Barrier**.

Semaphore

- Semaphore: A **integer variable** to synchronize the progress of interacting processes.
- Two standard operations, accessed **via** two atomic operations

1. **wait()**

2. **signal()**

- both operation maintain the busy waiting.
- Without waiting BUSY waiting.
- **Entry** to the critical section is controlled by **wait** operation.
- **Exit** from a critical region is taken care by **Signal** operation.

Wait and Signal Operations

1. wait () operation

- If argument **S** is zero, then no operation performed.
- If argument **S** is positive, then it decrements the value.

Example :

```
wait(S)
{
    while (S <= 0)
        // busy wait
        // ie) doesn't operation;
        S--;
}
```

do {

entry section

critical section

exit section

remainder section

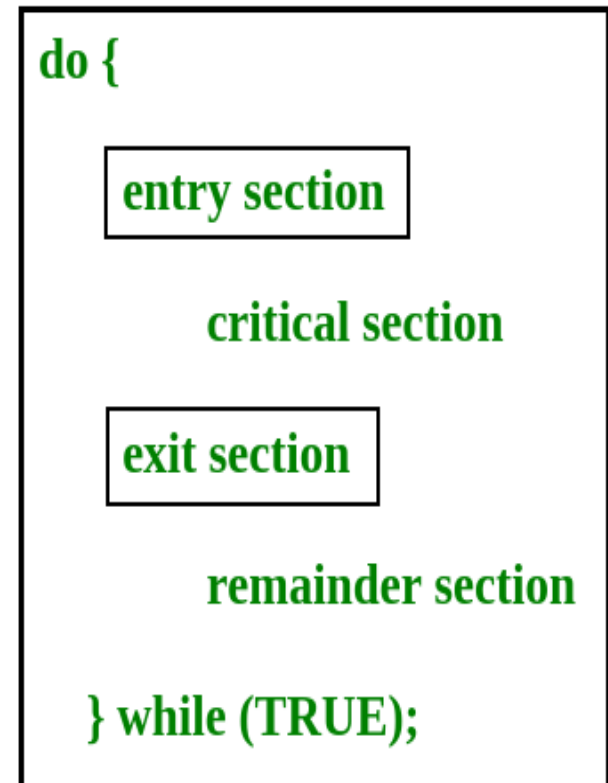
} while (TRUE);

2. The **signal () operation** increments the value of its arguments S.

- If argument S is positive, then it decrements the value.

signal() operation

```
signal (S)
{
    S++;
}
```



Semaphore Usage

Two synchronization tools

1. **Counting semaphore** – integer value can range over an unrestricted domain.
2. **Binary semaphore** – integer value can range only between 0 and 1
 - Same as a **mutex lock**
 - Can implement a counting semaphore S as a binary semaphore.
 - With semaphores we can solve various synchronization problems

Semaphore Implementation

- There are 2 type of variable
 - 1. Wait
 - 2. Signal
- These two variable used to solve CSP's.
- To acheive process Synchronization in multi processing environment...
- Doesn't require BUSY WAITING.
- Must guarantee that NO two processes can execute the **wait()** and **signal()** on the same semaphore at the same time.
- Thus, the implementation becomes the critical section problem where the **wait** and **signal** code are placed in the critical section.

Process1

- The common variable S
- Initially S = 1.

Definition of P(S) or wait(S):

```
wait(S)
{
    while (S <= 0); // busy wait
    S--;
```

Definition of V(S) or signal(S):

```
signal(S)
{
    S++;
```

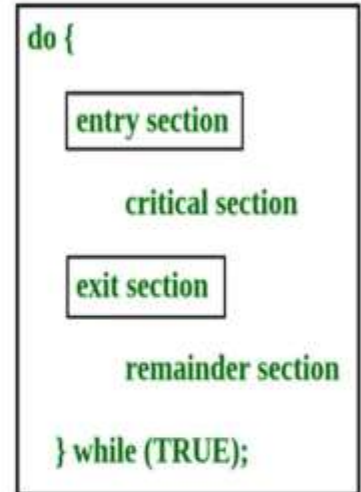
Process2

Definition of P(S) or wait(S):

```
wait(S)
{
    while (S <= 0); // busy wait
    S--;
```

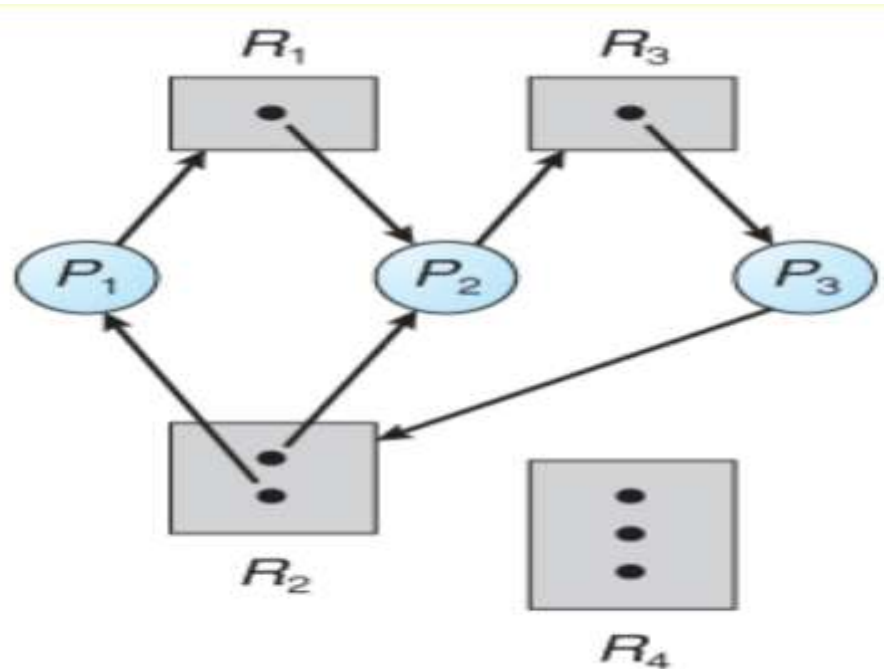
Definition of V(S) or signal(S):

```
signal(S)
{
    S++;
```



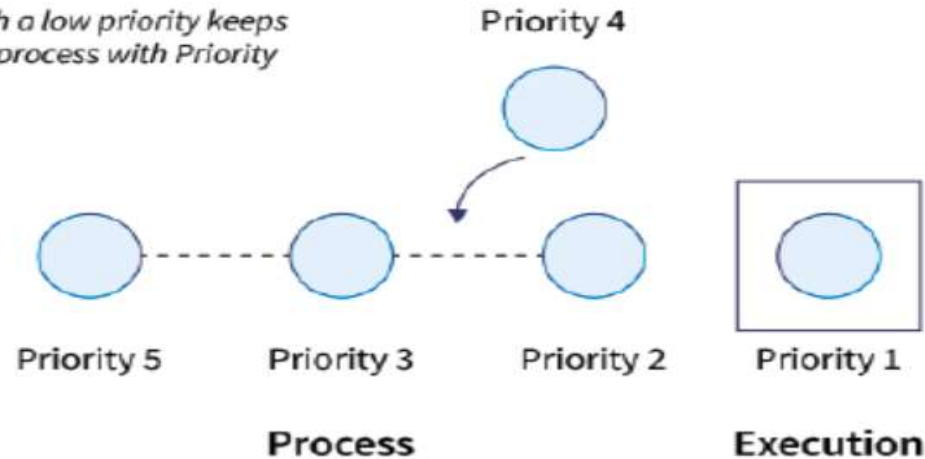
Deadlock:

Deadlock may occur when two **or** more processes try to get the same multiple resources at a same time.



Starvation:

If new process with a low priority keeps on coming then a process with Priority 5 will be starved



- Starvation is a problem of resource management, the process does not have resources because it is being used by other processes.
- This problem occurs mainly in a **priority-based scheduling algorithm** in which the requests with high priority get processed first and **the low priority processes get blocked for indefinite time.**

Semaphore Implementation **with no Busy waiting**

- With each semaphore there is an associated waiting queue

Each entry in a waiting queue has two data items:

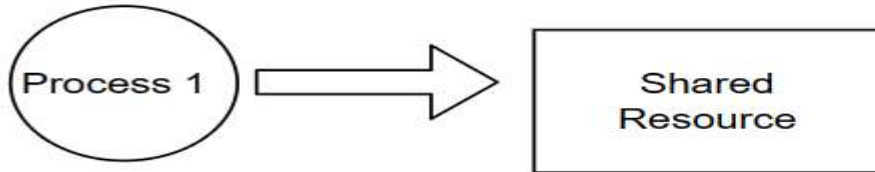
- Value (integer)
- Pointer to next record in the list

Two operations:

- **block** – place the process invoking the operation on the appropriate waiting queue
- **wakeup** – remove one of processes in the waiting queue and place it in the ready queue.

Semaphore Implementation **with Busy waiting**

1. Process 1 is using the shared resource.



2. Process 2 is now in need of the shared resources.



3. Process 2 enters busy waiting till it has access to the shared resource.



Implementation with **no Busy waiting** (Cont.)

```
wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

Readers-Writers Problem

- **A data set** is shared among a number of concurrent processes.
 - **Readers** : only read the data set; they do **not** perform any updates
 - **Writers** : can both read and write
- **Problem** – allow multiple readers to read at the same time.
 - Only one single writer can access the shared data at the same time.

Readers-Writers Problem (cont...)

- Shared Data

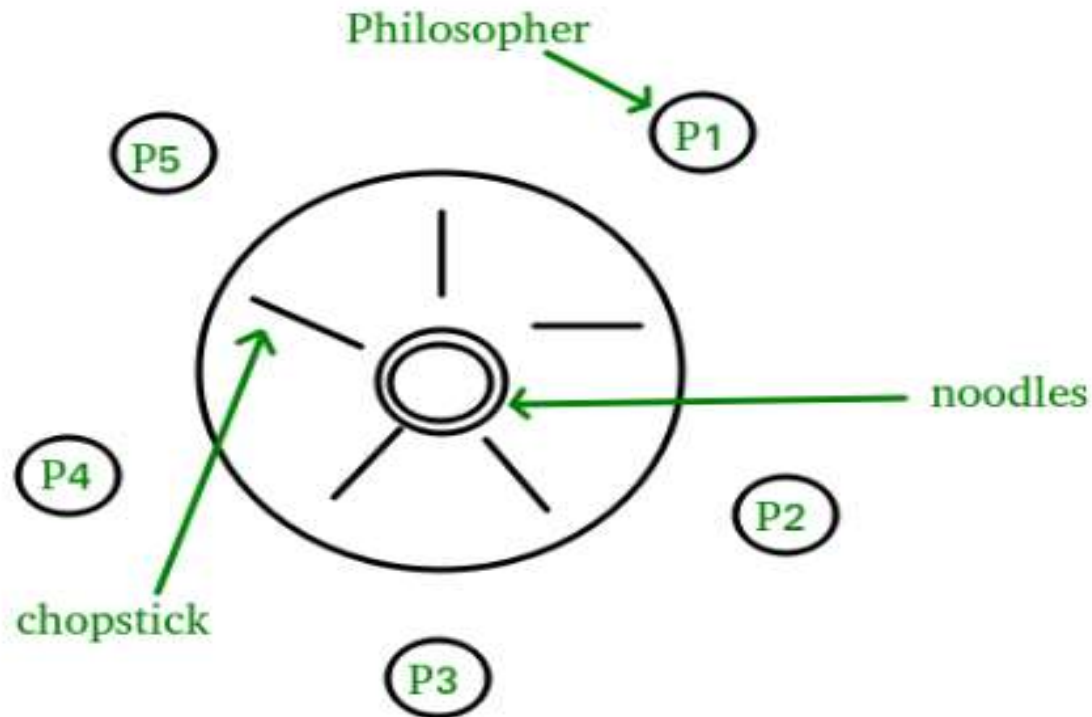
- Data set
- Semaphore **mutex** initialized to 1 (controls access to readcount)
- Semaphore **wrt** initialized to 1 (writer access)
- Integer **readcount** initialized to 0 (how many processes are reading object)

Readers-Writers Problem(cont..)

- The structure of a writer process

```
do {  
    wait (wrt) ;  
    //  writing is performed  
    signal (wrt) ;  
} while (TRUE);
```

Dining-Philosophers Problem



- Shared data
 - Bowl of rice (data set)
 - Semaphore **chopstick** [5] initialized to 1

Dining-Philosophers Problem

- The K philosophers are seated around a circular table with one chopstick between each pair of philosophers.
- There is **one chopstick between each philosopher**.
- A philosopher may eat if he can pick up the two chopsticks adjacent to him.
- One chopstick may be picked up by any one of its adjacent followers but not both.

Dining-Philosophers Problem

- There are two states of the philosopher: **THINKING** and **EATING**.
- **Here two semaphores:** Mutex and a semaphore array for the philosophers.
- **Mutex** is used such that no two philosophers may access the pickup **or** put it down at the same time.
- **Semaphore array** is used to control the behavior of each philosopher.
- **But**, semaphores can result in deadlock due to programming errors.

Dining-Philosophers Problem

Solution

- **Mutual Exclusion Principle:** No two Philosophers can have the two forks simultaneously.
- **Free from Deadlock:** Each philosopher can get the chance to eat in a certain finite time.
- **Free from Starvation:** When few Philosophers are waiting then one gets a chance to eat in a while.
- **No strict Alternation**
- **Proper utilization of time**

```
Void Philosopher {  
    while(1) {  
        take_chopstick[i];  
        take_chopstick[(i+1) % 5] ;  
        ... // EAT ...  
        put_chopstick[i];  
        put_chopstick[(i+1) % 5] ;  
        ... // THINKING ..  
    }  
}
```

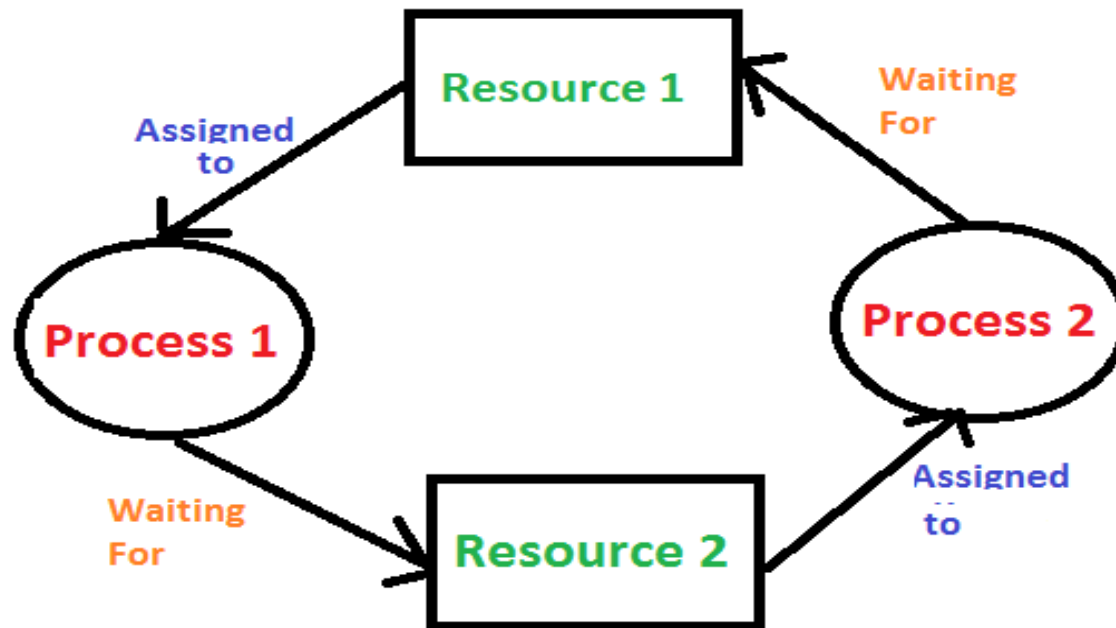
Deadlock

- ***Deadlock*** is a situation where a set of processes are blocked because each process is **holding a resource and waiting for another resource** acquired by some other process.
- A situation occurs in operating systems when there are **two or more processes that hold some resources and wait for resources** held by other(s).

Deadlock

For example,

- Process 1 is holding Resource 1 and waiting for resource 2 which is acquired by process 2, and process 2 is waiting for resource 1.



Deadlock Characterization

1. Necessary Condition
2. Resource -Allocation Graph

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

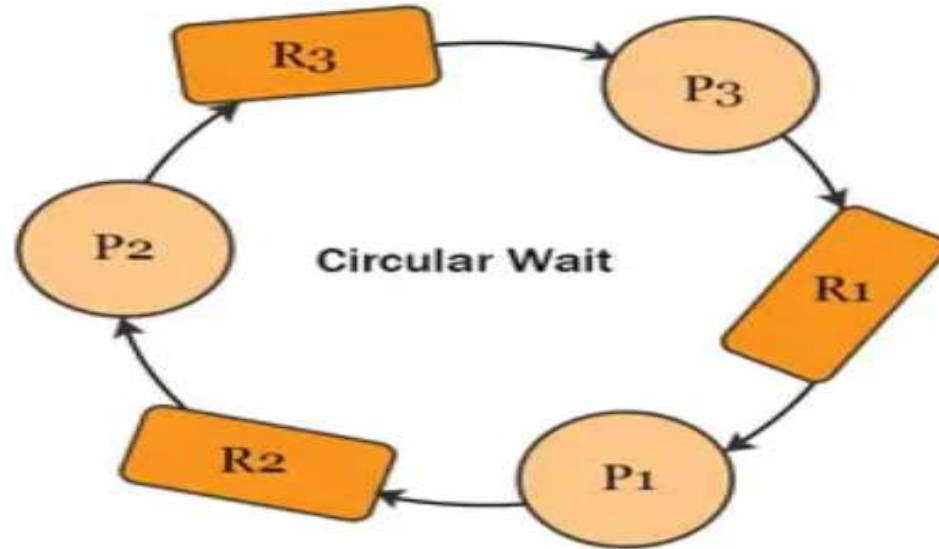
1. Necessary Condition

- **Mutual exclusion**: only one thread at a time can use a resource(not sharable).
- **Hold and wait**: a thread **holding at least one** resource is waiting **to acquire additional** resources held by other threads.
- **No preemption**: a resource can be released only voluntarily by the thread holding it, after that thread has completed its task.

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- Circular wait:



Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- Circular wait:

A set $\{T_0, T_1, \dots, T_n\}$ of waiting threads such that

T_0 is waiting for a resource that is held by T_1 ,

T_1 is waiting for a resource that is held by $T_2, \dots$,

T_{n-1} is waiting for a resource that is held by T_n ,

and

T_n is waiting for a resource that is held by T_0 .

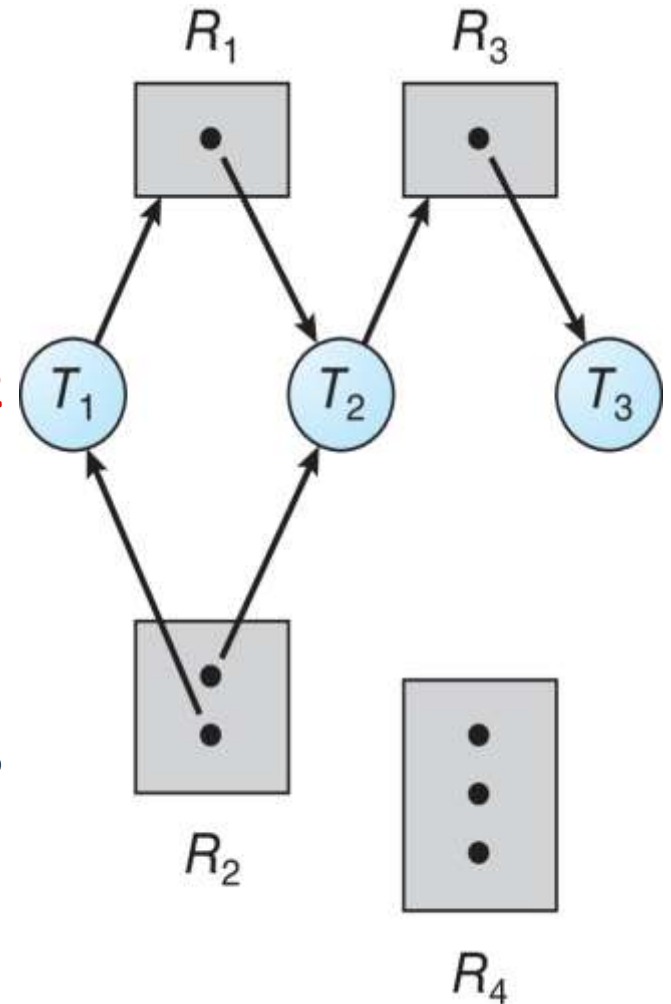
2. Resource-Allocation Graph

A set of vertices V and a set of edges E .

- V is partitioned into two types:
 - $T = \{T_1, T_2, \dots, T_n\}$, the set consisting of all the threads in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- Request edge – directed edge $T_i \rightarrow R_j$
- Assignment edge – directed edge $R_j \rightarrow T_i$

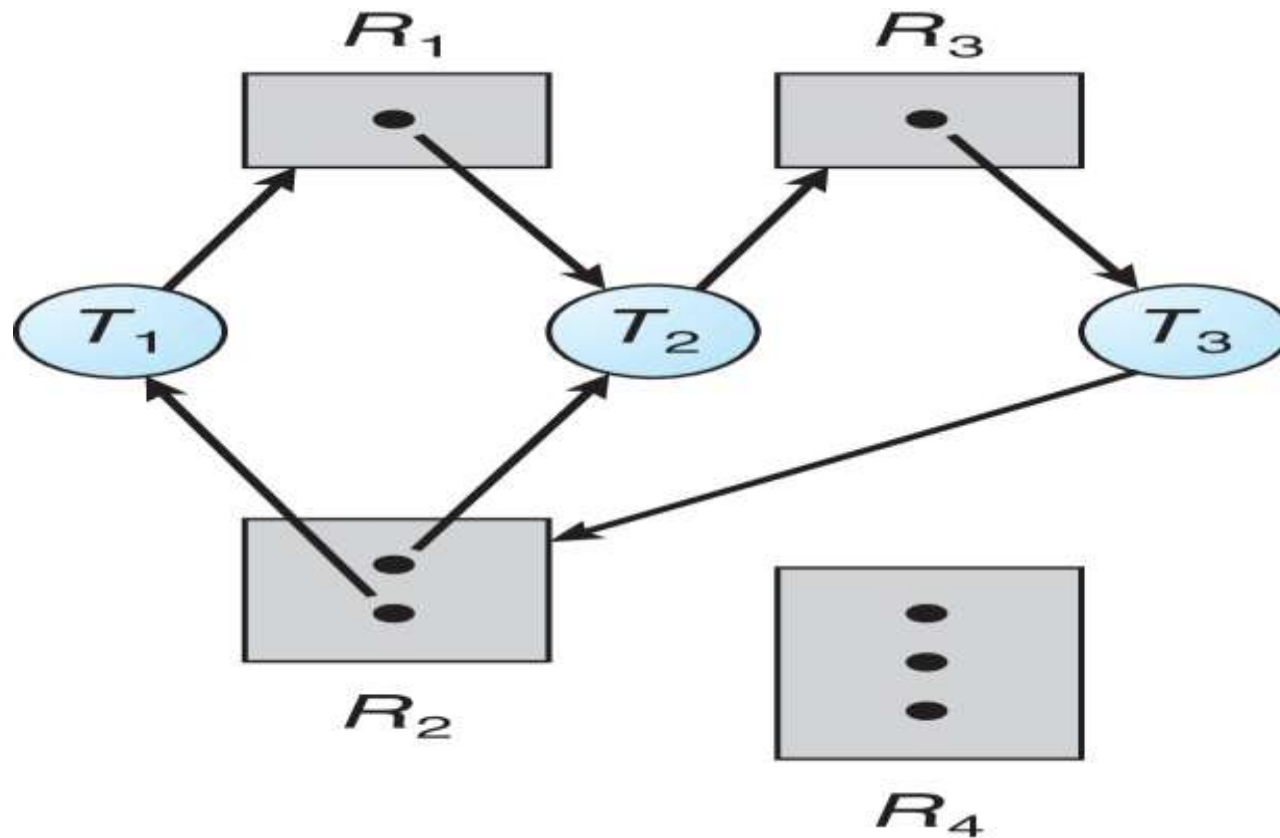
Resource Allocation Graph Example

- **One** instance of R_1
- **Two** instances of R_2
- **One** instance of R_3
- **Three** instance of R_4
- T_1 holds one instance of R_2 and is waiting for an instance of R_1
- T_2 holds one instance of R_1 , one instance of R_2 , and is waiting for an instance of R_3
- T_3 is holds one instance of R_3

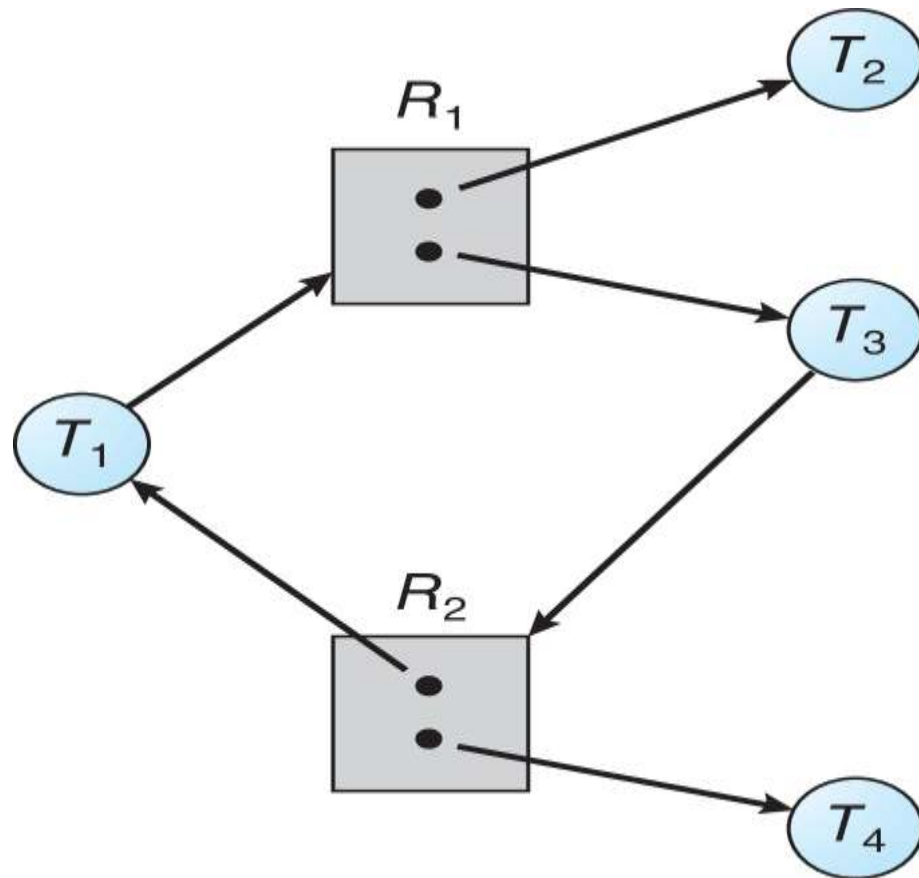


Resource Allocation Graph with a Deadlock

- $P1 \rightarrow R1 \rightarrow P2 \rightarrow R3 \rightarrow P3 \rightarrow R2 \rightarrow P1$



Graph with a Cycle But no Deadlock



Basic Facts

- ❑ If graph contains **no cycles** $\Rightarrow$ **no deadlock**
- ❑ If graph contains a cycle $\Rightarrow$
 - if **only one instance** per resource type, then deadlock
 - if **several instances** per resource type, possibility of deadlock

Methods for Handling Deadlocks

1. Ensure that the system will **never** enter a deadlock state:
 - **Deadlock prevention**
 - **Deadlock avoidance**
2. Allow the system to enter a deadlock state and then recover
3. Ignore the problem and pretend that deadlocks never occur in the system.

Deadlock Prevention

Deadlock prevention algorithms ensure that **at least one of the necessary condition** (*Mutual Exclusion, Hold and wait, No preemption and circular wait*) does not hold true.

Deadlock Prevention Deadlocks can be prevented by preventing **at least one of the four required condition.**

Deadlock Prevention

1. *Mutual Exclusion:* → *Sharable*

(more process trying to access single resource)

- Shared resources such as **read only file** do not lead to deadlocks.
- Unfortunately, some resources, such as printer and tap drives, require exclusive access by single process.

Deadlock Prevention (Cont.)

2. Hold and wait:

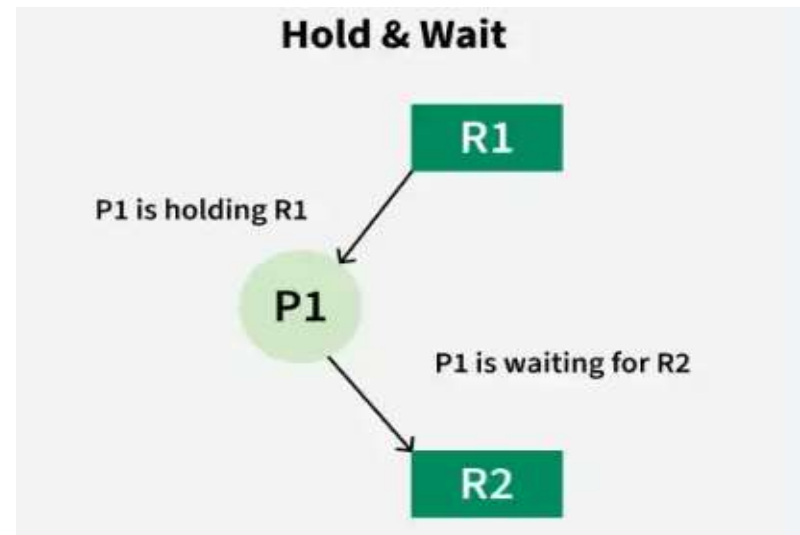
- Allocate all required resources to the process before the start of its execution this way Hold and Wait condition is eliminated.

Example: printer

- Allows process to request resources only when the process has None.

Disadvantages:

- Low Resource Utilization
- Starvation



Deadlock Prevention (Cont.)

3. No Preemption:

- if a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are Released.
- if a process requests resources, check if resource are available.
- if not available, allocated to some other process that is waiting (Forced).

Deadlock Prevention (Cont.)

4.Circular Wait:

- Assign numerical value to resources.
- One way to avoid Circular Wait is to number all the resources, and to require that processes request resources only in strictly (increasing or decreasing) order.

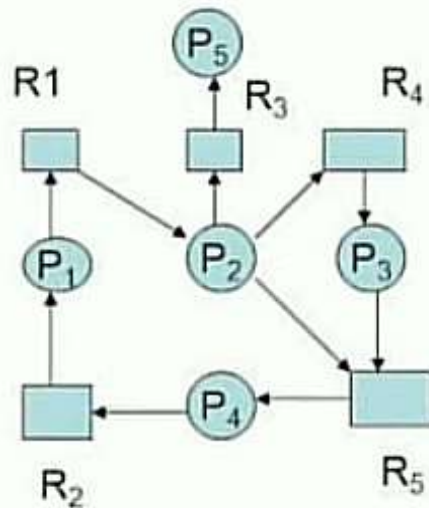
Printer → 5

Disk → 7

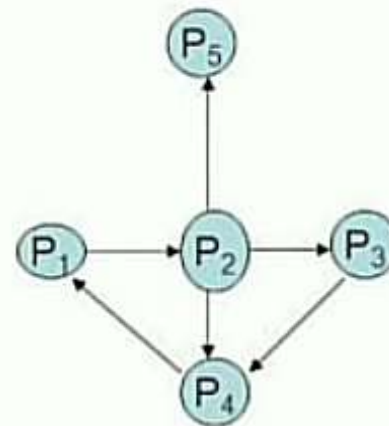
Scanner → 11

Deadlock Detection and Recovery

- ❑ The OS doesn't apply any mechanism to avoid or prevent the deadlocks.
- ❑ Therefore the system considers that the **deadlock will definitely occur.**

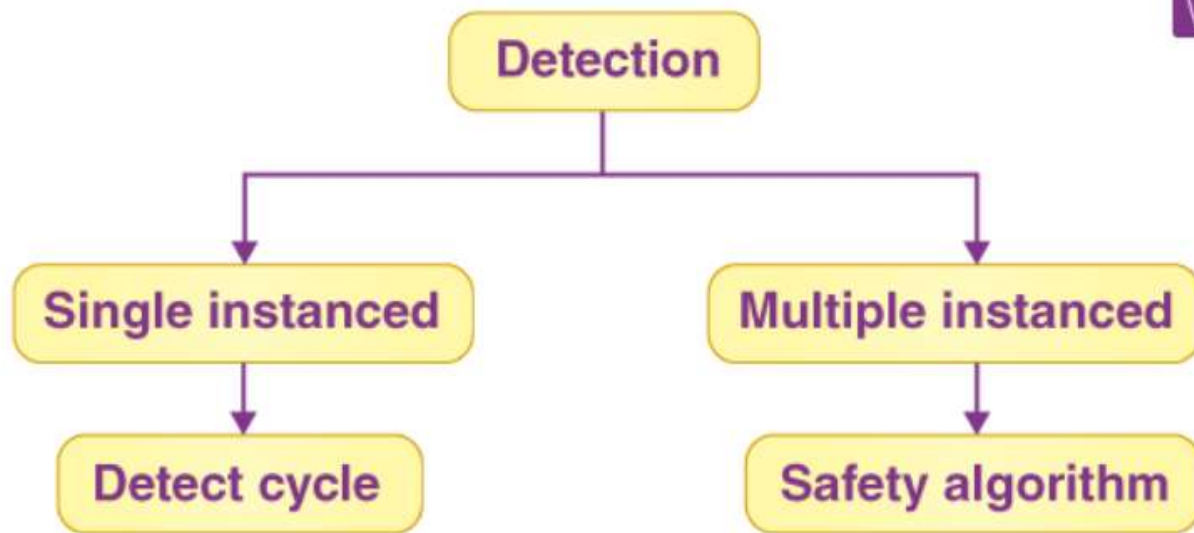


Resource allocation graph



Corresponding wait for Graph

- ❑ In case, it finds any of the deadlock then the OS will recover the system using some recovery techniques.
- ❑ The main task of the OS is detecting the deadlocks. The OS can detect the deadlocks with the help of Resource allocation graph.



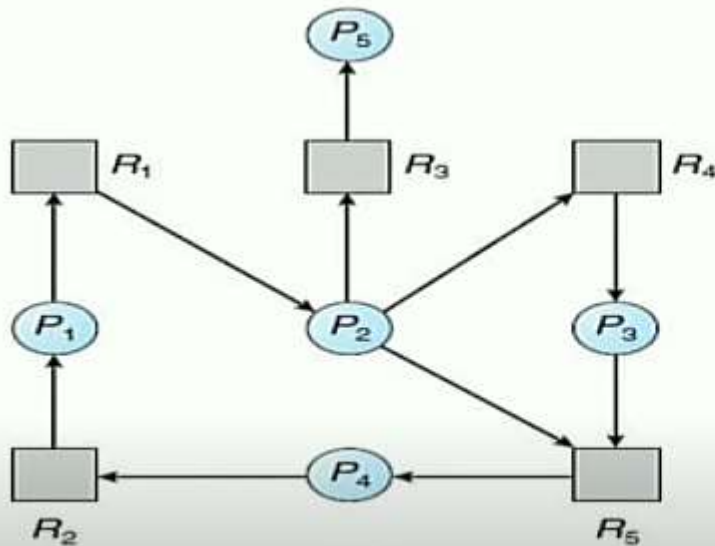
Single Instance of Each Resource Type

If all resources consume only a single instance, then we can delimit a deadlock detection algorithm that uses a variant of the RAG, called as **wait-for graph**.

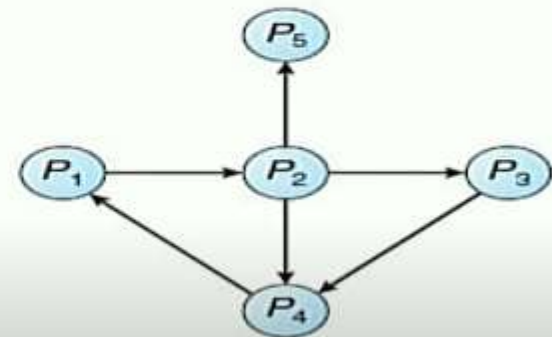
Nodes that are processes

$P_i \rightarrow P_j$ if P_i is waiting for P_j

- An edge from P_i to P_j in a wait-for graph indicates that process P_i is waiting for process P_j to issue a resource that P_i needs

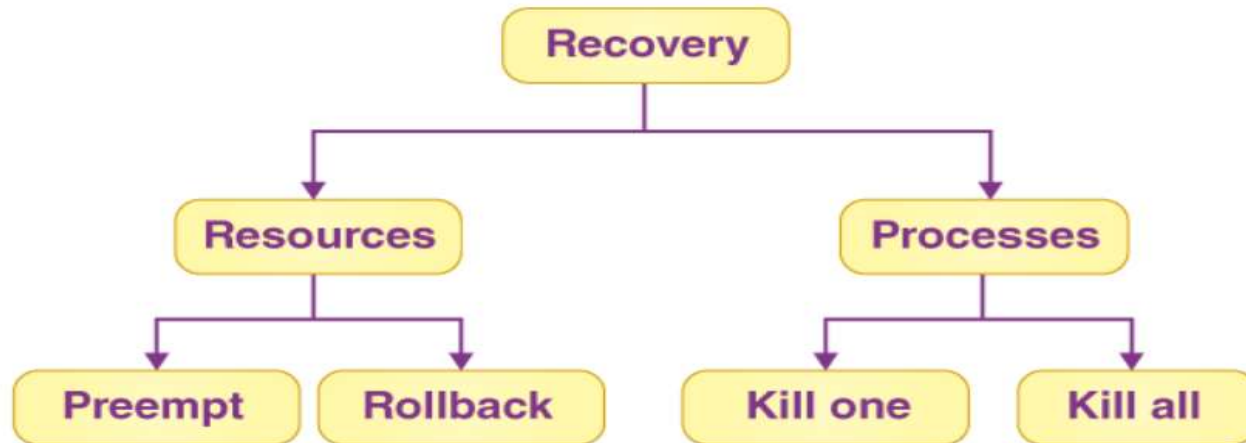


(a)



(b)

Deadlock Recovery



1. Preempt the Resource:

- We can **snatch** one of the resources from the owner of the resource (process) and give it to the other process with the expectation that it will complete the execution and will release this resource sooner.

2. Rollback to a Safe State

- System passes through various states to get into the deadlock state.
- The operating system can rollback the system to the previous safe state.
- For this purpose, OS needs to implement check pointing at every state.
- The moment, we get into deadlock, we will rollback all the allocations to get into the previous safe state.

1. Kill a process

- Killing a process can solve our problem but the bigger concern is to decide which process to kill.
- Generally, Operating system kills a process which has done least amount of work until now.

2. Kill all process

- This is **not a suggestible approach** but can be implemented if the **problem becomes very serious**.
- Killing all process will lead to inefficiency in the system because all the **processes will execute again from starting**.

Deadlock Avoidance

1. Safe State
2. Resource-Allocation-Graph Algorithm
3. Banker's Algorithm

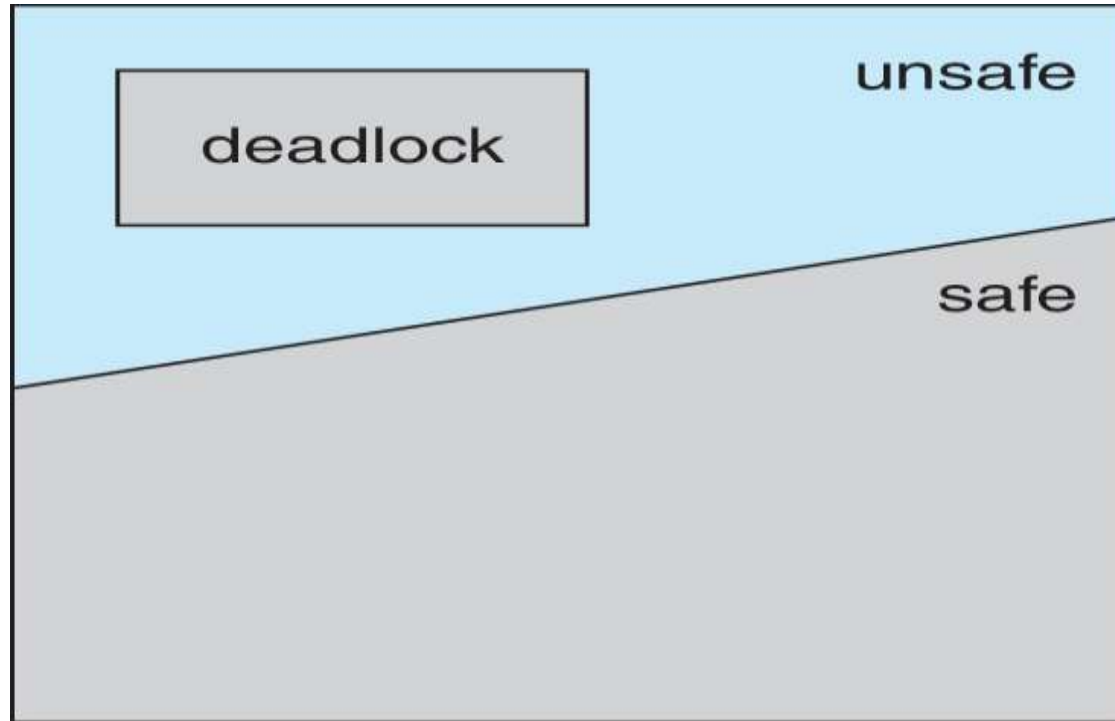
Deadlock Avoidance

Deadlock avoidance in OS is a strategy to prevent deadlocks by carefully **managing resource allocation**, ensuring the **system always remains in a safe state**.

Basic Facts

- **If a system is in safe state** $\Rightarrow$ no deadlocks.
- **If a system is in unsafe state** $\Rightarrow$ possibility of deadlock.
- **Avoidance** $\Rightarrow$ ensure that a system will never enter an unsafe state.

Safe, Unsafe, Deadlock State



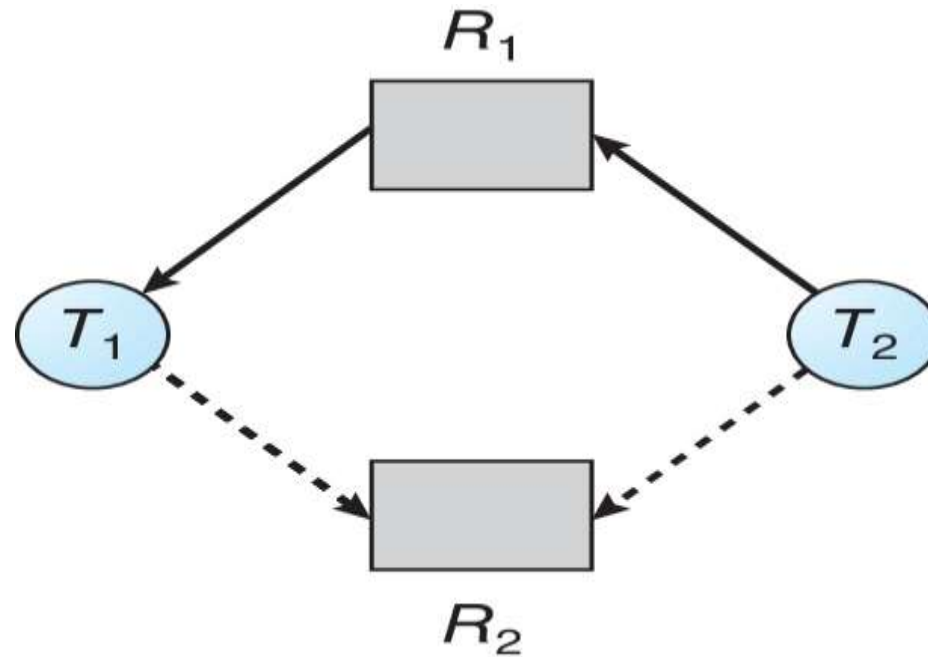
Key Concepts in Banker's Algorithm

- **Safe State:** There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and allow other processes to eventually complete without entering a deadlock.
- **Unsafe State:** Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.

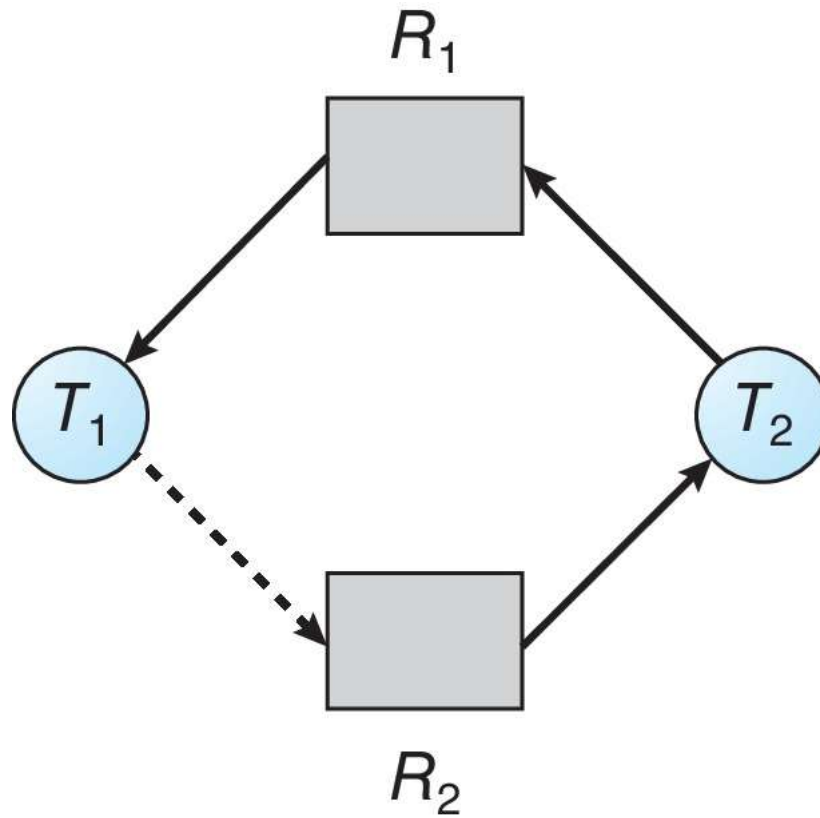
Avoidance Algorithms

- Single instance of a resource type
 - Use a Resource-Allocation Graph
- Multiple instances of a resource type
 - Use the Banker's Algorithm

Resource-Allocation Graph



Unsafe State In Resource-Allocation Graph



3. Banker's Algorithm

- Multiple instances of resources
- Each thread must a priori claim maximum use
- When a thread requests a resource, it may have to wait
- When a thread gets all its resources it must return them in a finite amount of time

Data Structures for the Banker's Algorithm

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i,j] = k$, then process T_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i,j] = k$ then T_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i,j] = k$, then T_i may need k more instances of R_j to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Example of Banker's Algorithm

Example: Considering a system with five processes P_0 through P_4 and three resources of type A, B, C. Resource type A has 10 instances, B has 5 instances and type C has 7 instances. Suppose at time t_0 following snapshot of the system has been taken:

Example (Cont.)

Process	Allocation	Max	Available
	A B C	A B C	A B C
P ₀	0 1 0	7 5 3	3 3 2
P ₁	2 0 0	3 2 2	
P ₂	3 0 2	9 0 2	
P ₃	2 1 1	2 2 2	
P ₄	0 0 2	4 3 3	

Q.1 What will be the content of the Need matrix?

$\text{Need}[i, j] = \text{Max}[i, j] - \text{Allocation}[i, j]$

So, the content of Need Matrix is:

Process	Need		
	A	B	C
P ₀	7	4	3
P ₁	1	2	2
P ₂	6	0	0
P ₃	0	1	1
P ₄	4	3	1

UNIT-2

Operating Systems & System Programming

(23ACA06)

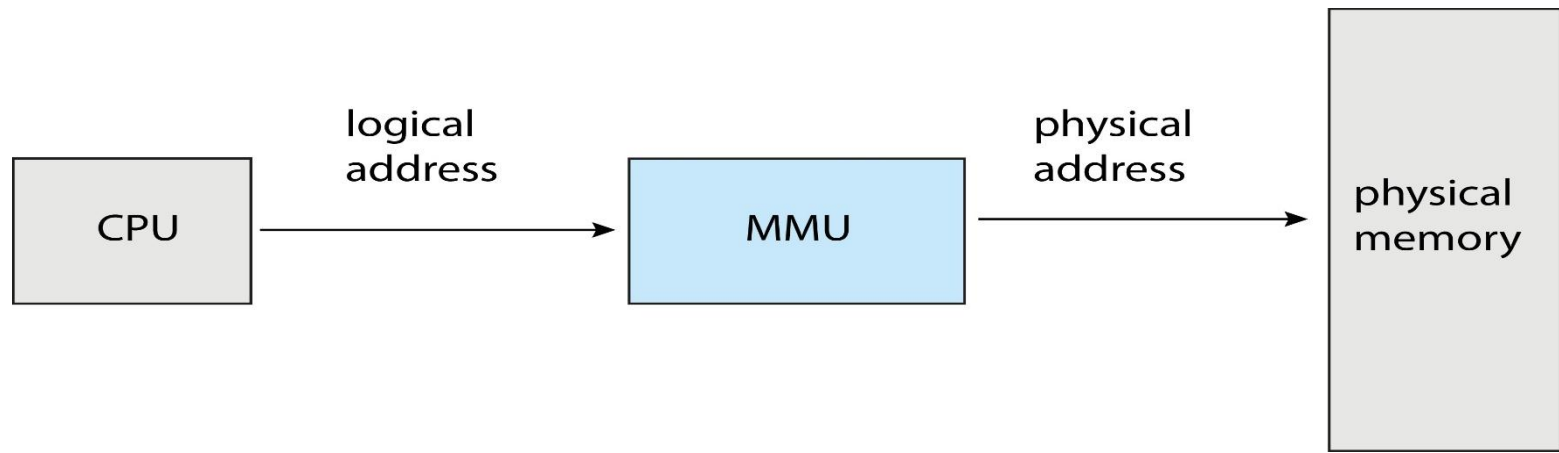
Memory Management

- Memory Management in OS is the process of controlling and coordinating the computers main memory to efficiently allocate and deallocate memory space for program and process.
- That is MM ensures efficient use of the computer's memory resources.

Logical vs Physical Address Space

Logical address space is generated by the CPU during Program Execution and also referred as Virtual address.

- The logical space is the set of all logical address generated by the program.

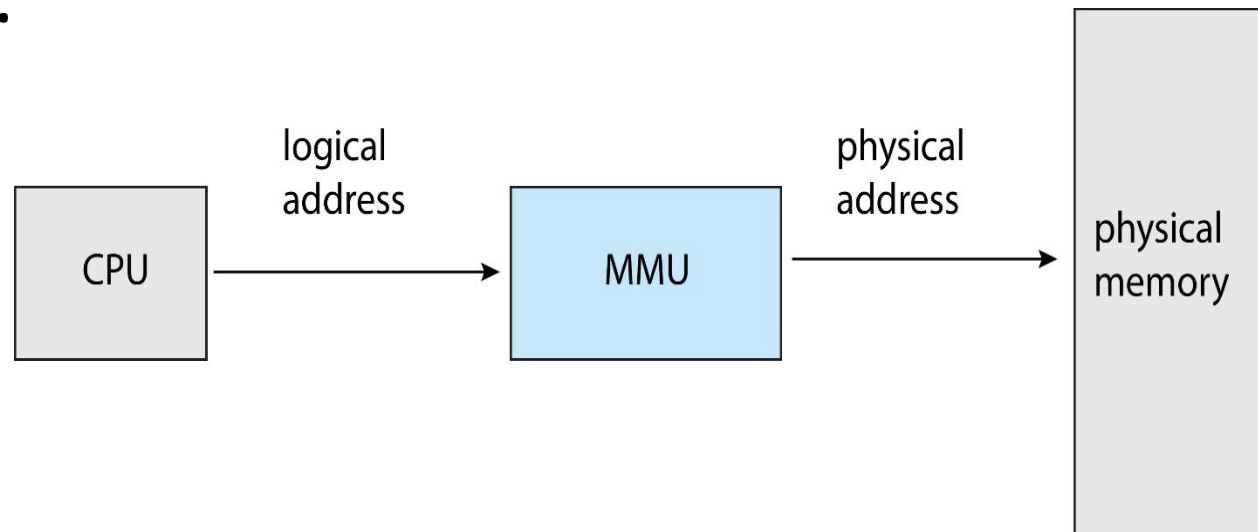


Physical address Space

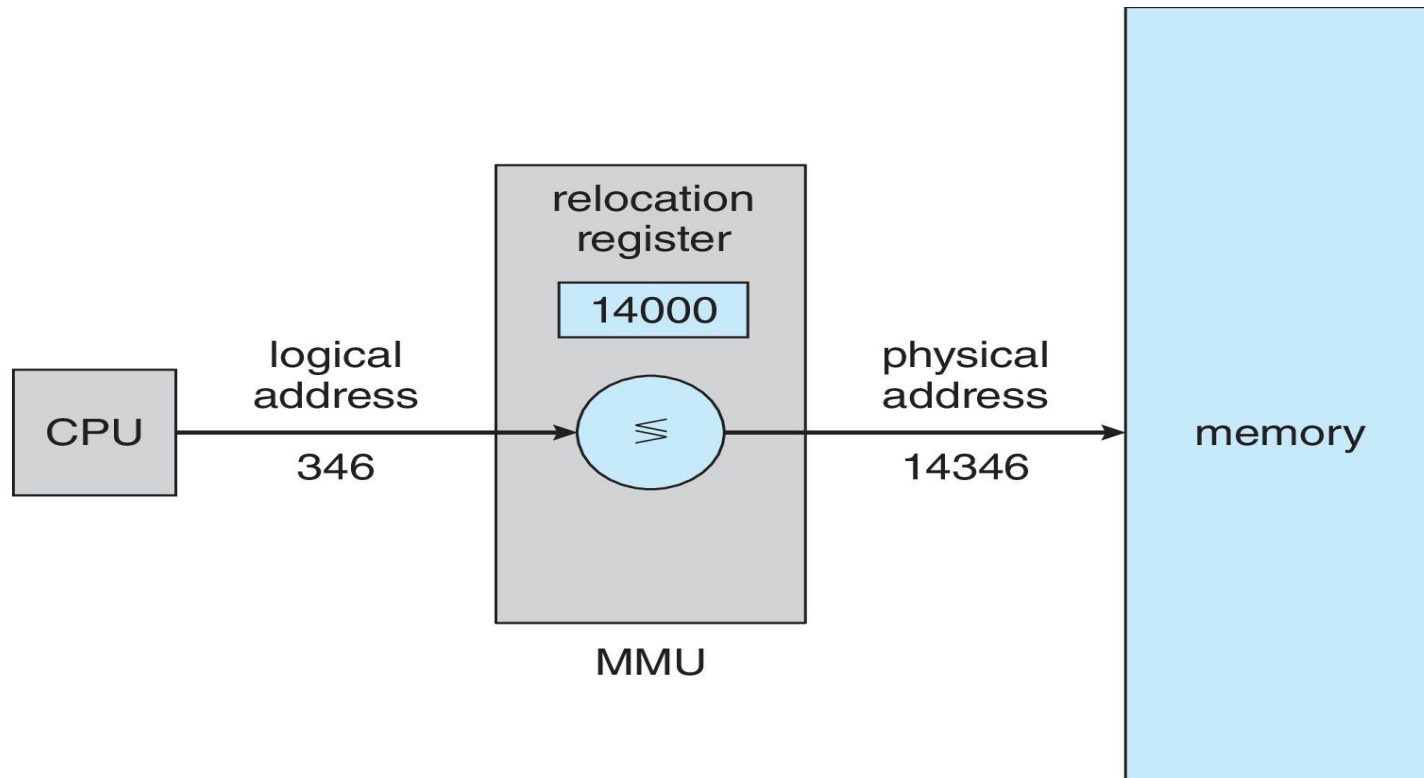
- The physical address is the actual address in the main memory where data is stored and It's a location in physical memory.
- The physical addresses are **used by MMU** to translate logical address into physical addresses.
- Logical Address & Physical Address are the **same** in compile time.
- Logical address & physical address are the **different** in run time address binding scheme.

Memory-Management Unit (MMU)

- Its hardware device that maps virtual to physical address.
- In MMU scheme, the value in relocation register is added to every address generated by the user process at the time it's sent to memory.



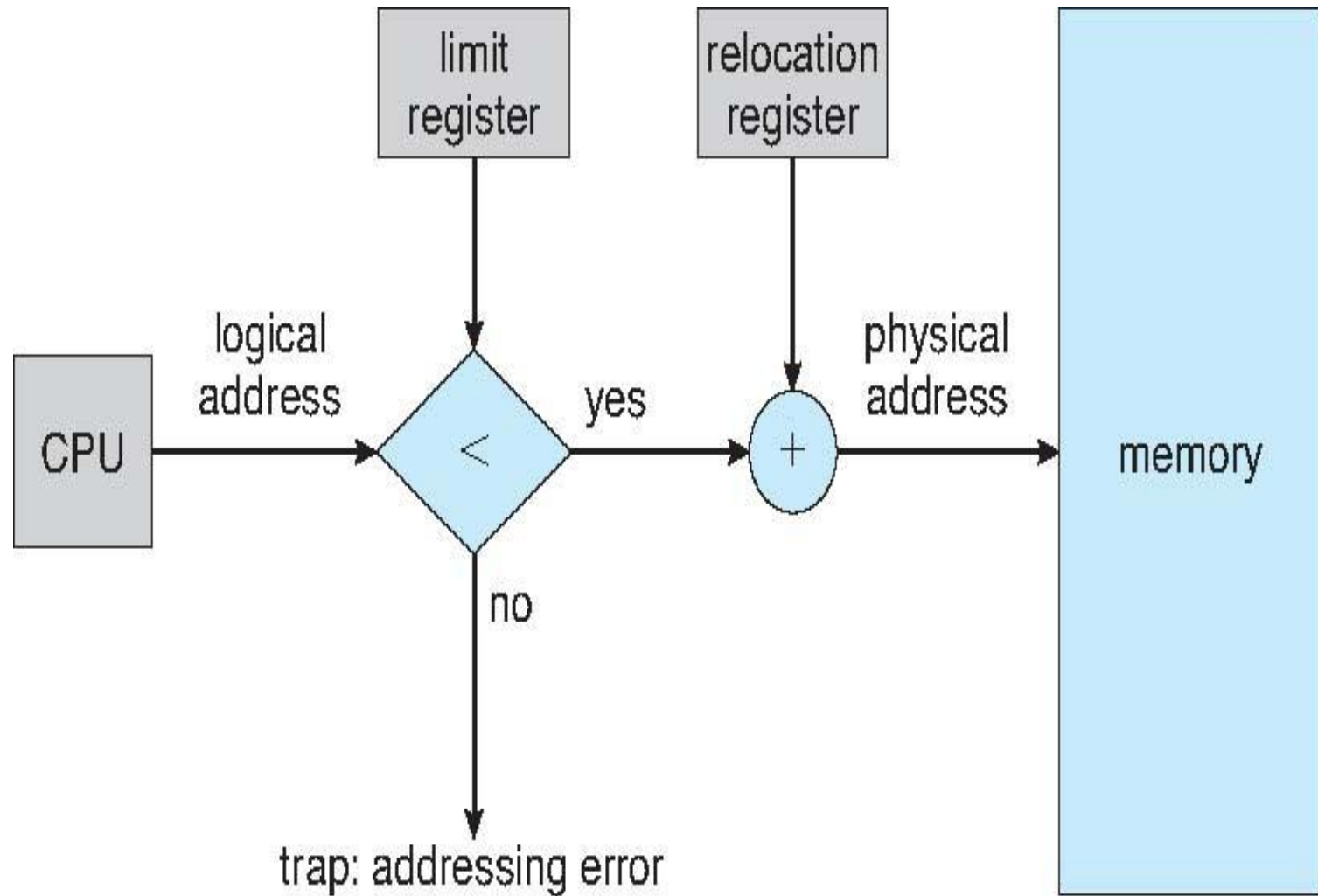
- The user Program deal with logical address; its never sees the actual physical address.



Contiguous Memory Allocation

- Contiguous memory allocation is single contiguous section/part of memory is allocated to a process **or** file needing it.
- Memory usually divided into **two partitions**:
 - Operating System**
 - User Process**
- We can implement contiguous memory allocation **by dividing the memory partitions into fixed size partitions.**

Hardware Support for Relocation and Limit Registers

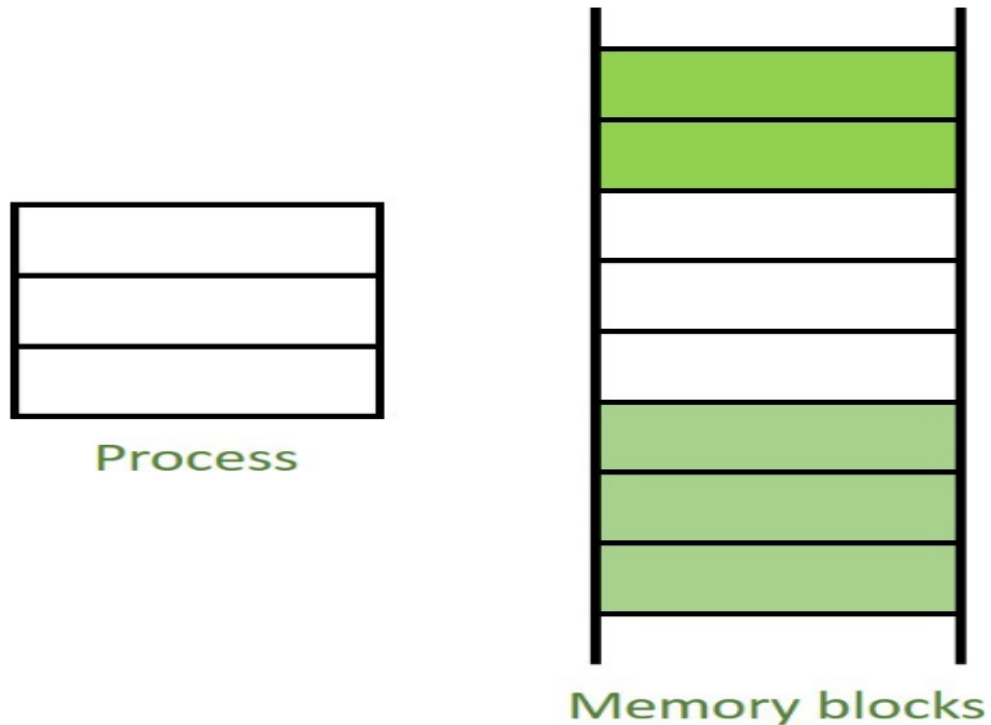


Contiguous Memory Allocation

Relocation registers used to protect user processes from each other, and from changing operating-system code and data

- **Base register** contains value of smallest physical address
- **Limit register** contains logical addresses – **each logical address must be less than the limit register**

1. Contiguous Memory Allocation



Contiguous Memory Allocation

Advantages

- **Faster Execution:** Less time is consumed in the execution of processes due to the fact that memory is provided in a sequential manner.
- **Minimal Overhead:** Fewer address translations, which all contribute to the efficiency of the system.
- **Easier to Control:** Organization of the memory in the operating system is sequential, which makes it possible to have efficient control of the processes.

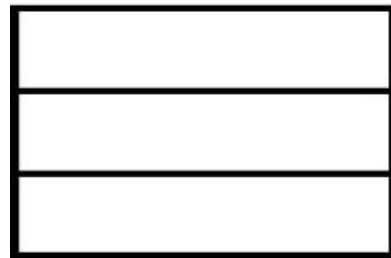
Disadvantages

- **Internal and External Fragmentation:** the memory blocks can be implemented that they may become fragmented, which results in inefficiency.
- **Limited Multiprogramming:** Fixed partitioning that it hinders the ability to control and allocate several processes.
- **Wastage of Memory:** With fixed partitions, memory can be wasted, especially where its usage is not optimal as required.

2. Non-Contiguous Memory Allocation

- Allocates the memory space in **different locations to the process** it's requirements.
- All available memory space in **distributed pattern** so the freely available memory space is also **scattered here and there**.
- This technique of memory allocation helps to **reduce the wastage of memory**.

Non-Contiguous Memory Allocation



Process



Memory blocks

Noncontiguous Memory Allocation

Advantages

- **No Memory Wastage:** Dynamic allocation makes sure that memory is used as is available and that there is no memory wasted.
- **Increased Multiprogramming:** Higher degrees of multiprogramming are feasible.
- **Reduced fragmentation:** Based on its primary uses, it mainly minimizes [internal fragmentation](#).

Disadvantages

- **Slower Execution:** Tasks could be longer in duration because memory is frequently split into blocks.

Contiguous Memory Allocation	Non-Contiguous Memory Allocation
Contiguous memory allocation allocates consecutive blocks of memory to a file/process.	Non-Contiguous memory allocation allocates separate blocks of memory to a file/process.
Faster in Execution.	Slower in Execution.
It is easier for the OS to control.	It is difficult for the OS to control.

Both Internal fragmentation and external fragmentation occurs in Contiguous memory allocation method.

Only External fragmentation occurs in Non-Contiguous memory allocation method.

It includes single partition allocation and multi-partition allocation.

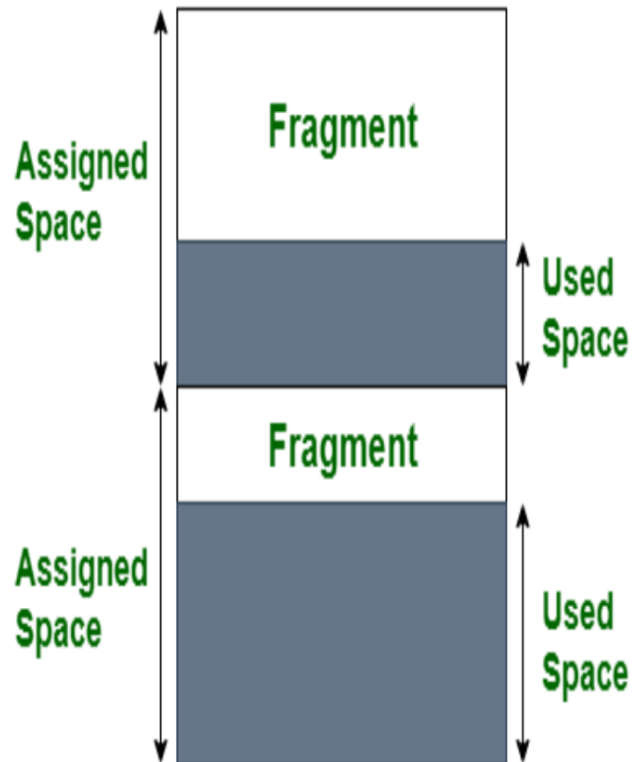
It includes paging and segmentation.

Wastage of memory is there.

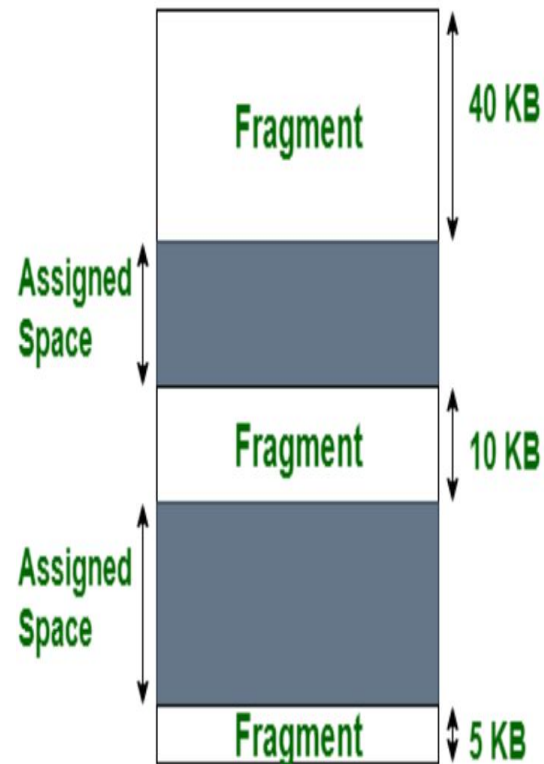
No memory wastage is there.

Internal and External Fragmentation

INTERNAL



EXTERNAL



Process 07
needs 50KB
memory space

- **Internal fragmentation:** (within allocated memory blocks)
- Occurs when a process is allocated a memory block larger than its actual memory requirement, resulting in unused space within the allocated block.

Internal fragmentation:

- **Cause:** Fixed-size memory allocation schemes like **paging**.
- **Example:** If a process needs 7KB of memory, but the OS allocates a 8KB block (due to fixed block sizes), the extra 1KB is internal fragmentation

External Fragmentation:

(occurs between allocated blocks)

- Occurs when free memory is broken into small, non-contiguous blocks, making it **difficult to allocate large contiguous blocks even if the total free memory is sufficient.**
- **Cause:** Often associated with dynamic memory allocation and the deallocation of memory blocks over time

Example:

When processes are allocated and deallocated, free blocks of memory can become scattered, leaving small, unusable gaps between them.

Feature	Internal Fragmentation	External Fragmentation
Location	Within allocated blocks	Between allocated blocks
Cause	Fixed-size allocation	Dynamic allocation, deallocation
Impact	Wasted space within blocks	Difficulty in allocating large blocks
Solution	<u>Best-fit allocation</u> , dynamic partitioning	<u>Paging</u> , <u>segmentation</u>

Paging

- **Paging** is a storage mechanism used to **retrieve processes from the secondary storage into main memory** in the form of pages.
- **Main idea** behind the paging is **to divide each process in the form of pages**.
- **Main memory** will also be **divided in form of frames**.
- **One page of the process** is to be stored in one of the frames of the memory.
- **The pages can be stored** at the different locations of the memory **but** the priority is always to find the contiguous frames or holes.

Paging

- Paging is **fixed size partitioning** scheme.
- In paging, secondary memory and main memory are divided into **equal fixed size partitioning**.
- **Divide physical memory** into fixed-sized blocks called **frames**
- **Divide logical memory** into blocks of same size called **pages**
- Set up a **page table** to translate logical to physical addresses.
- The size of the **last part may be less than** the page size.

Translating logical address into physical address

- CPU always generates a **logical address**.
- A **physical address** is needed to access the main memory.

Step 1:

CPU generates a logical address consisting of two parts-

1. Page Number.

2. Page Offset.

- **Page Number** specifies the **Specific Page** of the process from which CPU wants to read the data.
- **Page Offset** specifies the **Specific Word** on the page that CPU wants to read.

Translating logical address into physical address

Step 2:

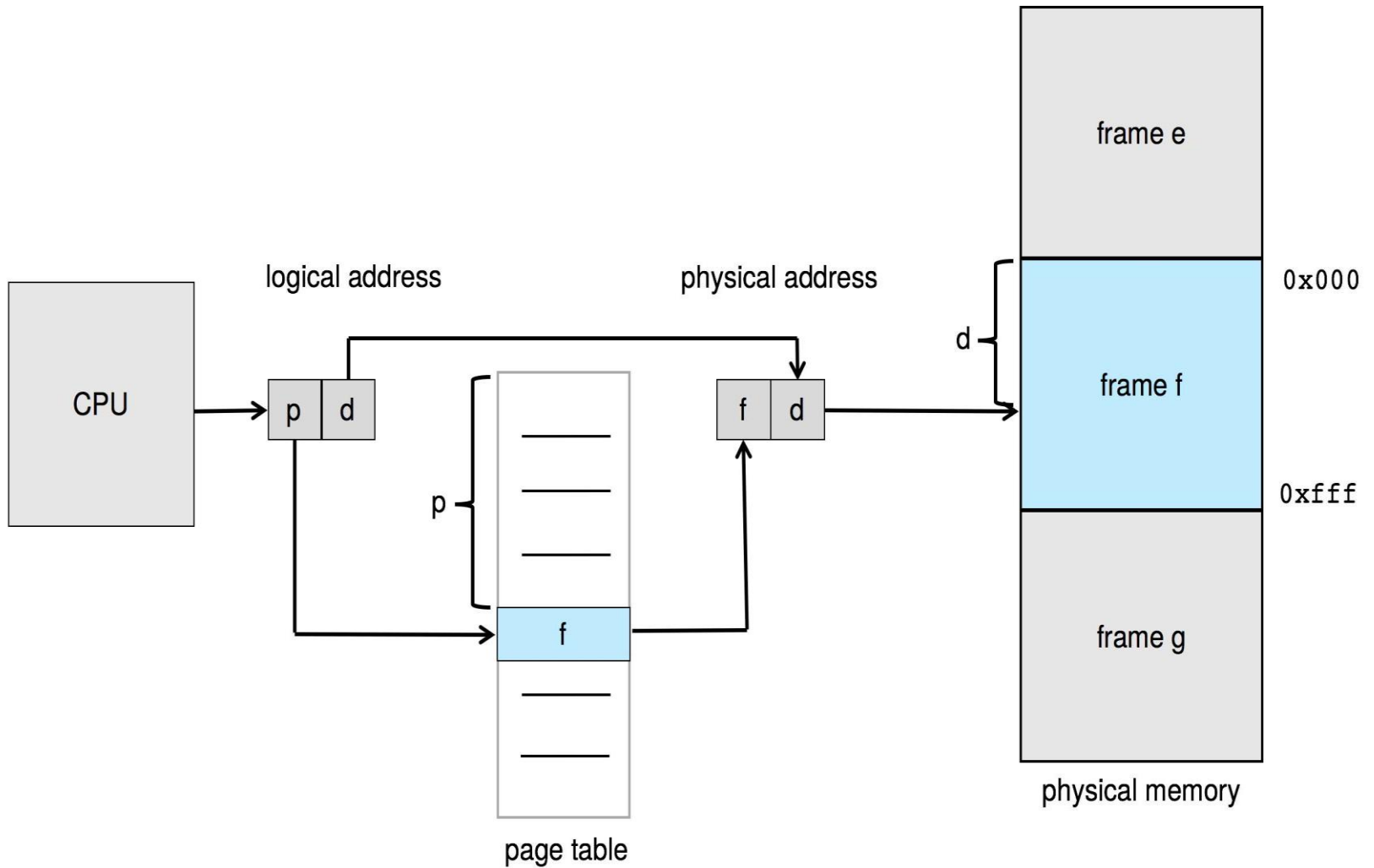
For the page number generated by the CPU, **Page Table** provides the corresponding frames number (base address of the frame) where that page is stored in the main memory.

Translating logical address into physical address

Step 3:

- Physical address consisting of two parts-
 1. Frame Number.
 2. Page Offset.
- The **Frame Number** combined with the page offset forms the required physical address.
- **Frame Number** specifies the **Specific Frame** where the required page is stored.
- **Page Offset** specifies the **Specific Word** that has to be read from that page.

Translating logical address into physical address



Paging Model of Logical and Physical Memory

page 0
page 1
page 2
page 3

logical
memory

0	1
1	4
2	3
3	7

page table

frame
number

0	
1	page 0
2	
3	page 2
4	page 1
5	
6	
7	page 3

physical
memory

Paging Example

- Logical address: $n = 2$ and $m = 4$.
- Using a page size of 4 bytes and
- Physical memory of 32 bytes (8 pages)

0	a
1	b
2	c
3	d
4	e
5	f
6	g
7	h
8	i
9	j
10	k
11	l
12	m
13	n
14	o
15	p

logical memory

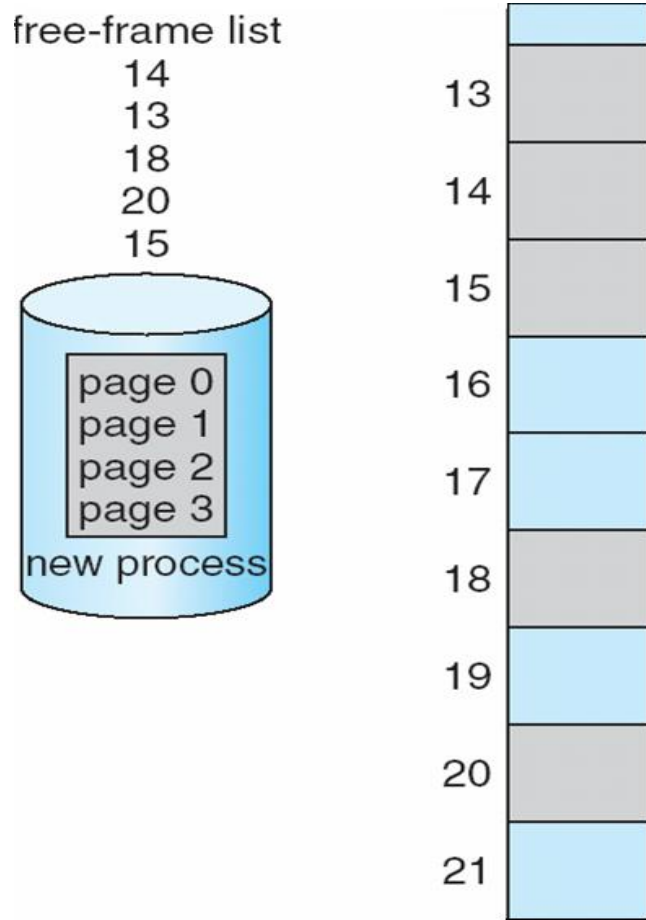
0	5
1	6
2	1
3	2

page table

0	
4	i j k l
8	m n o p
12	
16	
20	a b c d
24	e f g
28	

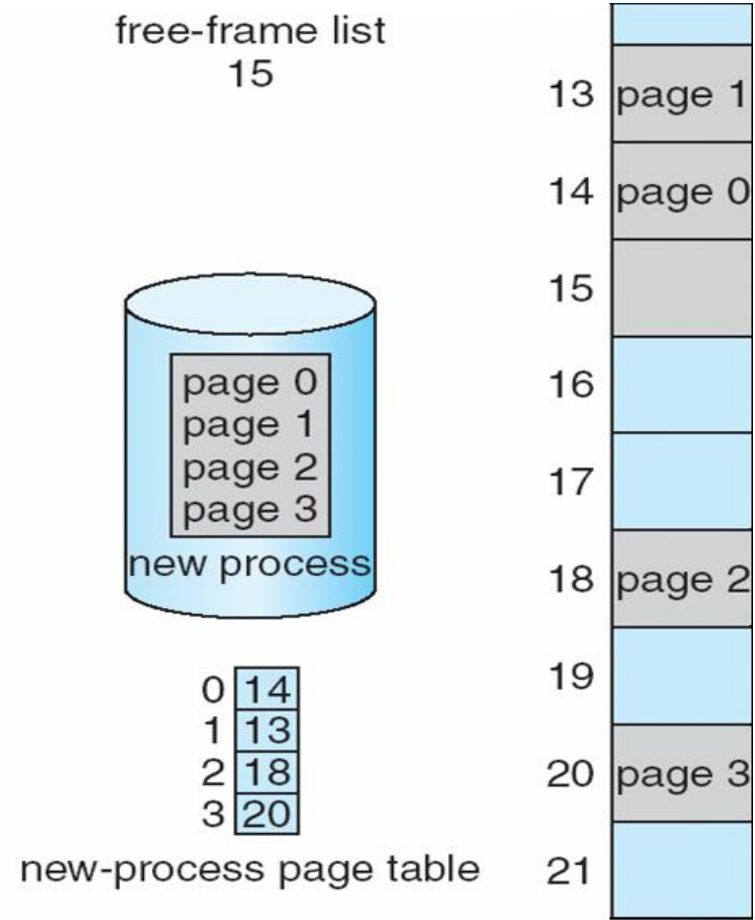
physical memory

Free Frames (before and After Allocation)



(a)

Before allocation



(b)

After allocation

Implementation of Page Table

- Page table is kept in main memory
 - **Page-table base register (PTBR)** points to the page table
 - **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires two memory accesses
 - One for the page table and one for the data / instruction
- The two-memory access problem can be solved by the use of a special fast-lookup hardware cache called **translation look-aside buffers (TLBs)** (also called **associative memory**).

Segmentation

- Segmentation is another way of dividing the addressable memory.
- Logical address space is basically the **collection of segments**.
- Each segment has **a name and a length**.
- In OS, segmentation is a memory management technique in which **the memory is divided into the variable size(segments) like Paging**.
- Each part known as a **segment which can be allocated to a process**.
- Each segment are stored in a table called a **segment table**.

Segmentation(cont)

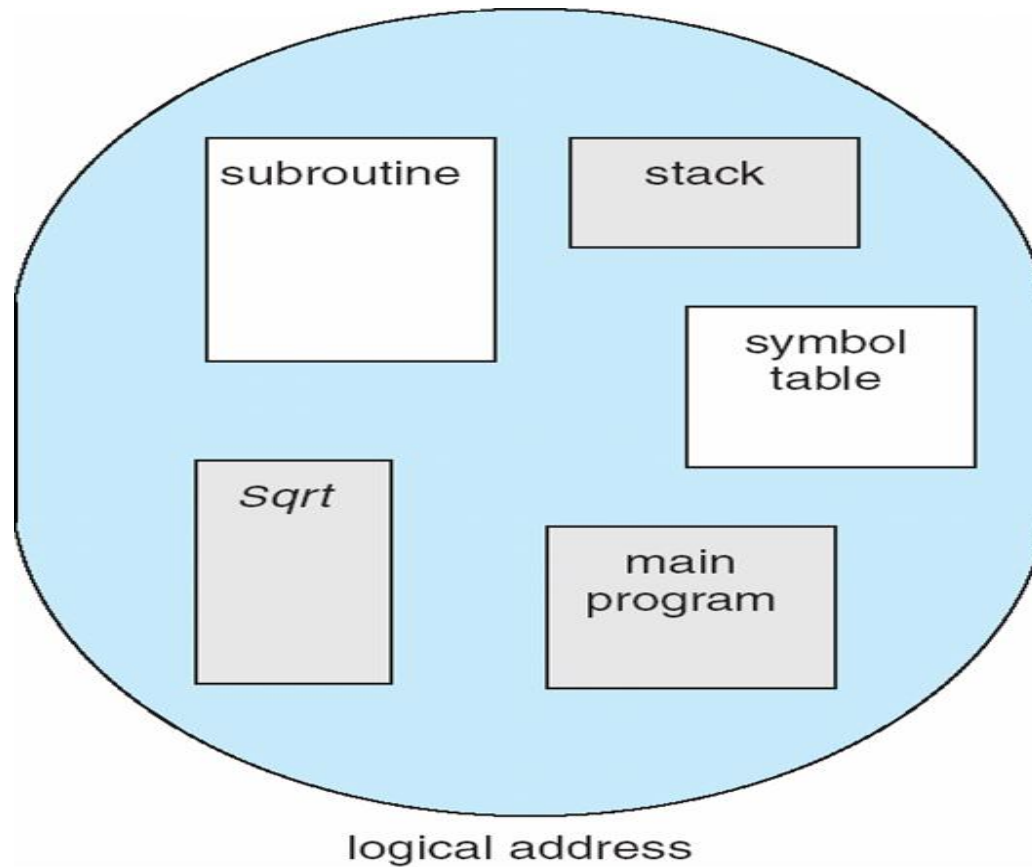
Example:-

A program is a collection of segments

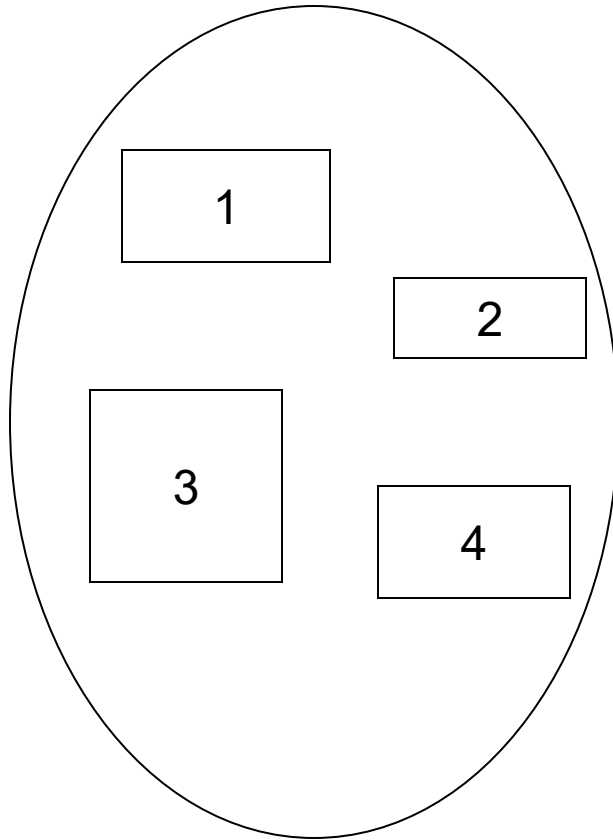
– A segment is a logical unit such as:

- ☐ Main Program
- ☐ Procedure
- ☐ Function
- ☐ Method
- ☐ Object
- ☐ Local Variables, Global Variables
- ☐ Common block
- ☐ Stack
- ☐ Symbol table
- ☐ Arrays

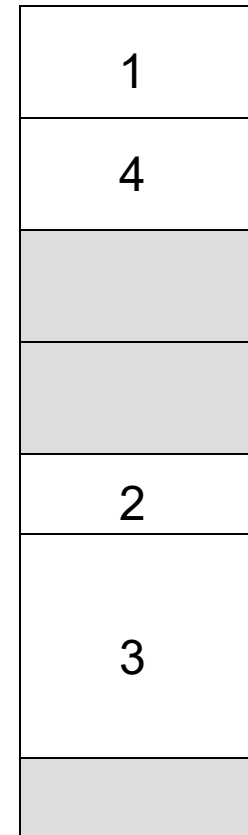
User's View of a Program



Logical View of Segmentation



user space

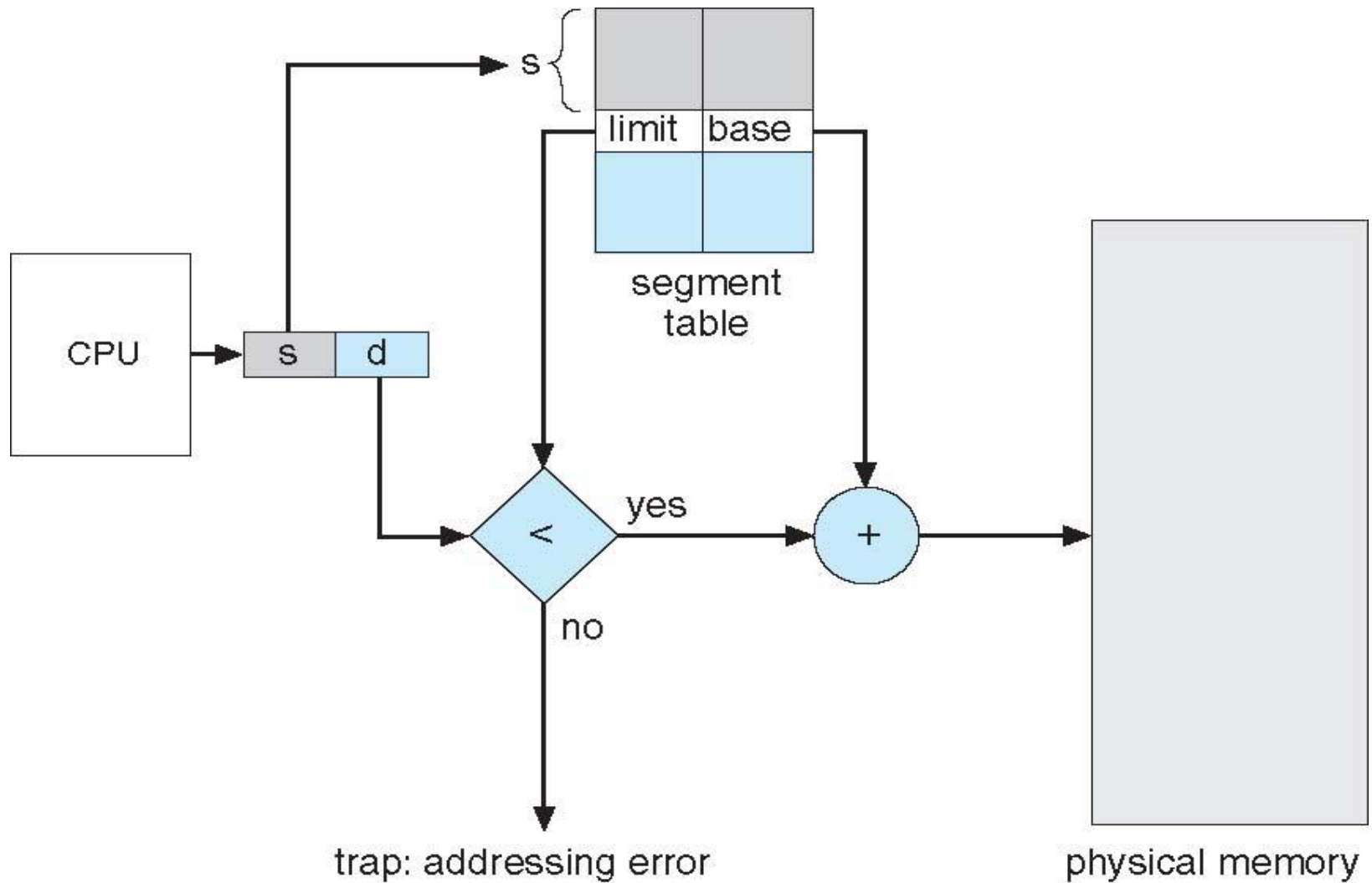


physical memory space

Segmentation Hardware(cont)

- Logical address consist of two tuple:
 $\langle \text{segment-number}, \text{offset} \rangle$
- **Segment table** - map two-dimensional physical addresses; each table entry has:
 - **Base**- contains the starting physical address where the segments reside in memory.
 - **Limit**- specifies the length of the segment.
- **Segment-table base register (STBR)** points to the segment table's location in memory.
- **Segment-table length register (STLR)** indicates number of segments used by a program.
- Segment number **s** is legal if $s < \text{STLR}$.

Segmentation Hardware



Virtual memory

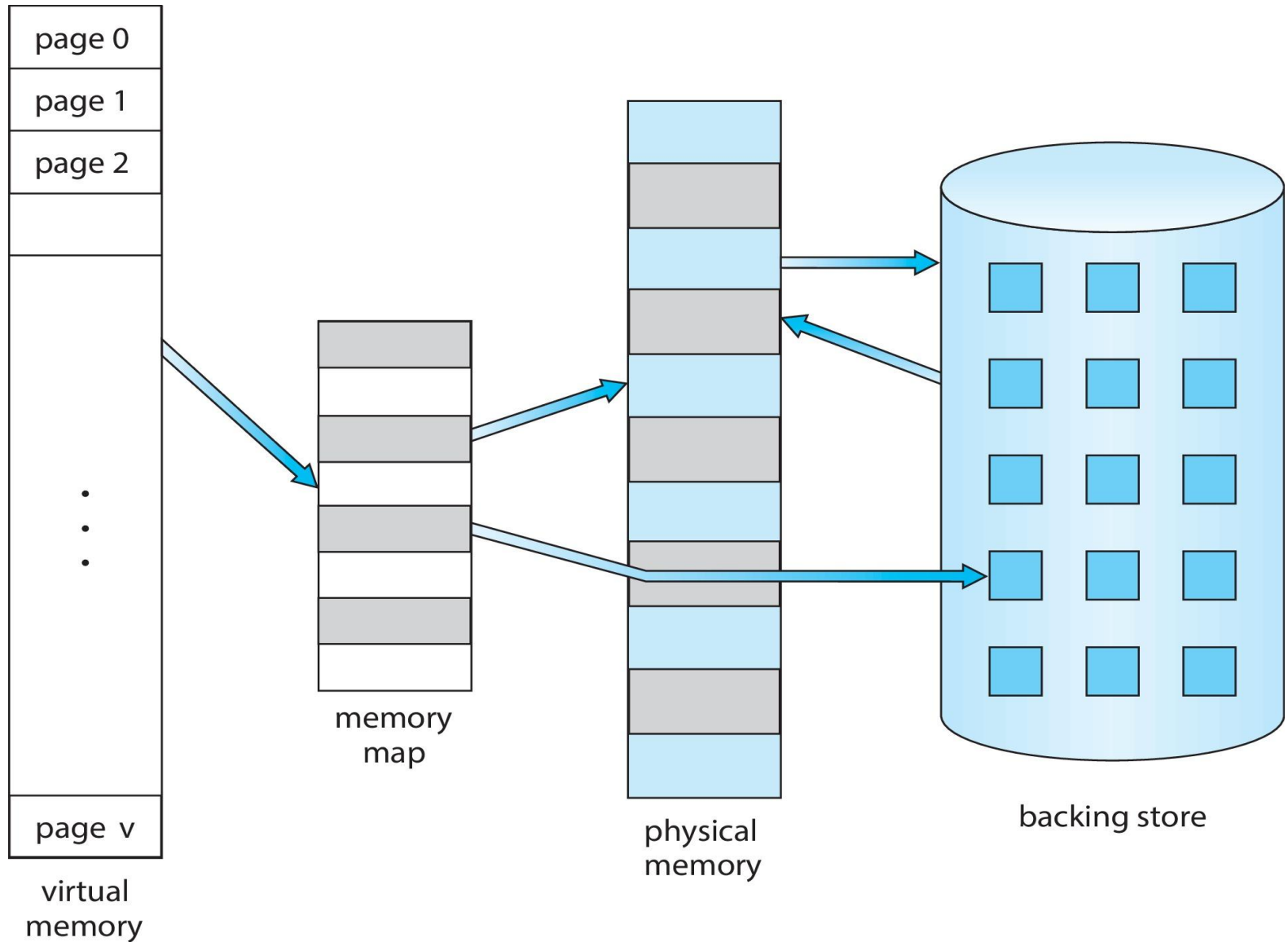
Virtual memory – separation of user logical memory from physical memory

- Only part of the program needs to be in memory for execution
- Logical address space can be much larger than physical address space
- Allows address spaces to be shared by several processes
- Allows for more efficient process creation
- More programs running concurrently
- Less I/O needed to load **or** swap processes

Virtual memory (Cont.)

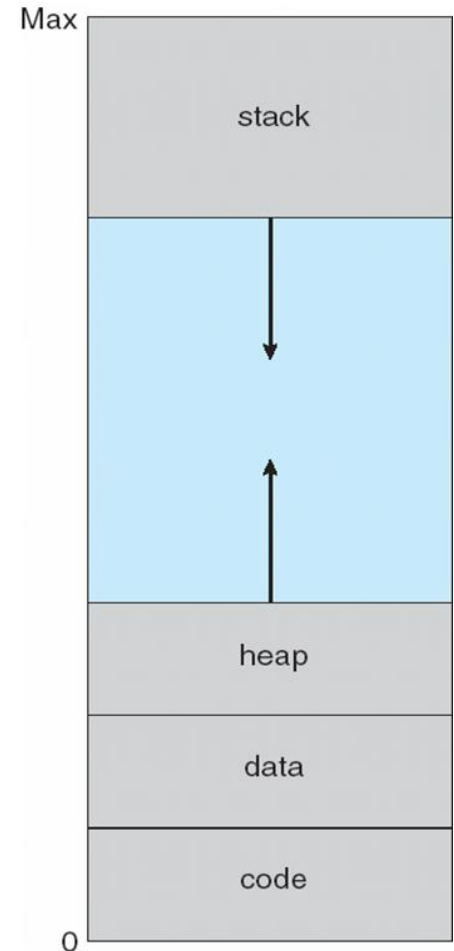
- **Virtual address space** – logical view of how process is stored in memory
 - Usually start at address 0, contiguous addresses until end of space
 - Meanwhile, physical memory organized in page frames
 - MMU must map logical to physical
- Virtual memory can be implemented via:
 - **Demand paging**
 - **Demand segmentation**

Virtual Memory is Larger than Physical Memory

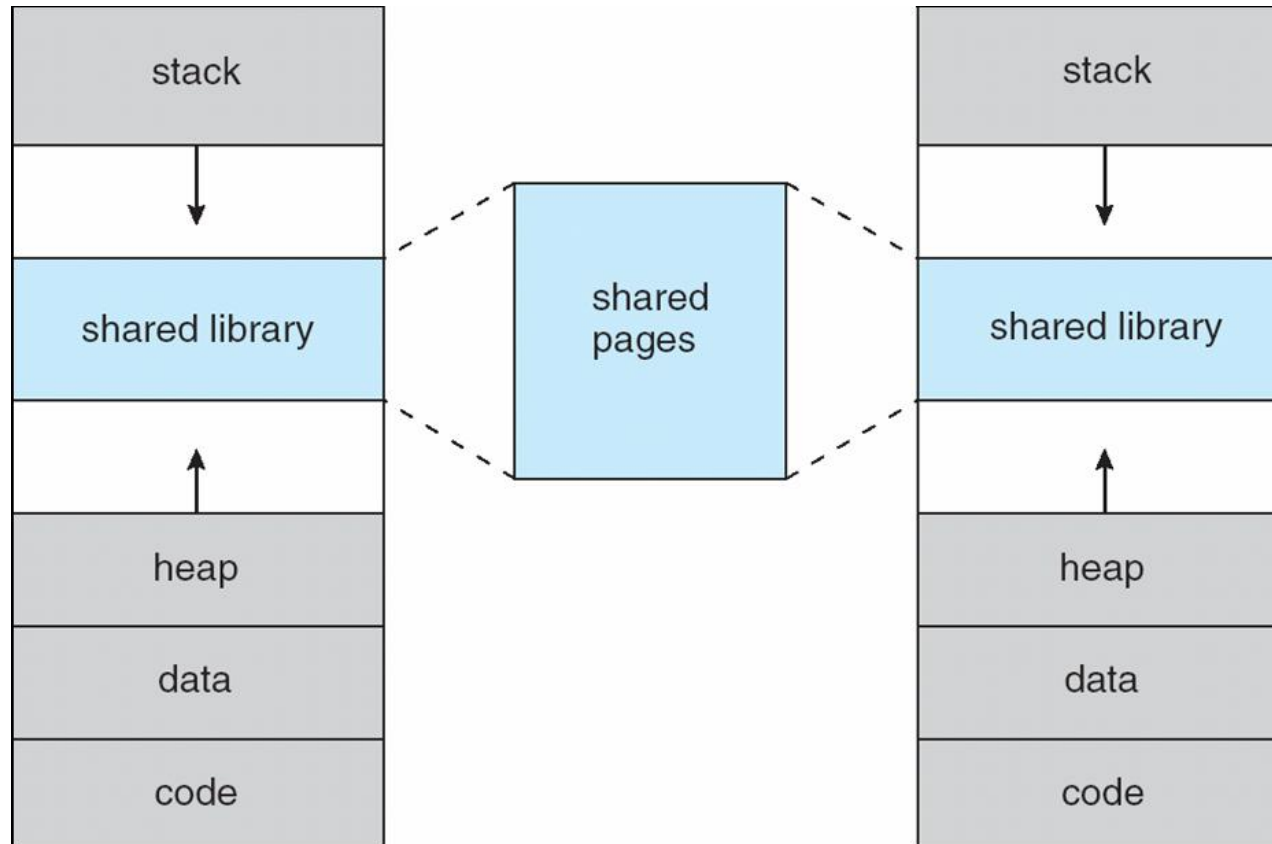


Virtual-address Space

- Logical address space for stack to start at Max logical address and grow “down” while heap grows “up”
 - **Maximizes address space use**
 - **Unused address space between the two is hole**
 - ▶ No physical memory needed until heap or stack grows to a given new page
- Enables **sparse** address spaces with holes left for growth, dynamically linked libraries, etc.
- **System libraries shared** via mapping into virtual address space
- **Shared memory** by mapping pages read-write into virtual address space
- Pages can be shared during `fork()`, speeding process creation



Shared Library Using Virtual Memory



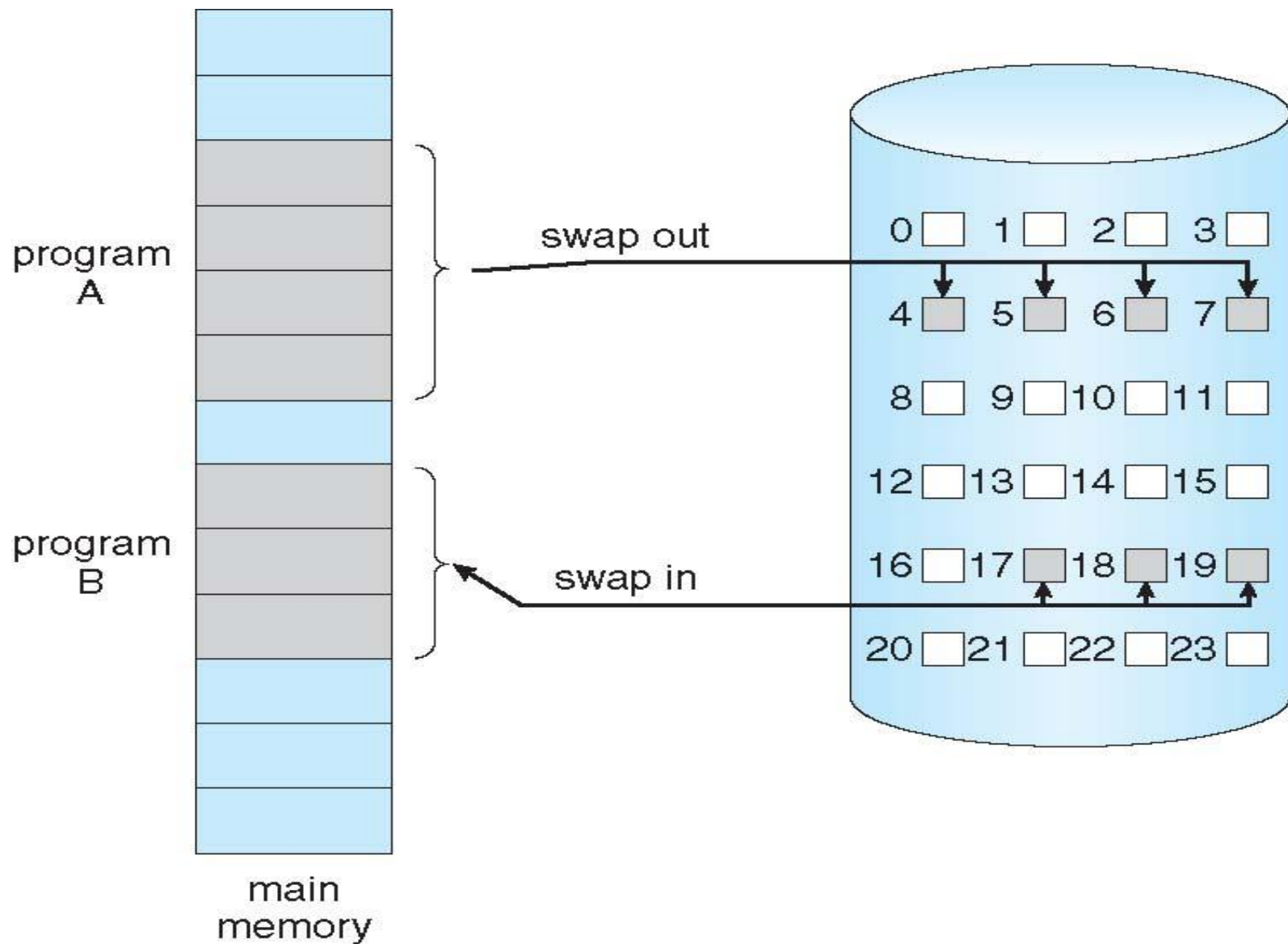
Demand Paging

- Could bring entire process into memory at load time **or** bring a page into memory **only when it is needed**
 - **Less I/O needed,**
 - **no unnecessary I/O**
 - **Less memory needed**
 - **Faster response**
 - **More users**
- Similar to paging system with swapping.
- Page is needed $\Rightarrow$ reference to it
 - invalid reference $\Rightarrow$ abort
 - not-in-memory $\Rightarrow$ bring to memory
- **Lazy swapper** – never swaps a page into memory unless page will be needed
 - Swapper that deals with pages is a **pager**

Basic Concepts

- With swapping, pager guesses which pages will be used before swapping out again. Instead, pager brings in only those pages into memory.
- How to determine that set of pages?
 - Need new MMU functionality to implement demand paging
- If pages needed are already **memory resident**
 - No difference from non-demand-paging
- If page needed and not memory resident
 - Need to detect and load the page into memory from storage
 - Without changing program behavior
 - Without programmer needing to change code

Demand Paging Structure



Valid-Invalid Bit

- With each page table entry a valid–invalid bit is associated
(**v** $\Rightarrow$ in-memory – **memory resident**, **i** $\Rightarrow$ not-in-memory)
- Initially valid–invalid bit is set to **i** on all entries
- Example of a page table snapshot:
During MMU address translation,
if valid–invalid bit in page table entry is **i** $\Rightarrow$ page fault

Frame #	valid-invalid bit
	v
	v
	v
	i
...	
	i
	i

page table

Page Table When Some Pages Are Not in Main Memory

0	A
1	B
2	C
3	D
4	E
5	F
6	G
7	H

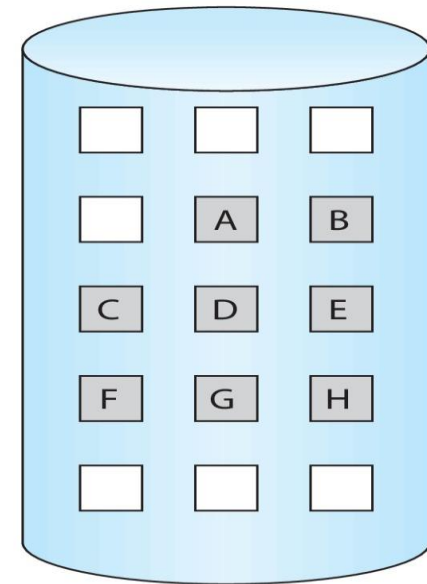
logical memory

valid-invalid bit	
frame	bit
0	4 v
1	i
2	6 v
3	i
4	i
5	9 v
6	i
7	i

page table

0	
1	
2	
3	
4	A
5	
6	C
7	
8	
9	F
10	
11	
12	
13	
14	
15	

physical memory

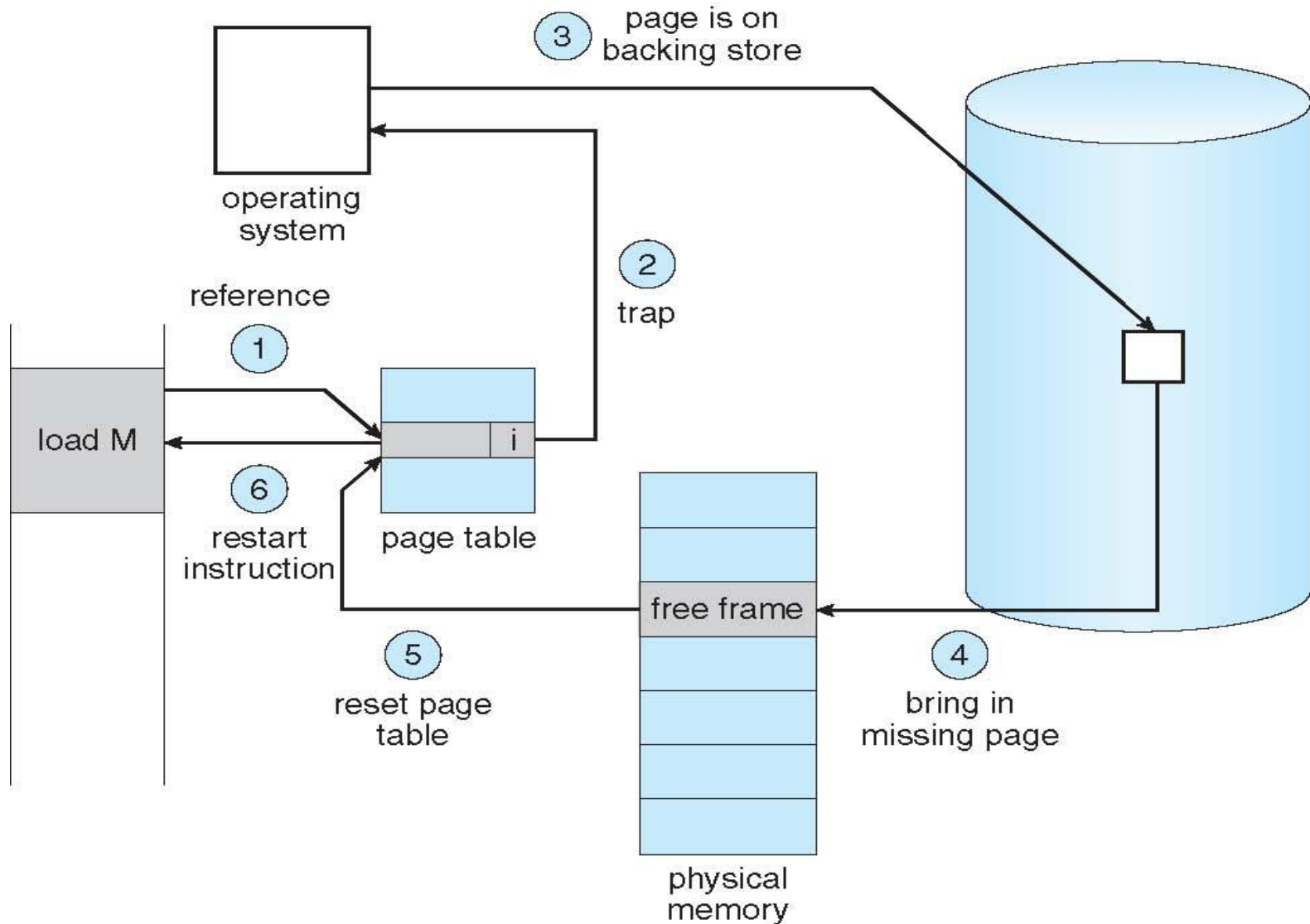


backing store

Steps in Handling Page Fault

1. If there is a reference to a page, first reference to that page will trap to operating system
 - **Page fault**
2. Operating system looks at another table to decide:
 - **Invalid reference $\Rightarrow$ abort**
 - **Just not in memory**
3. Find free frame
4. Swap page into frame **via** scheduled disk operation
5. Reset tables to indicate page now in memory
Set validation bit = **v**
6. Restart the instruction that caused the page fault

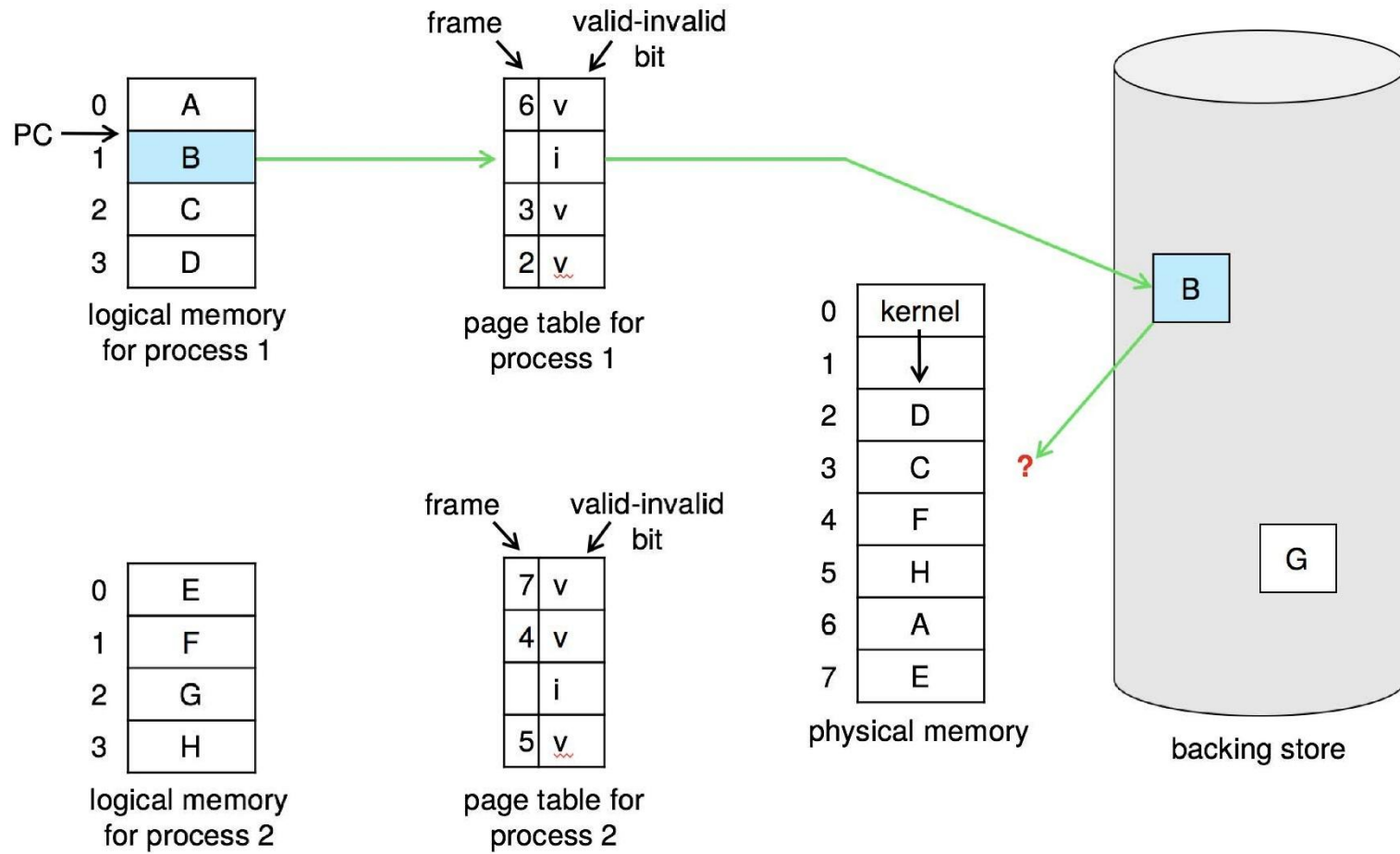
Steps in Handling a Page Fault (Cont.)



Page Replacement

- Prevent **over-allocation** of memory by modifying page-fault service routine to include page replacement
- Use **modify (dirty) bit** to reduce overhead of page transfers – only modified pages are written to disk
- **Page replacement** completes separation between logical memory and physical memory – large virtual memory can be provided on a smaller physical memory

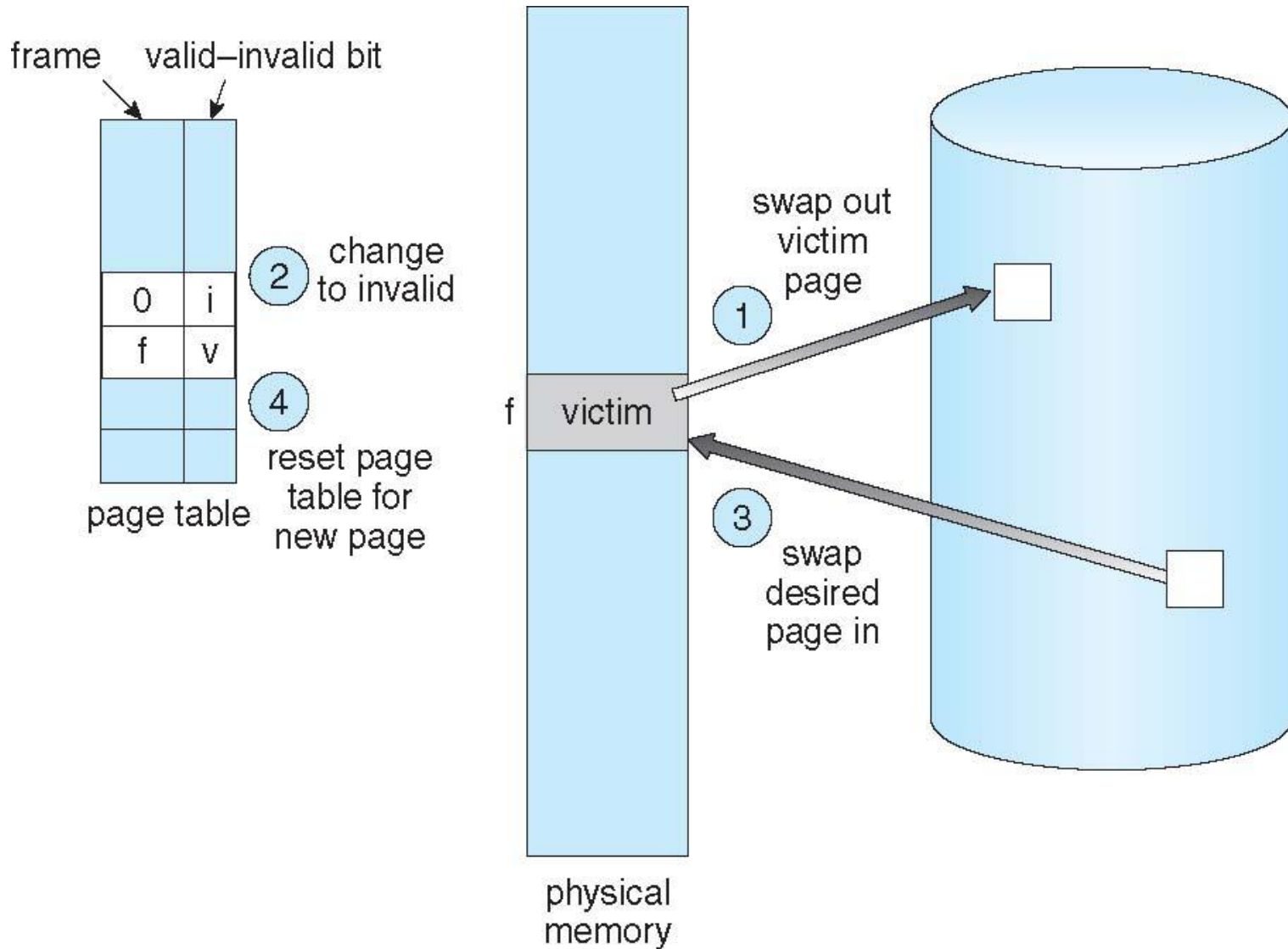
Need For Page Replacement



Basic Page Replacement

1. Find the location of the desired page on disk
2. Find a free frame:
 - If there is a free frame, **use it**
 - If there is no free frame, **use a page replacement algorithm to select a victim frame**
 - **Write victim frame to disk** if dirty
3. Bring the desired page into the (newly) free frame; update the page and frame tables
4. Continue the process by restarting the instruction that caused the trap

Page Replacement Architecture



Page and Frame Replacement Algorithms

- **Frame-allocation algorithm** determines
 - How many frames to give each process
 - Which frames to replace
- **Page-replacement algorithm**
 - Want lowest page-fault rate on both first access and re-access
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
 - String is just page numbers, not full addresses
 - Repeated access to the same page does not cause a page fault
 - Results depend on number of frames available
- In all our examples, the **reference string** of referenced page numbers is

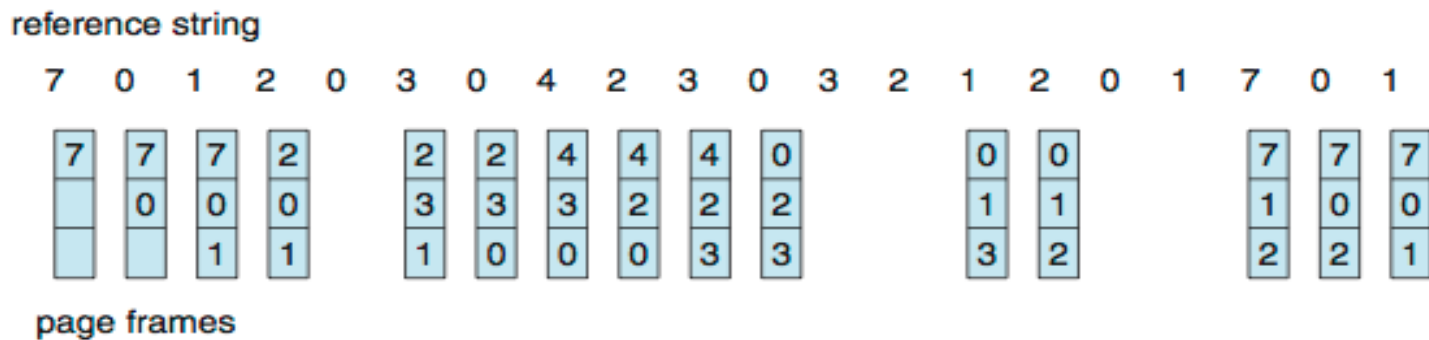
7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1

There are 6 Page Replacement Techniques:

1. First-In-First-Out (FIFO) Algorithm
2. Optimal Algorithm
3. Least Recently Used (LRU) Algorithm
4. LRU Approximation Algorithms
5. Counting Based Algorithms
6. Page Buffering Algorithm

1. First-In-First-Out (FIFO) Algorithm

- Reference string:
7,0,1,2,0,3,0,4,2,3,0,3,0,3,2,1,2,0,1,7,0,1
- 3 frames (3 pages can be in memory at a time per process)



15 page faults

- Can vary by reference string: consider
1,2,3,4,1,2,5,1,2,3,4,5
 - Adding more frames can cause more page faults!
 - **Belady's Anomaly**
- How to track ages of pages?
 - Just use a FIFO queue

2. Optimal Algorithm

- Replace page that **will not be used for (future) longest period of time**
 - 9 is optimal for the example
- How do you know this?
 - Can't read the future
- Used for measuring how well your algorithm performs

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		2			2		2					7		
	0	0	0		0		4			0		0					0		
		1	1		3		3			3		1					1		

page frames

3. Least Recently Used (LRU) Algorithm

- Use past knowledge rather than future (**must be replace the LRU element**)
- Replace page that has not been used in the most amount of time
- Associate time of last use with each page

reference string

7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

7	7	7	2		2		4	4	4	0			1		1		1		
	0	0	0		0		0	0	3	3			3		0		0		
		1	1		3		3	2	2	2			2		2		7		

page frames

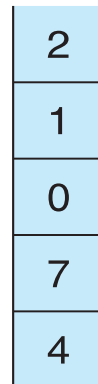
- 12 faults – better than FIFO but worse than OPT
- Generally good algorithm and frequently used
- But how to implement?

LRU Algorithm (Cont.)

- LRU and OPT are cases of **stack algorithms** that don't have Belady's Anomaly
- Use Of A Stack to Record Most Recent Page References

reference string

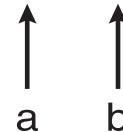
4 7 0 7 1 0 1 2 1 2 7 1 2



stack
before
a



stack
after
b



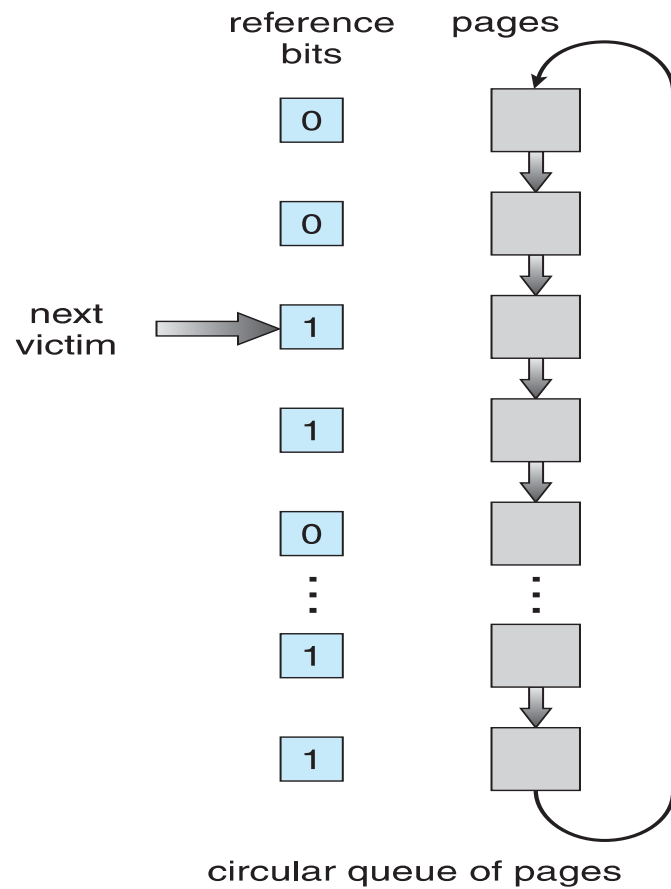
4.LRU Approximation Algorithms

- LRU needs special hardware and still slow
- **Reference bit**
 - With each page associate a bit, initially = 0
 - When page is referenced bit set to 1
 - Replace any with reference bit = 0 (if one exists)
 - We do not know the order, however

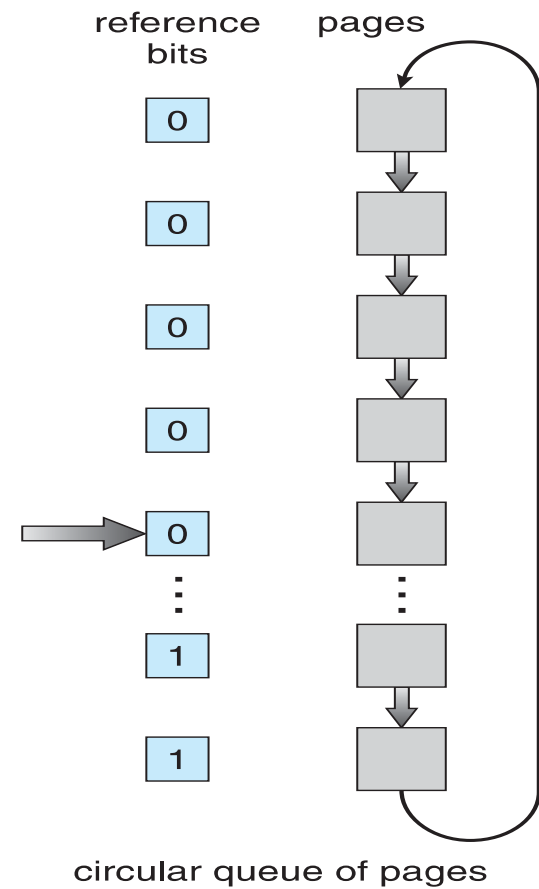
LRU Approximation Algorithms (cont.)

- **Second-chance algorithm**
 - Generally FIFO, plus hardware-provided reference bit
 - **Clock** replacement
 - If page to be replaced has
 - Reference bit = 0 -> replace it
 - reference bit = 1 then:
 - set reference bit 0, leave page in memory
 - replace next page, subject to same rules

Second-chance Algorithm



(a)



(b)

Enhanced Second-Chance Algorithm

- Improve algorithm by using reference bit and modify bit (if available) in concert
- Take ordered pair (reference, modify):
 - (0, 0) neither recently used nor modified – best page to replace
 - (0, 1) not recently used but modified – not quite as good, must write out before replacement
 - (1, 0) recently used but clean – probably will be used again soon
 - (1, 1) recently used and modified – probably will be used again soon and need to write out before replacement
- When page replacement called for, use the clock scheme but use the four classes replace page in lowest non-empty class
 - Might need to search circular queue several times

5. Counting Algorithms

- Keep a counter of the number of references that have been made to each page
 - Not common
- **Least Frequently Used (LFU) Algorithm:**
 - Replaces page with smallest count
- **Most Frequently Used (MFU) Algorithm:**
 - Based on the argument that the page with the smallest count was probably just brought in and has yet to be used

6. Page-Buffering Algorithms

- Keep a pool of free frames, always
 - Then frame available when needed, not found at fault time
 - Read page into free frame and select victim to evict and add to free pool
 - When convenient, evict victim
- Possibly, keep list of modified pages
 - When backing store otherwise idle, write pages there and set to non-dirty
- Possibly, keep free frame contents intact and note what is in them
 - If referenced again before reused, no need to load contents again from disk
 - Generally useful to reduce penalty if wrong victim frame selected

File Management

Contents

- File Concepts
- File Types
- File Operations
- Access Methods
- Directory Structure
- File System Structure

File Concept

- The file is a collection of related information that is recorded on storage device.
- The files stores information and data.
- It is a way of data collection which is used as medium of giving input and collecting output from any application.

File Types

1. Ordinary Files (Regular Files)

- It contains user information
- It contains text or executable programs
- It is either text file or binary file
- User can apply various operations

2. Directory Files

- It is like a folder that contains files or other folder
- It don't contain data
- But they have references to file contain within them

File Types

3. Device File

- ☐ contains device information
- It is used to communicate with hardware
- Device file is of two types

i. **Character special file**

It is a hardware file which read or write data character by character

e.g.: mouse, printer

ii. **Block file**

It contains data related to hard disk

File Types

4. FIFO Files

- It is a file which provides temporary buffer for two or more process to communicates by writing data & reading data from buffer
- The buffer associated with this file is allocated when any process opens the file & buffer is discarded when all processes which are connected to this file close their reference

File Operation

1. Create
2. Delete
3. Open
4. Close
5. Read
6. Write
7. Append
8. Seek
9. Get Attributes
10. Set Attributes
11. Rename

File System Structure

- The concept of file system is to access the disk efficiently. It allows data to store & retrieve from disk.
- File system structure is composed of many layers as following:
 1. **Input-Output Control Interface**
 - ✓ It consist of device driver and interrupt handler.
 - ✓ Both are used for data transfer between memory & disk
 - ✓ The device driver translates file level command & output is hardware instruction

2. Basic File System Layers

- ✓ It generates commands for device driver
- ✓ Device driver read or write physical block disk

3. File Organization Module Layer

- ✓ This layer translates logical block address to physical block address

4. Logical System Layer

- ✓ It manages metadata of any file like location & permission of file

5. Application Program Layer

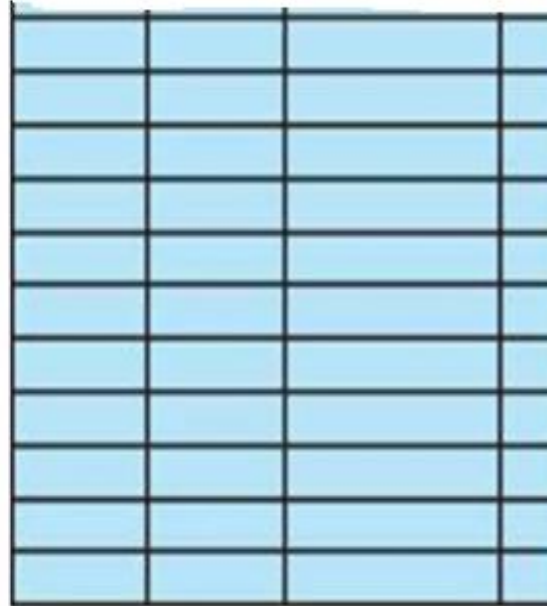
- ✓ This is a layer in which user creates an application program

Access Method

- The information stored in the  needs to be accessed and read into the computer memory.
- There are different methods available to do it.
 - 1) Sequential Access
 - 2) Indexed Access
 - 3) Direct Access

Sequential Access

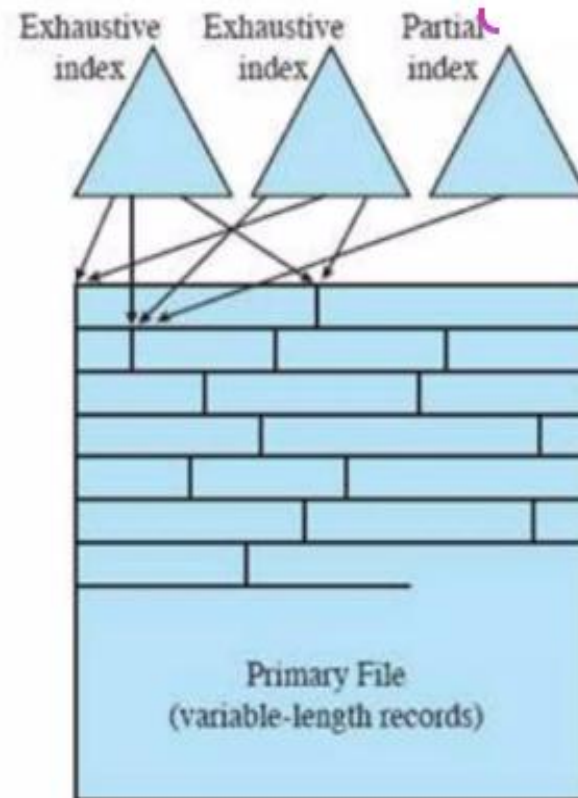
- Most common form of file structure
- A fixed format is used for records
- Key field uniquely identifies the record & determines storage order
- Typically used in batch applications
- Only organization that is easily stored on tape as well as disk



Fixed-length records
Fixed set of fields in fixed order
Sequential order based on key field

Indexed Access

- Records are accessed only through their indexes
- Variable-length records can be employed
- Exhaustive index contains one entry for every record in the main file
- Partial index contains entries to records where the field of interest exists
- Used mostly in applications where timeliness of information is critical
- Examples would be airline reservation systems and inventory control systems



Direct Access

- Access directly any block of a known address
- Makes use of hashing on the key value
- Often used where:
 - very rapid access is required
 - fixed-length records are used
 - records are always accessed one at a time



Directory Structure

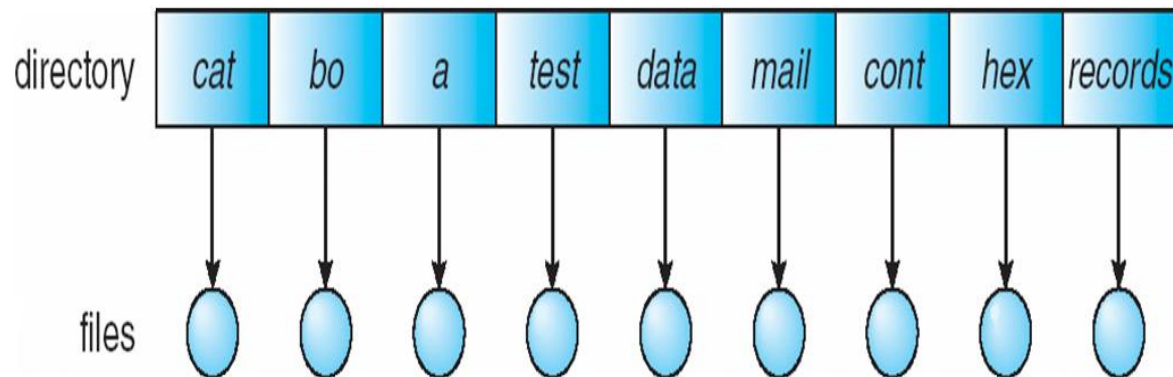
- A **directory** is a container that is used to contain folders and files. It organizes files and folders in a hierarchical manner.
- The operating system uses directories to track where files are stored, just like using folders to organize papers.
- Different directory structures can be used to suit various organizational needs.
- Understanding directory structures helps in organizing and accessing files more easily.

Different Types of Directories in OS

1. Single-Level Directory
2. Two-Level Directory
3. Tree Structure/ Hierarchical Structure
4. Acyclic Graph Structure
5. General-Graph Directory Structure

1) Single-Level Directory

- The single-level directory is the **simplest directory structure**. In it, all files are contained in the same directory which makes it easy to support and understand.



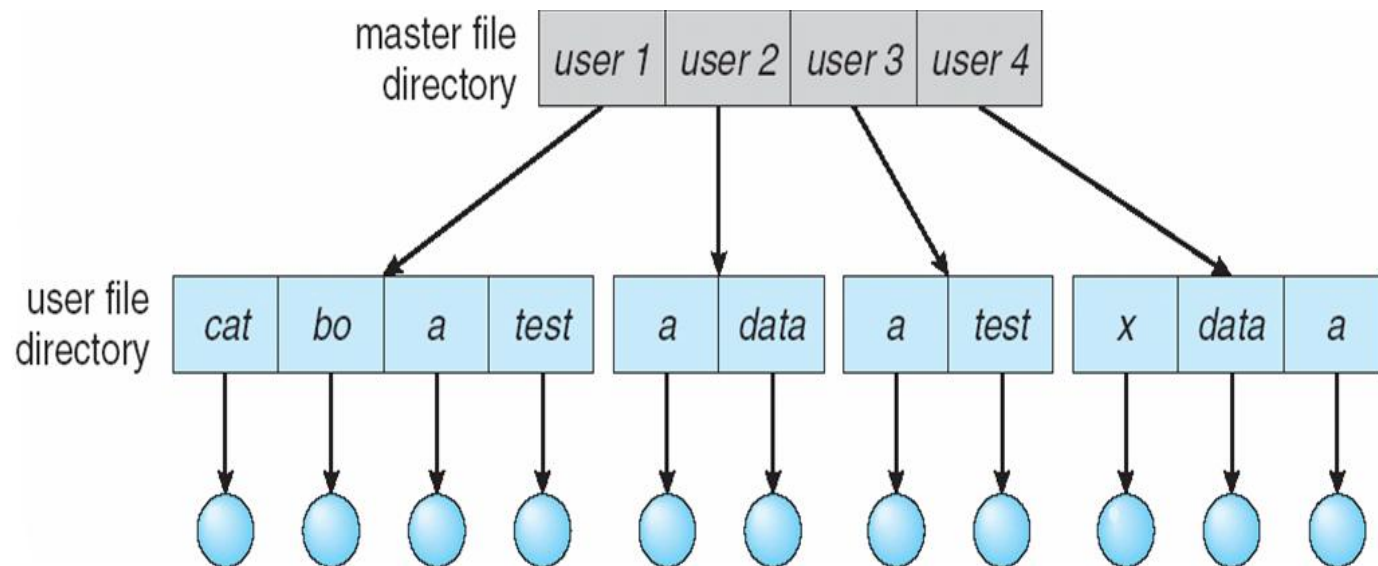
- Since it is a single directory, so its implementation is very easy.
- If the files are smaller in size, searching will become faster.
- The operations like file creation, searching, deletion, updating are very easy in such a directory structure.

Advantages

- Logical Organization
- Increased Efficiency
- Improved Security
- Facilitates Backup and Recovery
- Scalability

2) Two-Level Directory

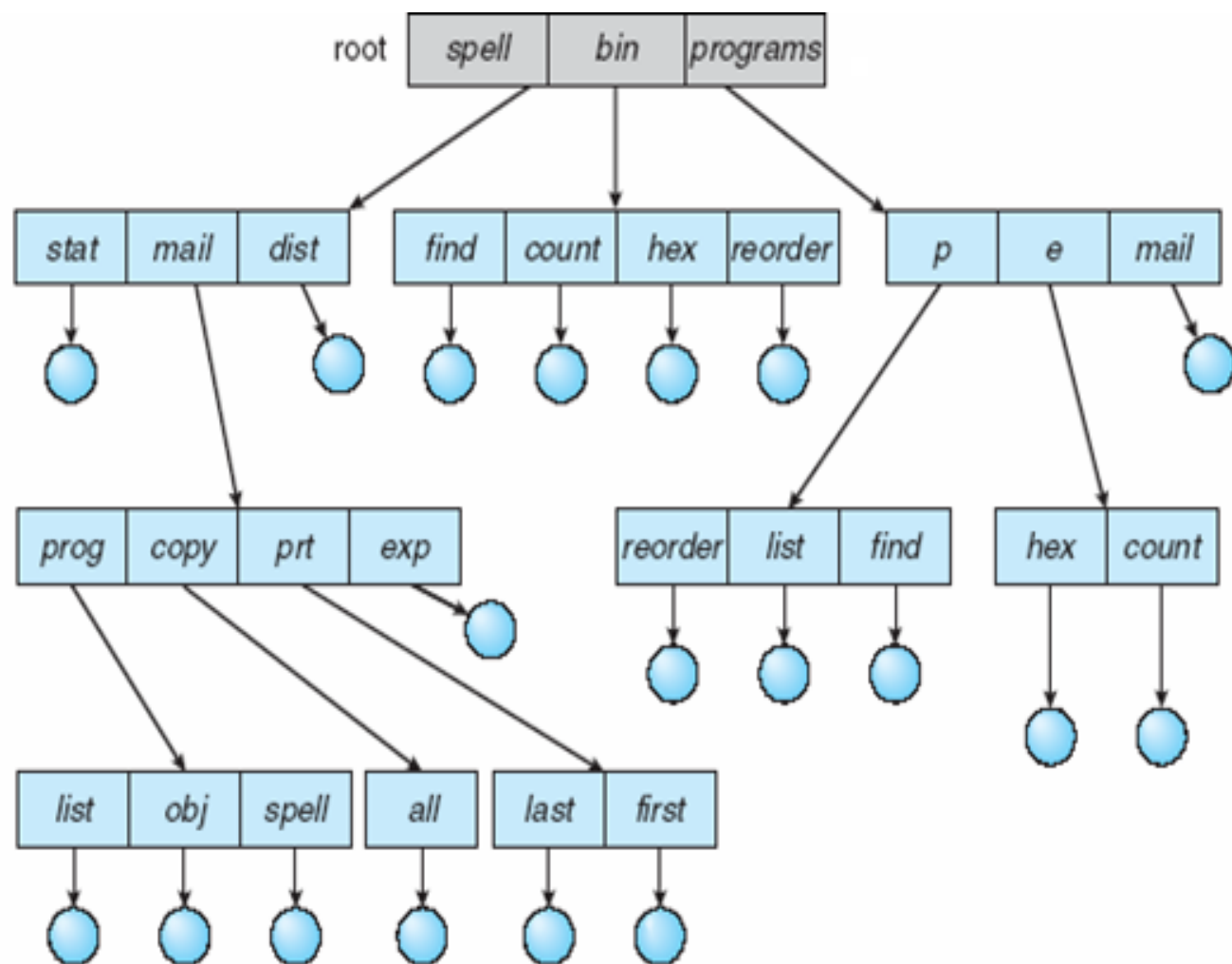
- each user has their own **user files directory (UFD)**.
- The UFDs have similar structures, but each lists only the files of a single user.
- System's **master file directory (MFD)** is searched whenever a new user id is created.



a single level directory often leads to confusion of files names among different users. The solution to this problem is to create a **separate directory for each user.**

3) Tree Structure/ Hierarchical Structure

- It used in our personal computers. User can create files and subdirectories too, which was a disadvantage in the previous directory structures.
- This directory structure resembles a real tree upside down, where the **root directory** is at the peak.
- This root contains all the directories for each user. The users can create subdirectories and even store files in their directory.

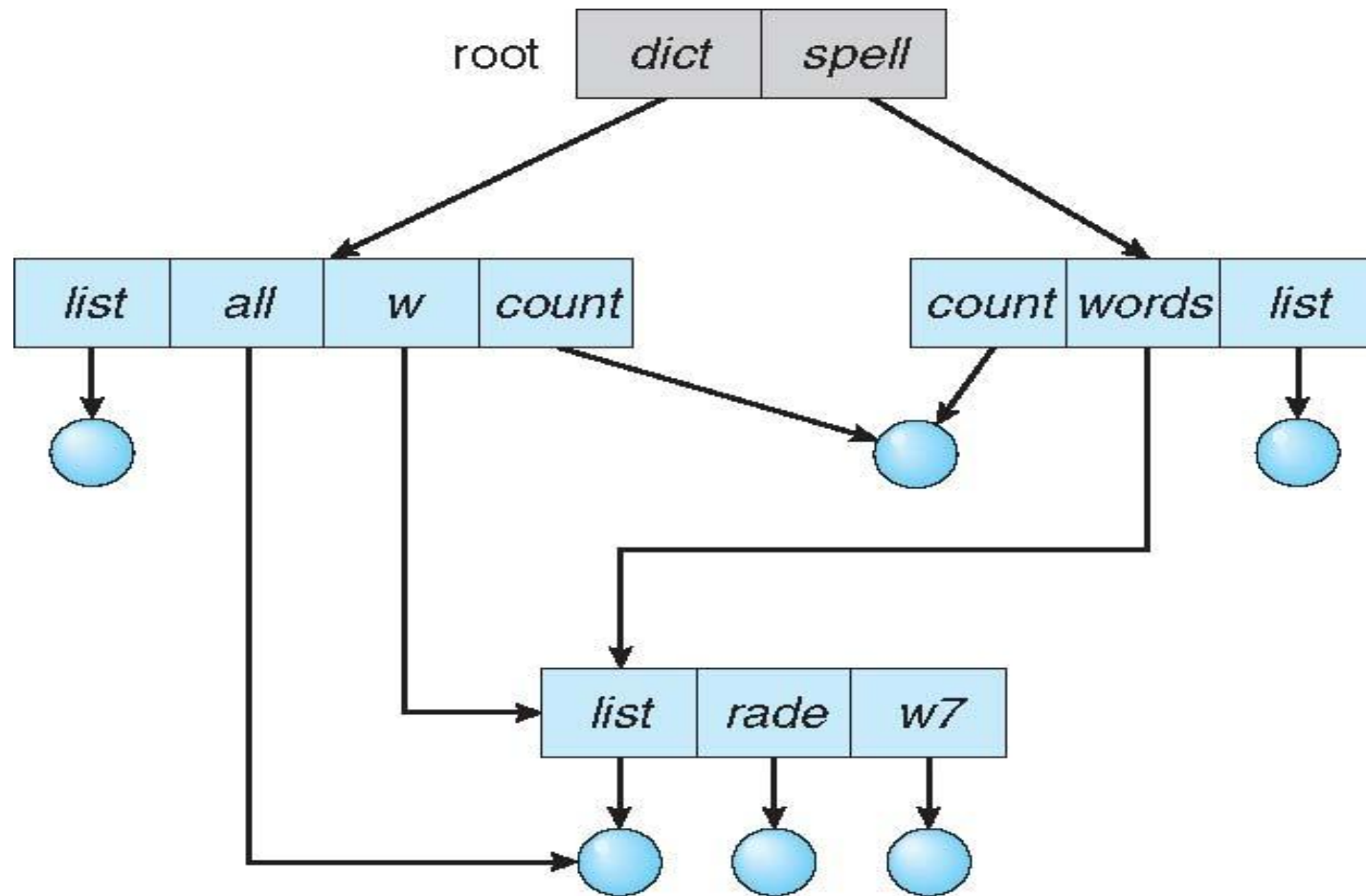


Advantages

- This directory structure allows subdirectories inside a directory.
- The searching is easier.
- File sorting of important and unimportant becomes easier.
- This directory is more scalable than the other two directory structures explained.

4) Acyclic Graph Structure

- AGS, where a file in one directory can be accessed from multiple directories.
- These files could be shared in between the users.
- It is designed in a way that multiple directories point to a particular directory or file with the help of links.



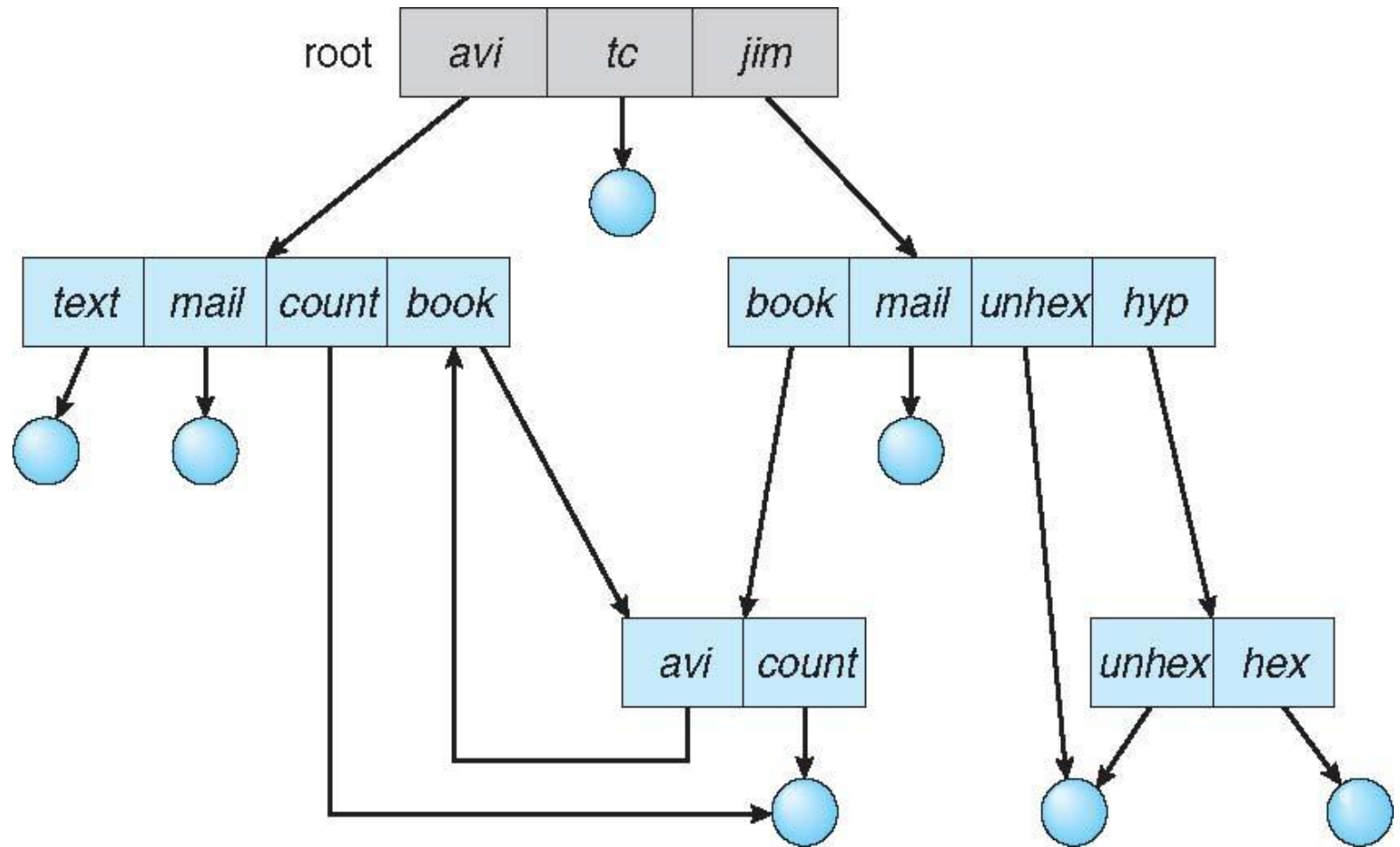
5) General-Graph Directory Structure

- Unlike the acyclic-graph directory, which avoids loops, the general-graph directory can **have cycles**, meaning a **directory can contain paths** that loop back to the starting point.
- This can make navigating and managing files more complex.

Advantages of General-Graph Directory

- More flexible than other directory structures.
- Allows cycles, meaning directories can loop back to each other.

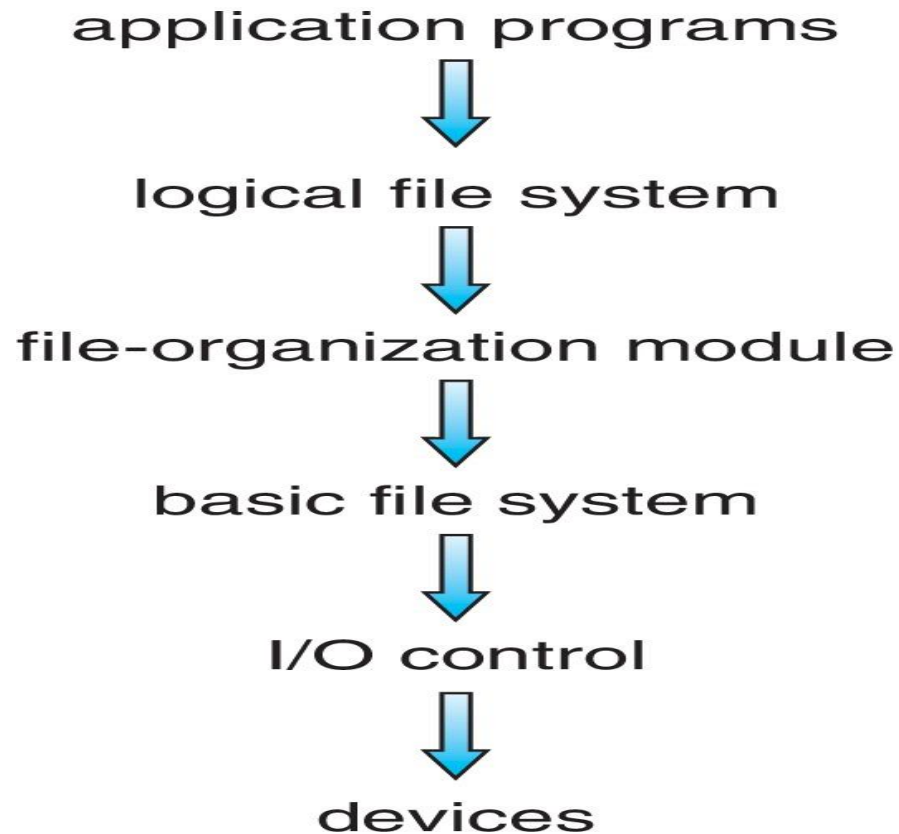
General-Graph Directory Structure



File-System Structure

- File structure
 - Logical storage unit
 - Collection of related information
- **File system** resides on secondary storage (disks)
 - Provided user interface to storage, mapping logical to physical
 - Provides efficient and convenient access to disk by allowing data to be stored, located retrieved easily
- Disk provides in-place rewrite and random access
 - I/O transfers performed in **blocks** of **sectors** (usually 512 bytes)
- **File control block (FCB)** – storage structure consisting of information about a file
- **Device driver** controls the physical device
- File system organized into layers

Layered File System



Allocation Method

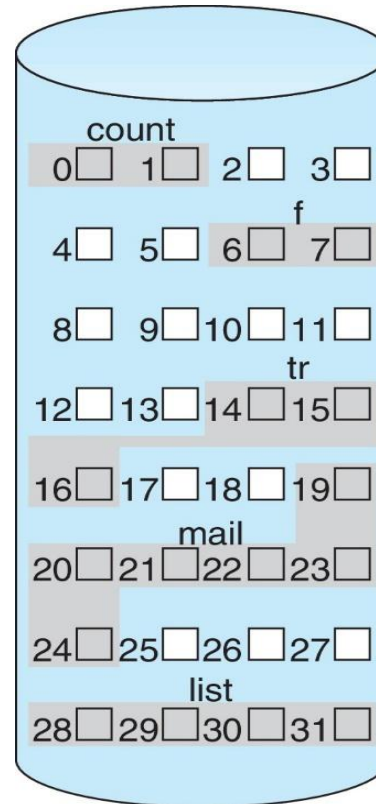
- An allocation method refers to how disk blocks are allocated for files:
 - Contiguous
 - Linked
 - File Allocation Table (FAT)

Contiguous Allocation Method

- An allocation method refers to how disk blocks are allocated for files:
- Each file occupies set of contiguous blocks
 - Best performance in most cases
 - Simple – only starting location (block #) and length (number of blocks) are required
 - Problems include:
 - Finding space on the disk for a file,
 - Knowing file size,
 - External fragmentation, need for **compaction off-line (downtime)** or **on-line**

Contiguous Allocation (Cont.)

- Mapping from logical to physical (block size = 512 bytes)
- $$\begin{array}{l} \text{LA}/512 \quad \swarrow \text{Q} \\ \quad \searrow \text{R} \end{array}$$
- Block to be accessed = starting address + Q
 - Displacement into block = R



directory

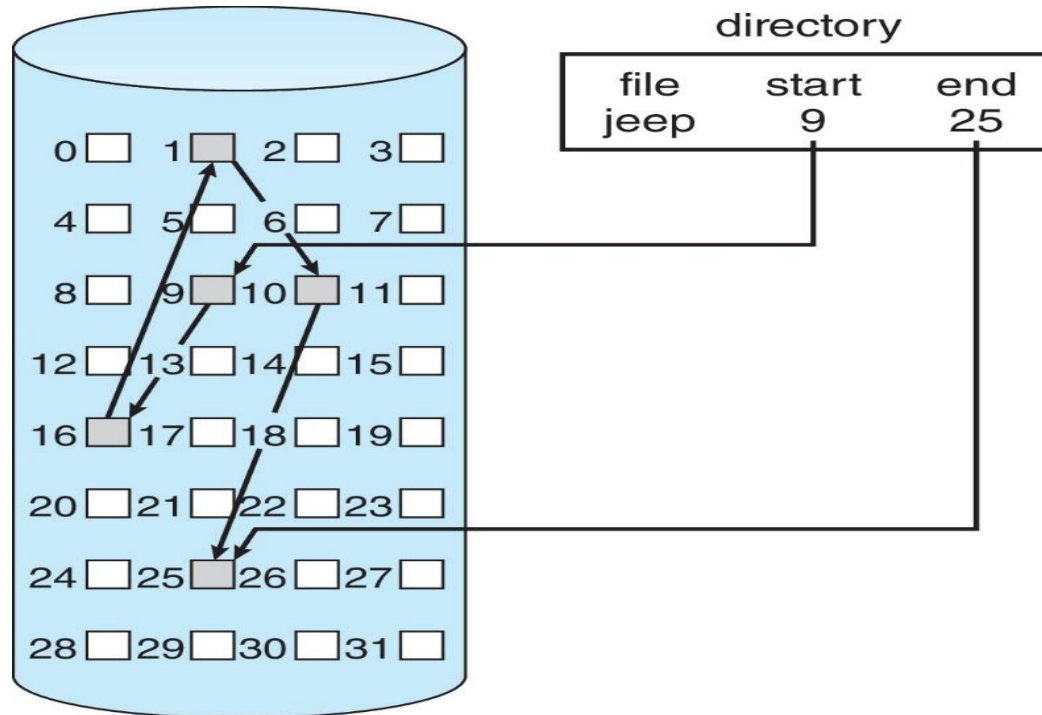
file	start	length
count	0	2
tr	14	3
mail	19	6
list	28	4
f	6	2

Linked Allocation

- Each file is a linked list of blocks
- File ends at nil pointer
- No external fragmentation
- Each block contains pointer to next block
- No compaction, external fragmentation
- Free space management system called when new block needed
- Improve efficiency by clustering blocks into groups but increases internal fragmentation
- Reliability can be a problem
- Locating a block can take many I/Os and disk seeks

Linked Allocation

- Each file is a linked list of disk blocks: blocks may be scattered anywhere on the disk Scheme



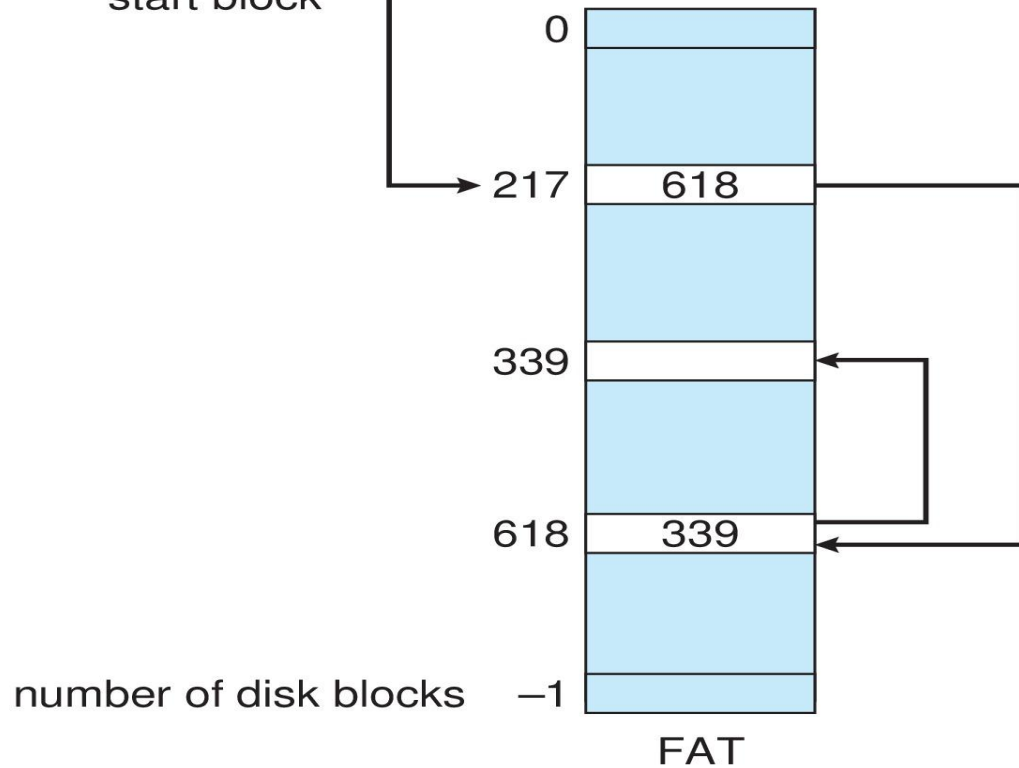
File-Allocation Table

directory entry

test	...	217
------	-----	-----

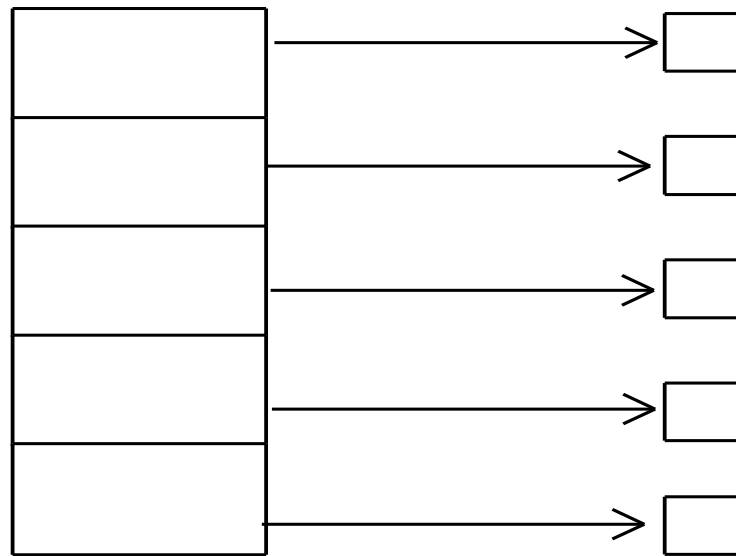
name

start block



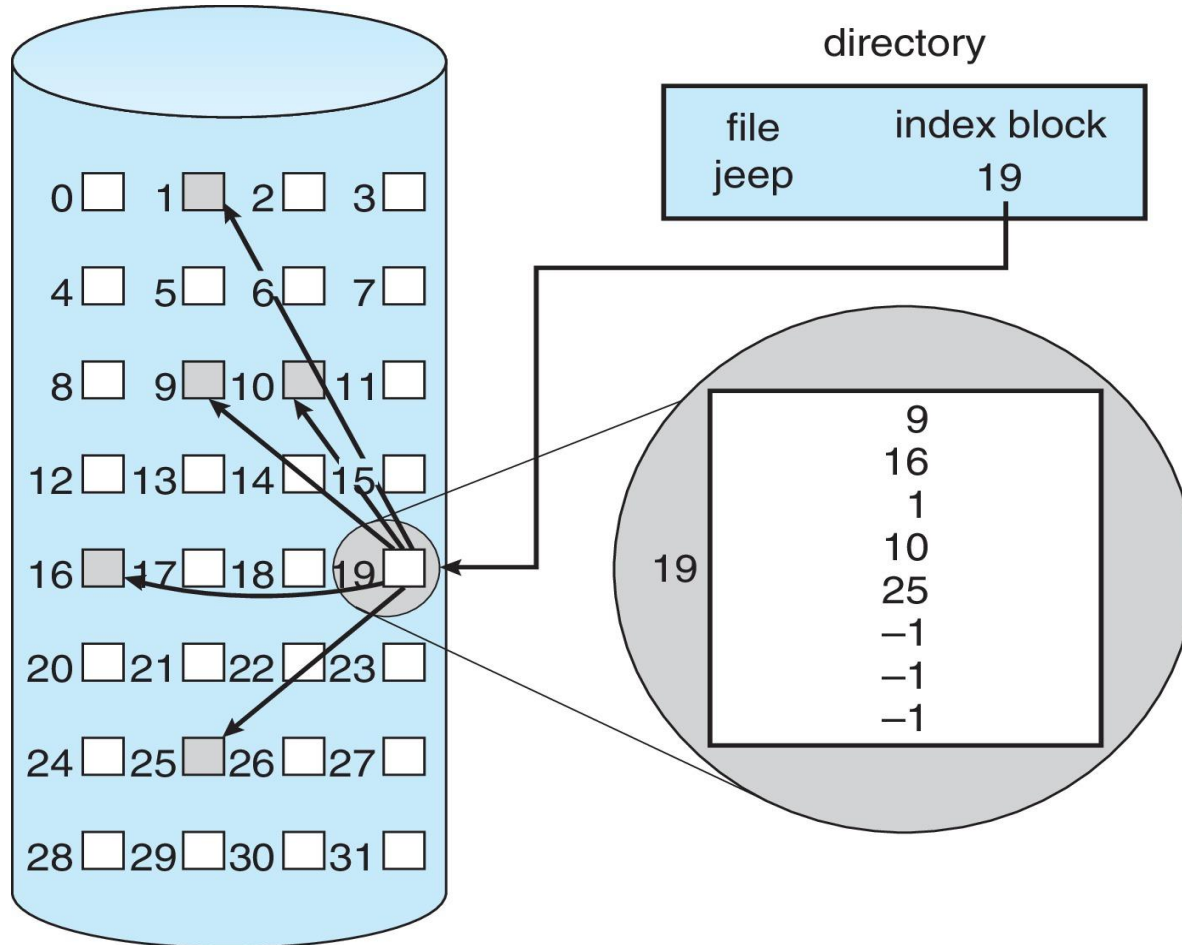
Indexed Allocation Method

- Each file has its own **index block(s)** of pointers to its data blocks
- Logical view



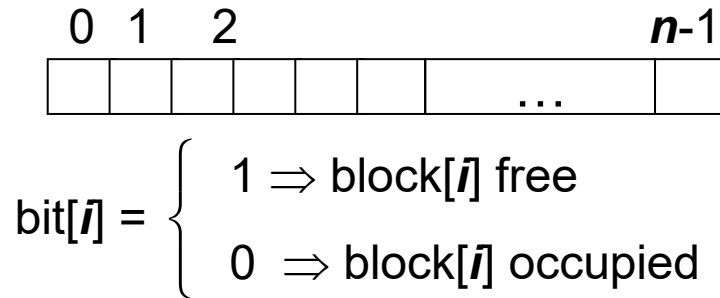
index table

Example of Indexed Allocation



Free-Space Management

- File system maintains **free-space list** to track available blocks/clusters
 - (Using term “block” for simplicity)
- **Bit vector** or **bit map** (n blocks)



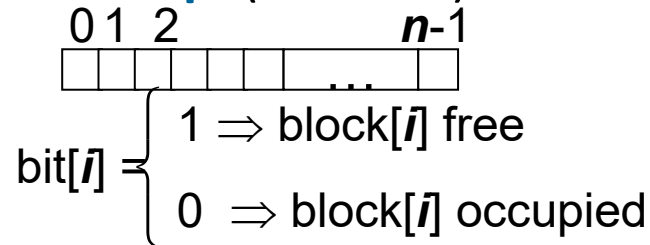
Block number calculation

(number of bits per word) *
(number of 0-value words) +
offset of first 1 bit

CPUs have instructions to return offset within word of first “1” bit

Free-Space Management

- File system maintains **free-space list** to track available blocks
- **Bit vector** or **bit map** (n blocks)



- Bit map requires extra space
 - Example:
 - block size = 4KB = 2^{12} bytes
 - disk size = 2^{40} bytes (1 terabyte)
 - $n = 2^{40}/2^{12} = 2^{28}$ bits (or 32MB)
 - if clusters of 4 blocks -> 8MB of memory
- Easy to get contiguous files

Free-Space Management (Cont.)

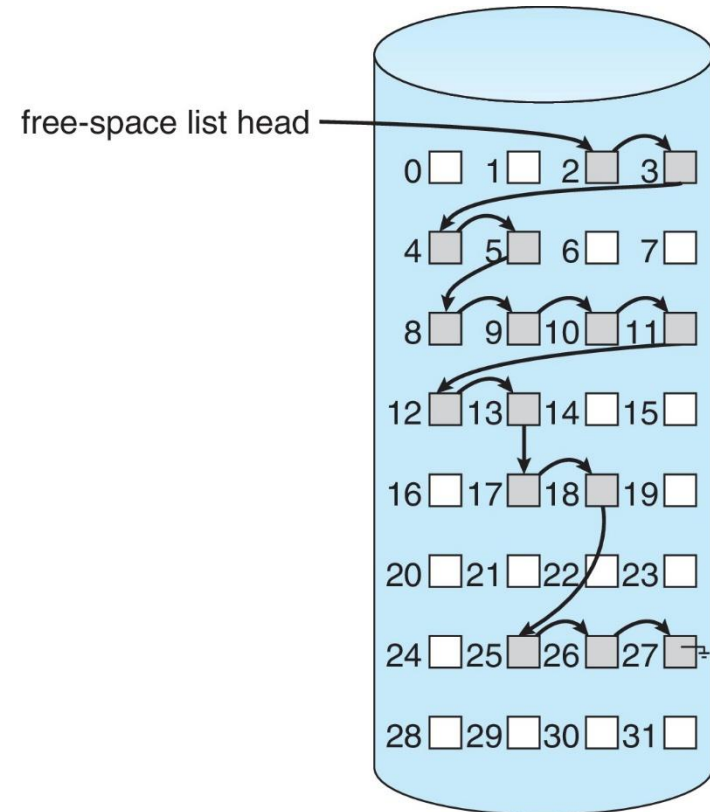
- Grouping
 - Modify linked list to store address of next $n-1$ free blocks in first free block, plus a pointer to next block that contains free-block-pointers (like this one)
- Counting
 - Because space is frequently contiguously used and freed, with contiguous-allocation allocation, extents, or clustering
 - Keep address of first free block and count of following free blocks
 - Free space list then has entries containing addresses and counts

Free-Space Management (Cont.)

- Space Maps
 - Used in **ZFS**
 - Consider meta-data I/O on very large file systems
 - Full data structures like bit maps cannot fit in memory → thousands of I/Os
 - Divides device space into **metaslab** units and manages metaslabs
 - Given volume can contain hundreds of metaslabs
 - Each metaslab has associated space map
 - Uses counting algorithm
 - But records to log file rather than file system
 - Log of all block activity, in time order, in counting format
 - Metaslab activity → load space map into memory in balanced-tree structure, indexed by offset
 - Replay log into that structure
 - Combine contiguous free blocks into single entry

Linked Free Space List on Disk

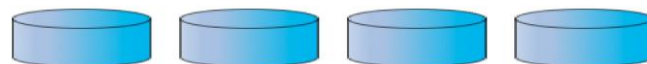
- Linked list (free list)
 - Cannot get contiguous space easily
 - No waste. Linked Free Space List on Disk of space
 - No need to traverse the entire list (if # free blocks recorded)



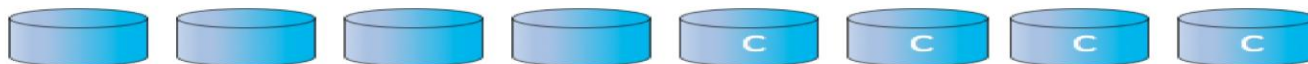
RAID Levels

- **RAID (Redundant Array of Independent)** is a method of **combining multiple physical disks** into a single logical storage unit.
- **Purpose:**
 - Improve **performance** (faster read/write).
 - Provide **fault tolerance** (data recovery in case of disk failure).
 - Increase **storage capacity**.
- **Implemented in:**
 - Hardware (RAID controllers).
- Software (OS-level RAID).

RAID Levels



(a) RAID 0: non-redundant striping.



(b) RAID 1: mirrored disks.



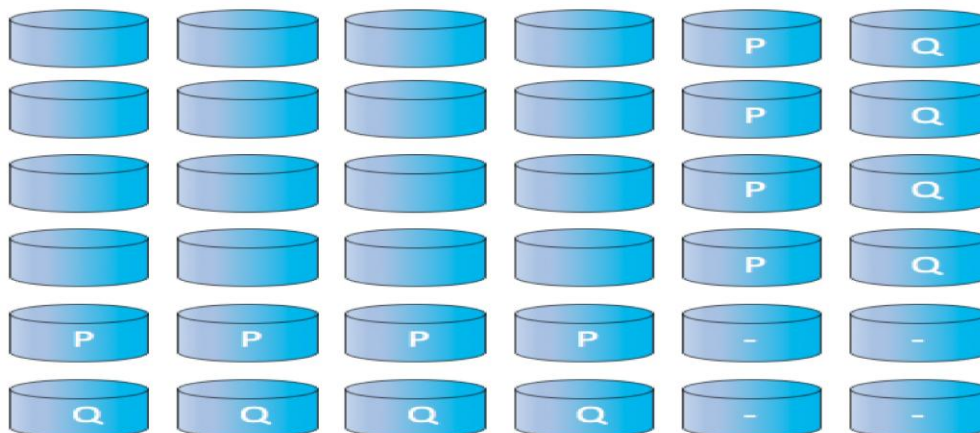
(c) RAID 4: block-interleaved parity.



(d) RAID 5: block-interleaved distributed parity.



(e) RAID 6: P + Q redundancy.



(f) Multidimensional RAID 6.

RAID Levels

RAID 0: Striping

- **Data is split (striped) across multiple disks.**
- No redundancy (if one disk fails → all data lost).
- **Performance:** High read/write speed.
- **Fault Tolerance:** None.
- **Storage Efficiency:** 100%.

Use case: Applications needing **high performance** but not critical (e.g., gaming, video editing).

RAID 1: Mirroring

- **Data is duplicated (mirrored)** across two or more disks.
- **If one disk fails, data is still safe** on the other.
- **Performance:** Faster reads (read from any disk), writes are slower (data written to both).
- **Fault Tolerance:** ✓ High (survives single disk failure).
- **Storage Efficiency:** 50% (half space used for redundancy).

Use case: Critical data storage (banking, medical records).

RAID 2: Bit-level Striping with Hamming Code

- Data striped at **bit level** with **error-correcting codes (ECC)**.
- Rarely used in practice (too complex, requires many disks).
- **Performance:** Moderate.
- **Fault Tolerance:** Good (ECC recovers errors).

Use case: Theoretical interest, not common in real systems.

RAID 3: Byte-level Striping with Dedicated Parity

- Data striped at **byte level** across disks.
- One disk stores **parity information**.
- If one disk fails, parity is used for recovery.
- **Performance:** Good for sequential access.
- **Fault Tolerance:** ✓ Yes, but parity disk is a bottleneck.
-  Use case: Applications with **large sequential reads/writes** (multimedia)

RAID 4: Block-level Striping with Dedicated Parity

- Data striped at **block level** (not bytes).
- One disk holds **parity**.
- Faster random reads than RAID 3, but parity disk still a bottleneck.
- **Performance:** Good reads, slower writes.
- **Fault Tolerance:** ✓ Yes.

Use case: Rarely used due to parity bottleneck

RAID 5: Block-level Striping with Distributed Parity

- Data and parity are **distributed across all disks**.
- No single parity disk → avoids bottleneck.
- **Performance:** Good read speed, better write speed than RAID 4.
- **Fault Tolerance:** ✓ Yes (can survive one disk failure).
- **Storage Efficiency:** $(N-1)/N$, where N = number of disks.

Use case: **Most popular RAID** – used in servers, databases, and file systems

RAID 6: Block-level Striping with Double Distributed Parity

- Similar to RAID 5, but stores **two sets of parity**.
- Can survive **two disk failures**.
- **Performance:** Read speed good, write speed slower (due to extra parity calculations).
- **Storage Efficiency:** $(N-2)/N$.

Use case: Mission-critical systems needing **high reliability** (enterprise servers)

RAID 10 (1+0): Combination of RAID 1 & RAID 0

- Data is **mirrored (RAID 1)** and then striped (RAID 0).
- Provides both **high performance** and **fault tolerance**.
- **Fault Tolerance:** ✓ Yes (can survive multiple failures, as long as not both in same mirror pair).
- **Storage Efficiency:** 50%.

Use case: High-performance databases, financial systems.

RAID Level	Technique	Performance	Fault Tolerance	Storage Efficiency	Use Case
RAID 0	Striping	High	✗ None	100%	Gaming, video editing
RAID 1	Mirroring	Read ↑, Write ↓	✓ Yes (1 disk)	50%	Banking, medical data
RAID 2	Bit Striping + ECC	Moderate	✓ Yes	Low	Rarely used
RAID 3	Byte Striping + Parity	Good sequential	✓ Yes	$(N-1)/N$	Multimedia
RAID 4	Block Striping + Parity	Good reads, slow writes	✓ Yes	$(N-1)/N$	Rare
RAID 5	Block Striping + Distributed Parity	Good	✓ Yes (1 disk)	$(N-1)/N$	Servers, databases
RAID 6	Block Striping + Double Parity	Read ↑, Write ↓	✓ Yes (2 disks)	$(N-2)/N$	Mission-critical
RAID 10	Striping + Mirroring	Very High	✓ Yes	50%	Enterprise systems

Disk Scheduling Algorithm

- A **Disk Scheduling Algorithm** is a method used by the operating system to **decide the order in which pending disk I/O requests** (read or write operations) are served.
- Disk's **read/write head** must move (seek) to different tracks to access data, multiple requests may arrive simultaneously.
- Without scheduling, the head may move inefficiently, causing high **seek time** and **latency**.

Goal of disk scheduling algorithms is

- **Minimize seek time** (time to move the head to the required track)
- **Reduce rotational latency** (waiting for the right sector under the head)
- **Improve throughput** (number of I/O requests served per unit time)
- **Provide fairness** among all requests

Types Of Disk Scheduling Algorithm

- First Come First Serve
- Shortest Seek Time First
- Elevator(SCAN)
- Circular Scan(C-Scan)
- Look
- C-Look

)First Come First Serve (FCFS)

- Handle I/O requests sequentially.
- Fair to all processes.
- Approaches random scheduling in performance if there are many processes/requests.
- Suffers from global zigzag effect.

)First Come First Serve (FCFS)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53

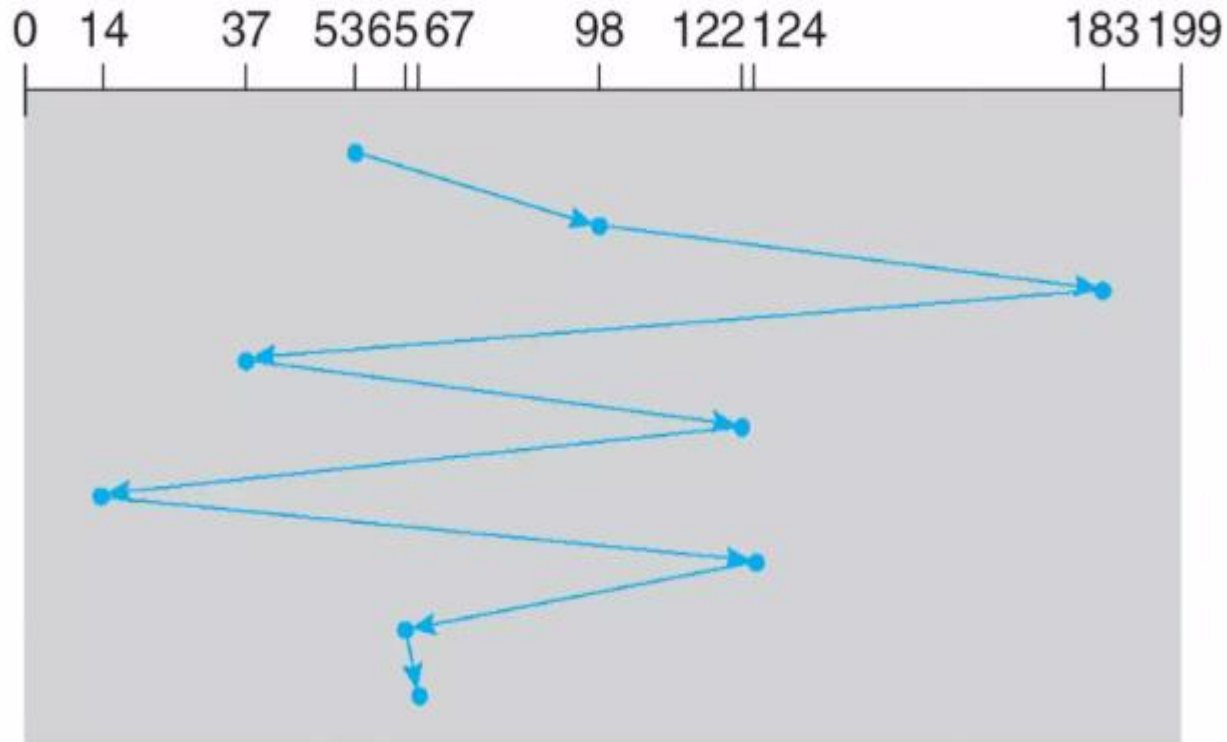


Illustration shows total head movement of 640 cylinders.

)Shortest Seek Time First (SSTF

- Selects the request with the minimum seek time from the current head position.
- Also called Shortest Seek Distance First (SSDF) – It's easier to compute distances.
- It's biased in favor of the middle cylinders requests.
- SSTF scheduling is a form of SJF scheduling; may cause starvation of some requests.

) Shortest Seek Time First (SSTF)

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53

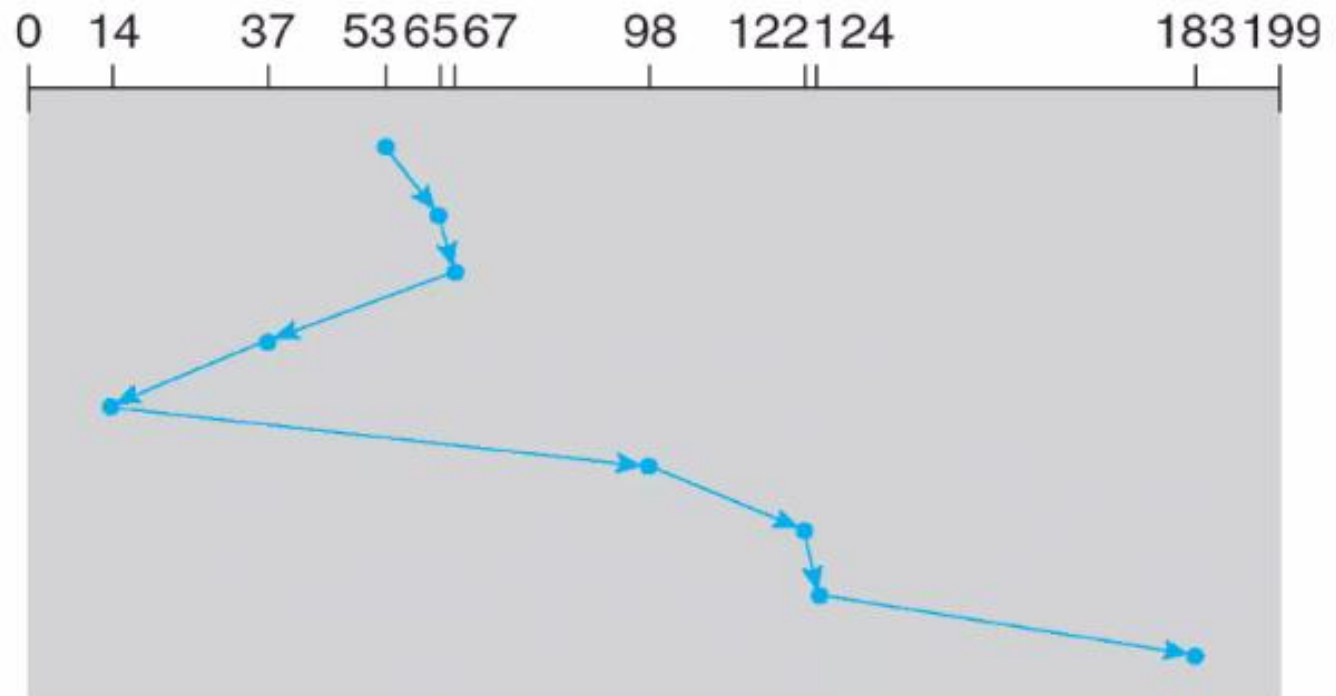


Illustration shows total head movement of 236 cylinders.

Scan

- The disk arm starts at one end of the disk, and moves toward the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.
- It moves in both directions until both ends.
- Tends to stay more at the ends so more fair to the extreme cylinder requests.

Scan

queue = 98, 183, 37, 122, 14, 124, 65, 67
head starts at 53



Illustration shows total head movement of 208 cylinders.

Look

- The disk arm starts at the first I/O request on the disk, and moves toward the last I/O request on the other end, servicing requests until it gets to the other extreme I/O request on the disk, where the head movement is reversed and servicing continues.
- It moves in both directions until both last I/O requests; more inclined to serve the middle cylinder requests.

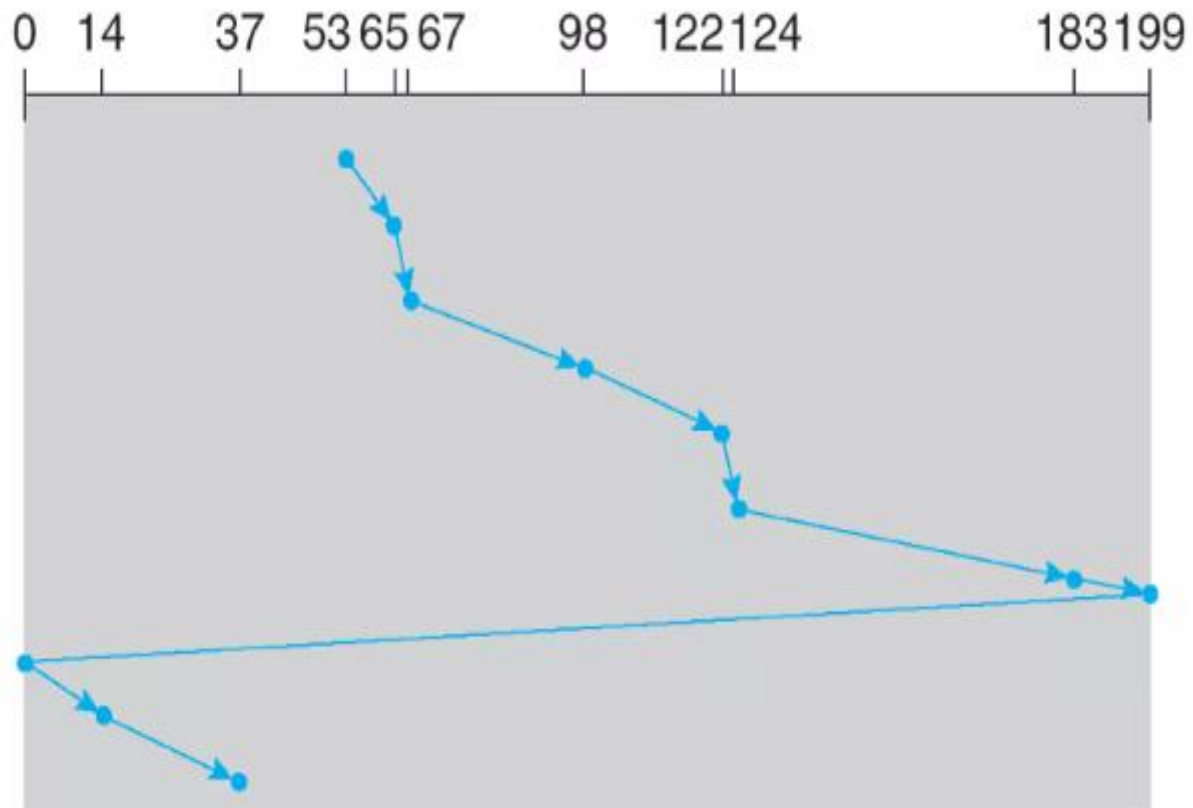
C-Scan

- The head moves from one end of the disk to the other, servicing requests as it goes. When it reaches the other end, however, it immediately returns to the beginning of the disk, without servicing any requests on the return trip.
- Treats the cylinders as a circular list that wraps around from the last cylinder to the first one.
- Provides a more uniform wait time than SCAN; it treats all cylinders in the same manner.

C-Scan

queue = 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



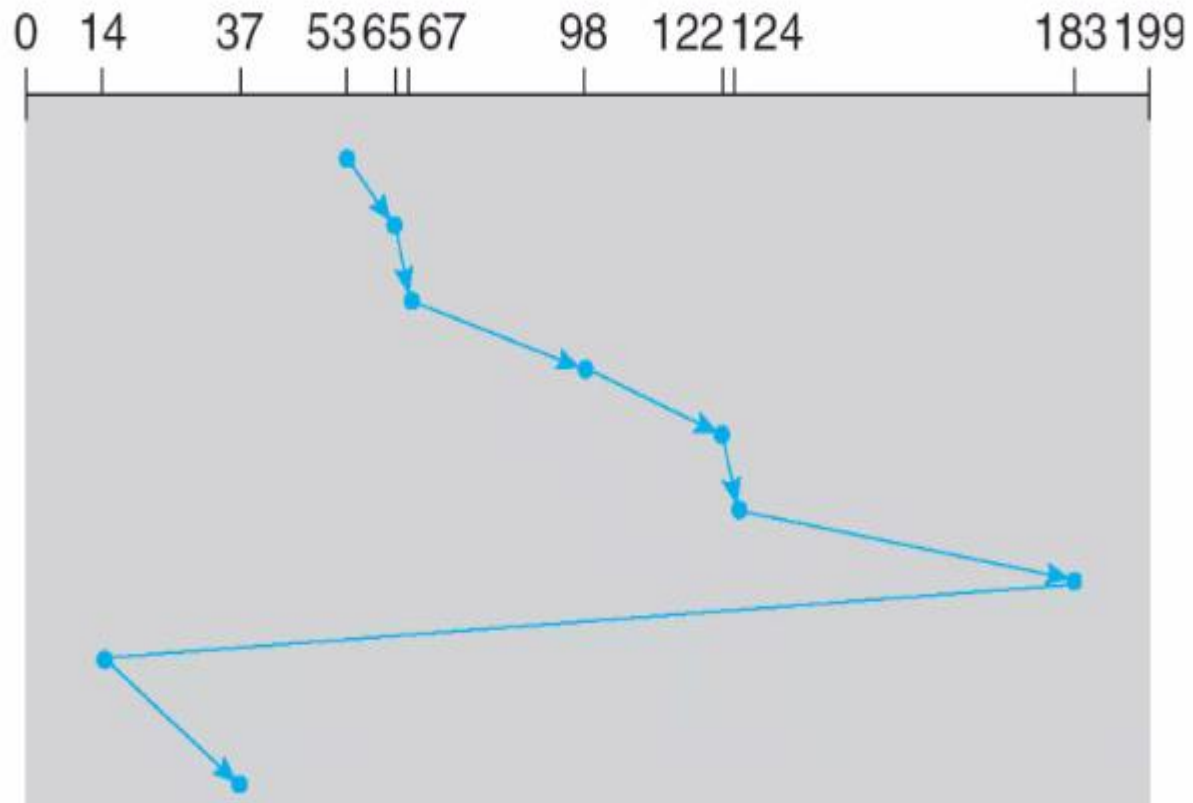
C-Look

- Look version of C-Scan.
- Arm only goes as far as the last request in each direction, then reverses direction immediately, without first going all the way to the end of the disk.
- In general, Circular versions are more fair but pay with a larger total seek time.
- Scan versions have a larger total seek time than the corresponding Look versions.

C-Look

queue 98, 183, 37, 122, 14, 124, 65, 67

head starts at 53



Elevator Algorithms

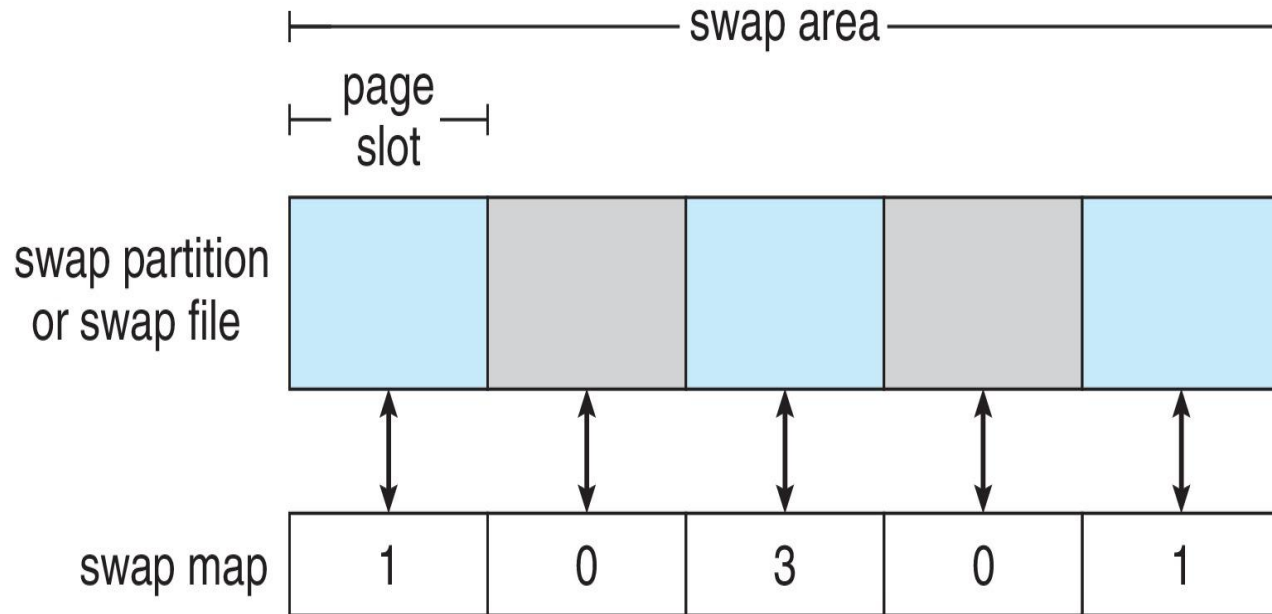
- Algorithms based on the common elevator principle.
- Four combinations of Elevator algorithms:
 - .Service in both directions or in only one direction –
 - .Go until last cylinder or until last I/O request –

Go until Direction	Go until the last cylinder	Go until the last request
Service both directions	Scan	Look
Service in only one direction	C-Scan	C-Look

Swap-Space Management

- Used for moving entire processes (swapping), or pages (paging), from DRAM to secondary storage when DRAM not large enough for all processes
- Operating system provides **swap space management**
 - Secondary storage slower than DRAM, so important to optimize performance
 - Usually multiple swap spaces possible – decreasing I/O load on any given device
 - Best to have dedicated devices
 - Can be in raw partition or a file within a file system (for convenience of adding)
 - Data structures for swapping on Linux systems:

Swap-Space Management



Stable Storage in OS

- **Stable storage** is a form of data storage that **survives system crashes and failures** without losing data.
- It ensures that once data is written, it **remains intact** even after hardware/software failures.

Purpose:

- Provides a **reliable way** to store critical system information.
- Used in situations where **data consistency and durability** are essential (e.g., databases, transaction systems).

Techniques to Achieve Stable Storage:

- **Redundancy**
 - Data is stored in multiple locations.
 - Even if one copy is corrupted, the other can be recovered.
- **Mirroring (Disk Duplication)**
 - Each write is made on two disks.
 - If one disk fails, the other holds the data.
- **Write-ahead Logging (WAL)**
 - Before modifying data, changes are written into a log.
 - Ensures recovery to a consistent state after crashes.
- **Error-Detection & Correction**
 - Techniques like **checksums, parity bits, ECC codes** are used.
 - Detects corruption and corrects errors.

Tertiary Storage Structure in Operating System

- **Tertiary storage** is a **low-cost, high-capacity storage** that is not directly accessed by the CPU.
- It is slower compared to **primary (RAM)** and **secondary storage (Hard disks, SSDs)**.
- Typically used for **backup, archival, and large-scale data storage**.

Characteristics:

- High capacity (can store terabytes or petabytes).
- Low cost per bit.
- Slow access speed (compared to RAM/Hard Disk).
- Usually **removable or external storage media**.

Examples of Tertiary Storage:

- **Magnetic Tape Storage**
 - Used for **backup and archival**.
 - Sequential access device (slow random access).
- **Optical Discs (CDs, DVDs, Blu-ray)**
 - Removable, portable storage.
- **Magnetic Disks / Removable Cartridges**
 - Sometimes used as tertiary storage.
- **Cloud Storage (Modern Perspective)**
 - Functions as tertiary storage for large datasets.

Storage Hierarchy (Speed vs Capacity):

- Primary Storage (Fastest, Expensive, Small) → RAM, Cache
- Secondary Storage (Moderate Speed, Cheaper) → HDD, SSD
- **Tertiary Storage (Slowest, Cheapest, Huge)** → Tapes, Optical, Cloud

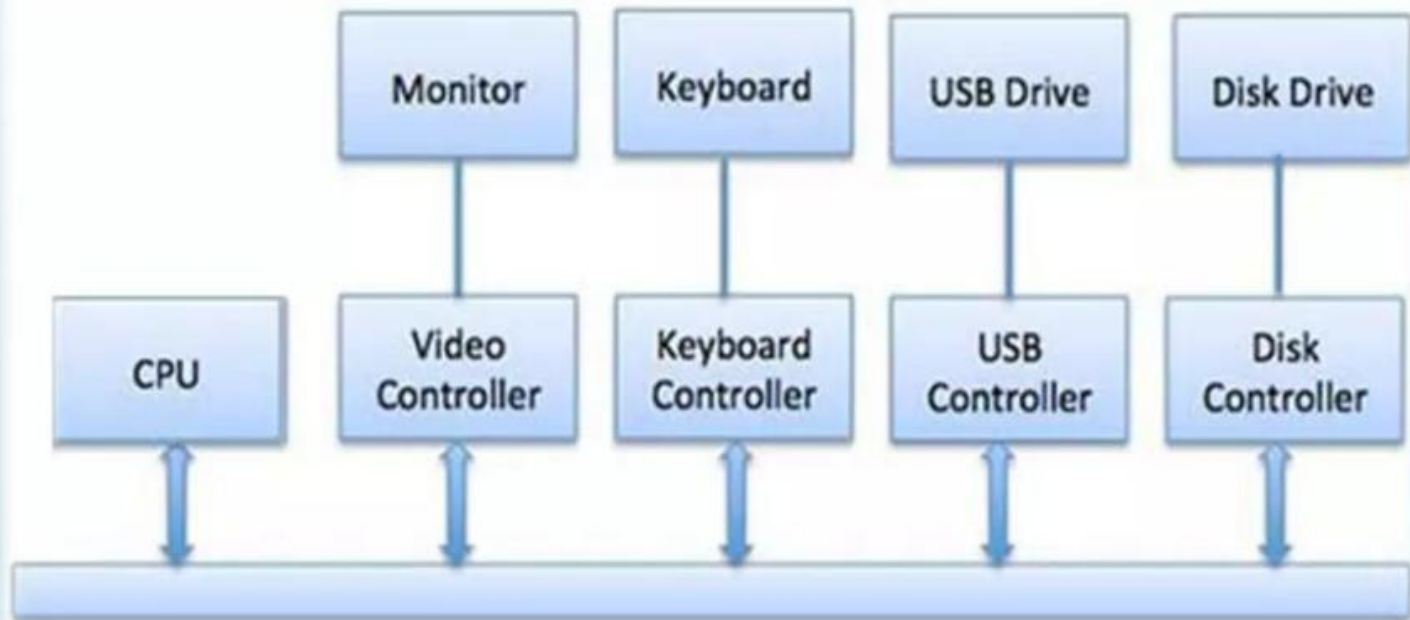
UNIT-3

Operating Systems & System Programming

(23ACA06)

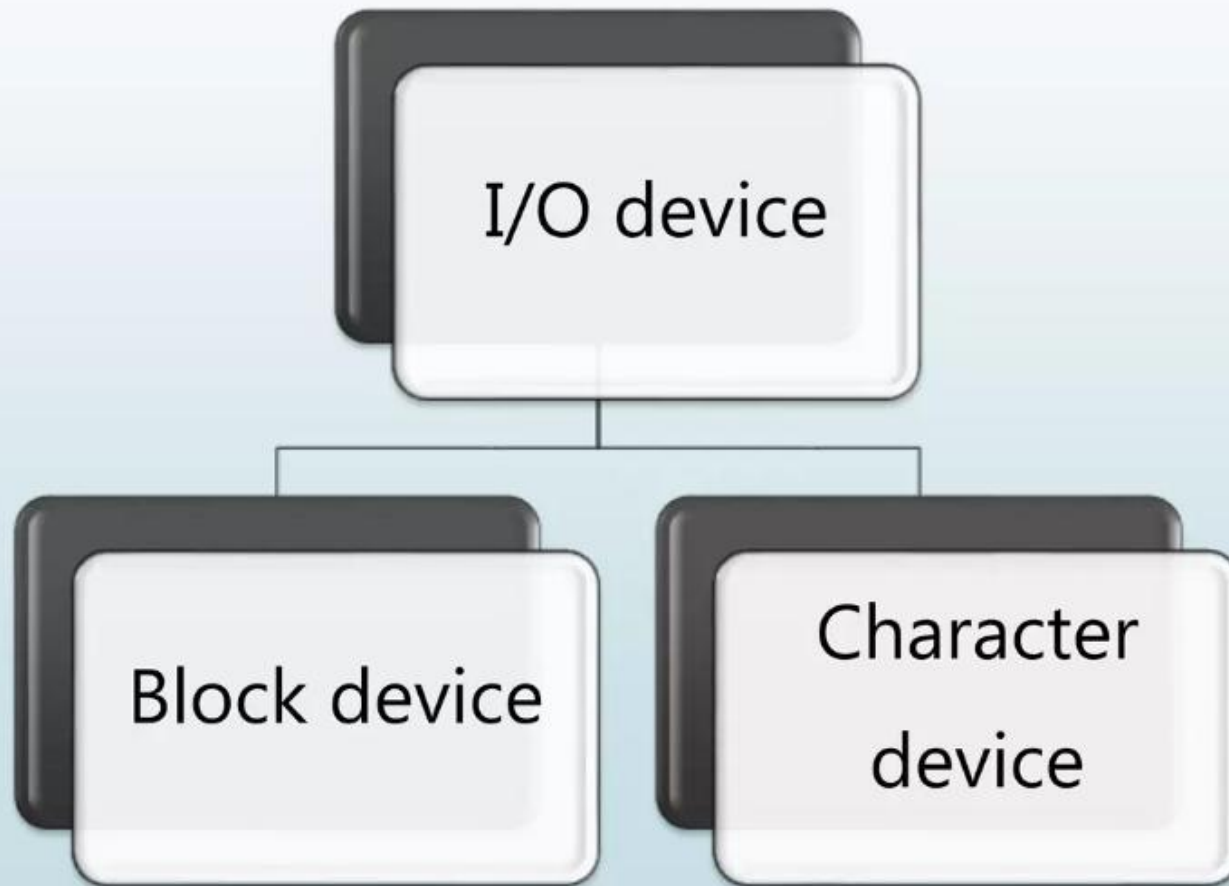
I/O Devices

- I/O devices are the pieces of hardware used by a human (or other system) to communicate with a computer. For instance, a keyboard or computer mouse is an input device for a computer, while monitors and printers are output devices.



I/O Devices- Two Categories

- I/O device are divided into two categories:



Device Controllers

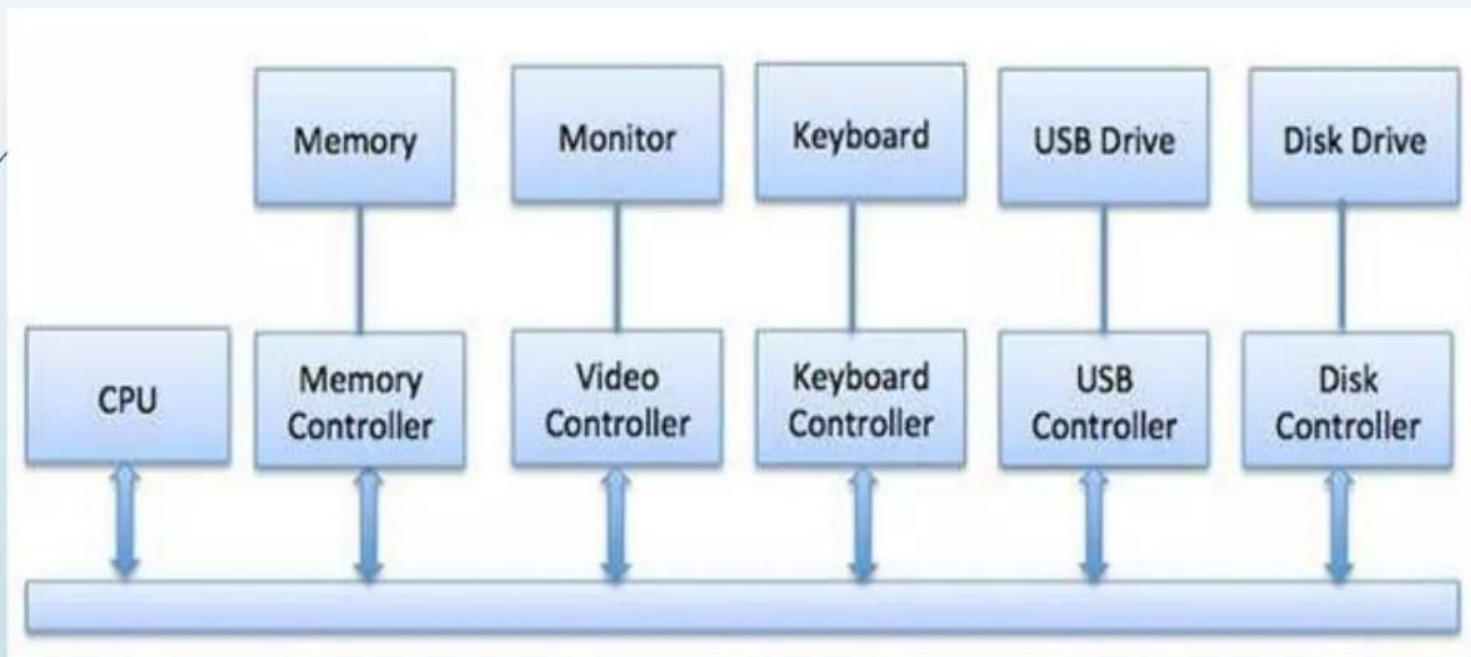
- I/O units typically consist of a mechanical component and an electronic component.
- The Device Controller works like an interface between a device and a device driver. I/O units (Keyboard, mouse, printer, etc.) typically consist of a mechanical component and an electronic component where electronic component is called the device controller.

Cont...

- Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller.
- Following is a model for connecting the CPU, memory, controllers, and I/O devices where CPU and device controllers all use a common bus for communication.

Cont...

- Graphically representation of device controller



Direct Memory Access

- Direct Memory Access (DMA) means CPU grants I/O module authority to read from or write to memory without involvement.
- DMA module itself controls exchange of data between main memory and the I/O device.
- CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.

DMA

- Slow devices like keyboards will generate an interrupt to the main CPU after each byte is transferred. If a fast device such as a disk generated an interrupt for each byte, the operating system would spend most of its time handling these interrupts.
- So a typical computer uses direct memory access (DMA) hardware to reduce this overhead.

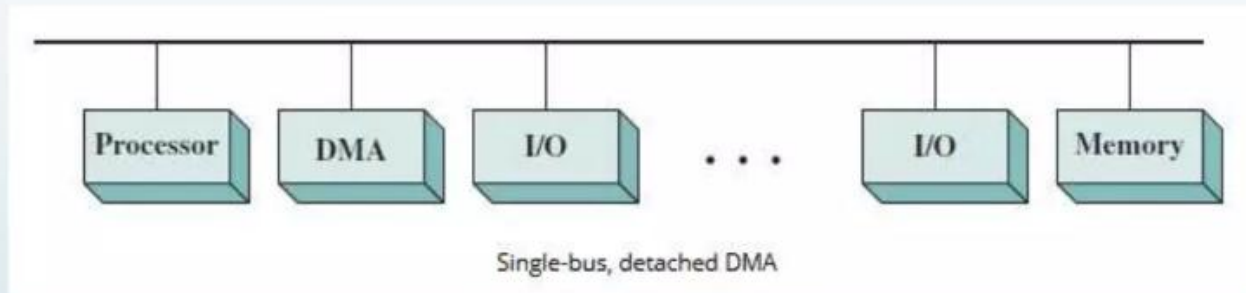
DMA Controller

- Direct Memory Access needs a special hardware called DMA controller (DMAC) that manages the data transfers and arbitrates access to the system bus. The controllers are programmed with source and destination pointers (where to read/write the data), counters to track the number of transferred bytes, and settings, which includes I/O and memory types, interrupts and states for the CPU cycles.

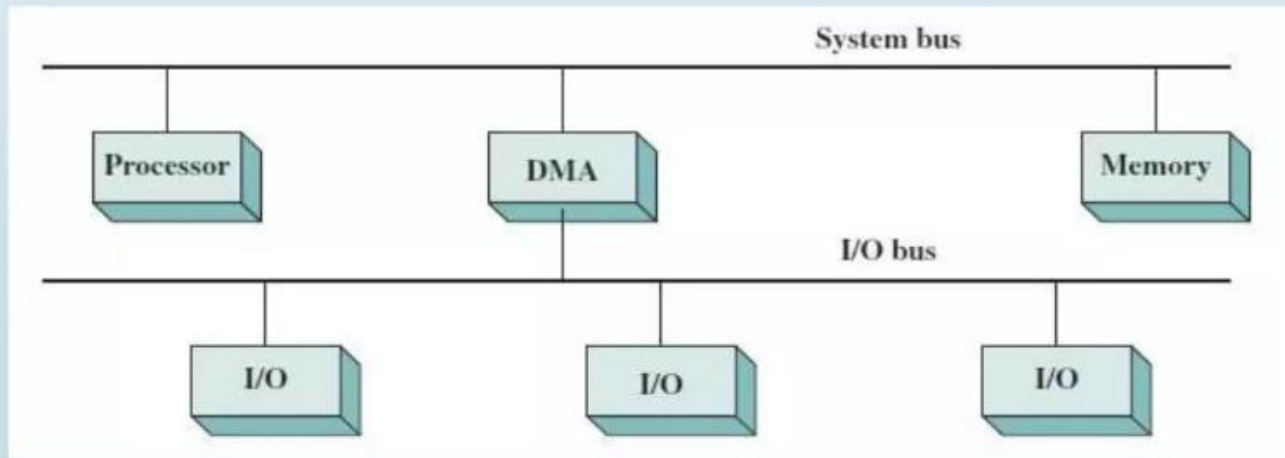
Configuration of DMA

➤ Two Configuration For DMA Transfer:-

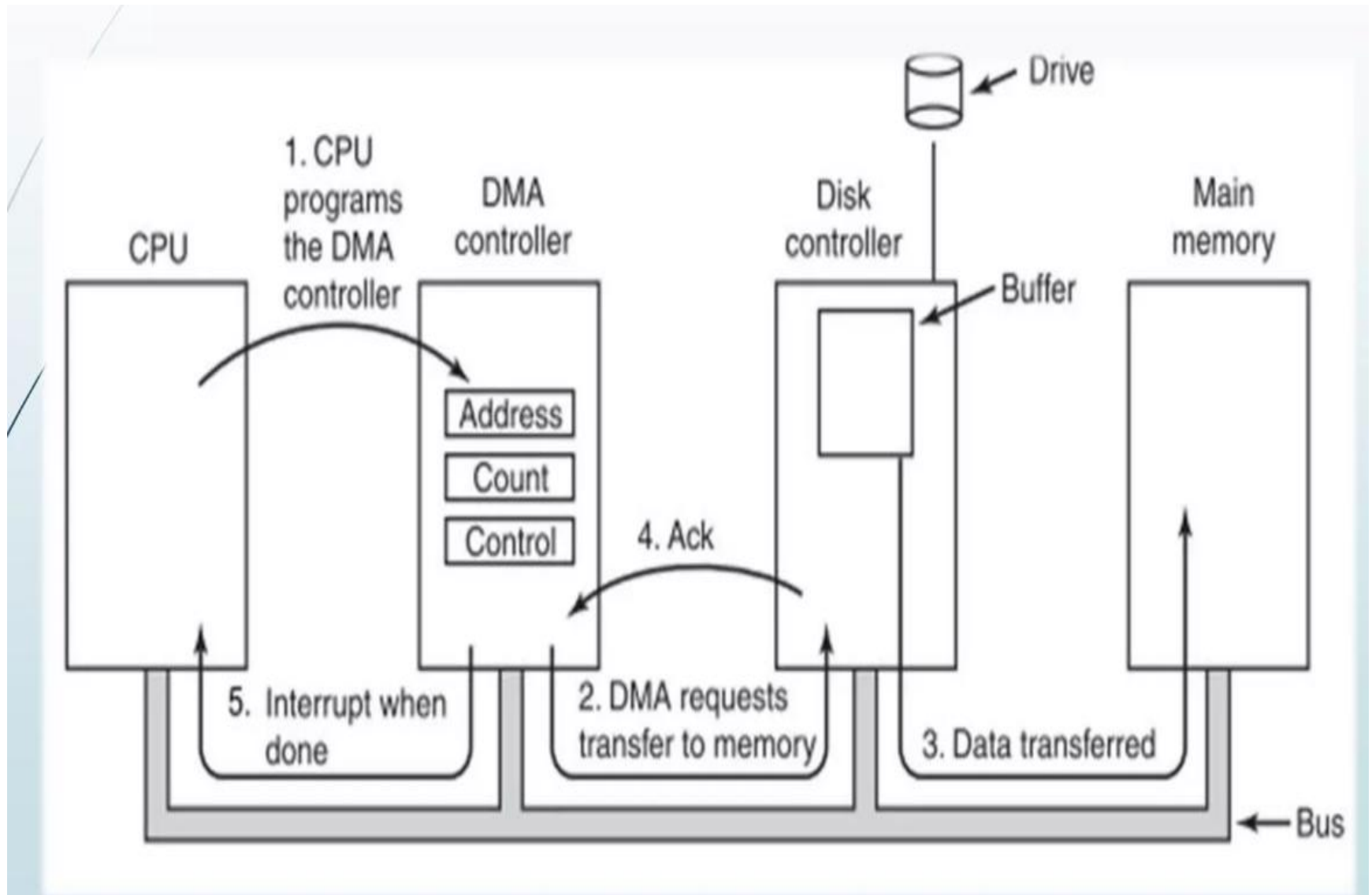
1. Single Bus



2. I/O Bus



DMA Controller (Single BUS)



DMA Controller (I/O BUS)

5. DMA controller transfers bytes to buffer, increases the memory address, decreases the counter C until C becomes zero.

6. When C becomes zero, DMA interrupts CPU to signal transfer completion.

1. Device driver is told to transfer disk data to buffer at address X.

2. Device driver tells Disk Controller to transfer C bytes from disk to buffer at address X.

DMA/bus/
interrupt
controller

CPU

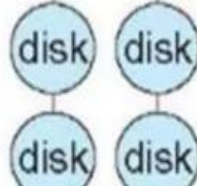
cache

CPU memory bus

memory^x buffer

PCI bus

IDE disk
controller



3. Disk Controller Initiates DMA Transfer

4. Disk Controller sends each byte to DMA controller.

DMA Mode Of Operations

1. Word-at-a-Time Mode

- In this mode, the DMA controller requests the transfer of one word and gets it.
- If the CPU also wants the bus, it has to wait.
- The mechanism is also called “**cycle stealing**” because the device controller sneaks in and steals an occasional bus cycle from the CPU once in a while, delaying it slightly.

2. Block Transfer Mode

- In block transfer mode the DMA is programmed to transfer a block and does not pause or halt until the whole block is transferred irrespective of the requests received meanwhile.
- The DMA retains control of the memory bus for the duration of the memory transfer.
- The negative aspect of operating in this mode is that the CPU will not be able to access the memory bus until the DMA controller completes the memory transfer.

3. Transparent Mode

- In transparent mode, the DMA controller transfers data only when the CPU is performing operations that do not use the system buses.
- The primary advantage of transparent mode is that the CPU never stops executing its programs and the DMA transfer is free in terms of time, while the disadvantage is that the hardware needs to determine when the CPU is not using the system buses, which can be complex.

Input/Output (I/O) Software

- **I/O System:** Manages all input and output operations between the computer and external devices (disk, printer, keyboard, display, etc.).
- **I/O Software:** A set of system software components that provide a framework for I/O operations.
- **Goals:**
 - Provide abstraction (hide hardware details).
 - Ensure efficiency (fast data transfer).
 - Guarantee protection (controlled access).

Goals of I/O Software

- **Device Independence** → Applications should work with any device without modification.
- **Uniform Naming** → All devices accessed in a standard way (**e.g., Linux treats devices as files**).
- **Error Handling** → Consistent error reporting across devices.
- **Buffering** → Handle speed mismatch between CPU and devices.
- **Concurrency** → Support multiple processes accessing I/O simultaneously.
- **Portability** → Applications portable across systems.

Device-independent I/O Software

- **I/O Software:** Provides mechanisms for communication between CPU, memory, and peripheral devices.
- **Device-independent I/O software** abstracts hardware differences and offers a **uniform interface** for applications.
- Ensures **portability, ease of programming, and system efficiency.**

Goals of Device-Independent I/O Software

- **Uniformity:** Same operations for all devices (e.g., read, write).
- **Portability:** Applications do not depend on specific device hardware.
- **Error Handling:** Provides consistent error detection and reporting.
- **Buffering:** Smooths out speed differences between devices and CPU.
- **Naming:** Provides a consistent naming scheme (file names, device paths).
- **Protection:** Ensures controlled access to devices.

Functions of Device-Independent I/O Software

- **Uniform Naming of Devices**

- Devices are identified using symbolic names (e.g., /dev/disk1).
- Hides hardware-specific details like device numbers.

- **Device Protection**

- Restricts access to devices using permissions (**read, write, execute**).
- Prevents multiple processes from conflicting over device use.

- **Blocking/Non-blocking Operations**

- **Blocking:** Process waits until I/O completes.
- **Non-blocking:** Process continues execution, I/O happens in the background.

Goals of Device-Independent I/O Software

1. Buffering

- Temporary storage of data during transfer.
- Types:
 - **Single Buffering:** One buffer between device and process.
 - **Double Buffering:** Two buffers for overlapping computation and I/O.
 - **Circular Buffering:** Multiple buffers arranged in a ring.

Goals of Device-Independent I/O Software

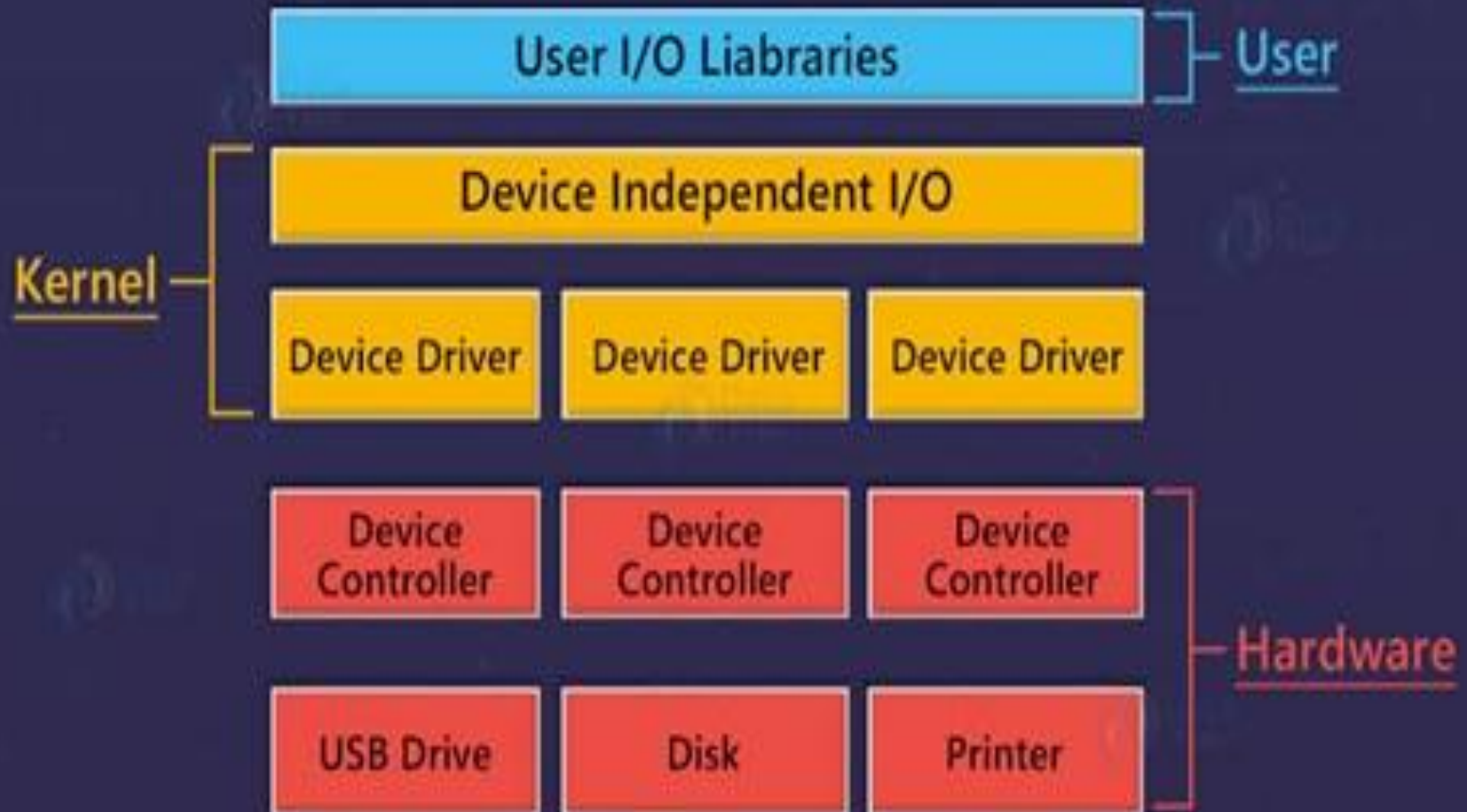
2. Error Handling

- Reports errors in a consistent **way (timeouts, device failures)**.
- Allows retry or recovery without exposing device-specific details.

3. Device-Independent Access

- Provides system calls like **open(), read(), write(), close()** applicable to files, disks, or terminals.

Levels of I/O Software



Layers of I/O Software

(a) User-Level I/O Software

- Part of application programs.
- Uses standard libraries (e.g., printf, scanf, fopen, fread).
- Simplifies programmer's work.

(b) Device-Independent I/O Software

- Provides uniform interface to all devices.
- Functions: Naming, protection, buffering, error reporting.

(c) Device Drivers

- Device-specific code in the OS kernel.
- Converts generic OS requests into device commands.
- Example: Printer driver, graphics driver.

(d) Interrupt Handlers

- Part of OS that responds to hardware interrupts.
- Alerts CPU when I/O operation completes.
- (e) Hardware Layer
- Actual I/O devices and controllers.
- Examples: Disk, keyboard, display, network card.

I/O Techniques

- **Programmed I/O**

- CPU actively waits and checks device status.
- Simple but wastes CPU time.

- **Interrupt-Driven I/O**

- Device interrupts CPU when ready.
- More efficient than polling.

- **Direct Memory Access (DMA)**

- Special controller transfers data between device and memory without CPU intervention.
- CPU is free for other tasks.

Device Drivers

- **Device Driver:** A software module that acts as an intermediary between the **operating system** and a **hardware device**.
- **Purpose:** To provide the OS with a standard interface to communicate with hardware without needing to know device-specific details.
- **Example:** Printer driver, graphics card driver, keyboard driver.

Role of Device Drivers

- **Hardware Abstraction**
 - Hides device-specific details from the OS.
 - Provides a uniform API for applications and the kernel.
- **Command Translation**
 - Converts OS/system calls (like read, write) into device-specific commands.
- **Communication Management**
 - Transfers data between I/O devices and main memory.
- **Interrupt Handling**
 - Handles device-generated interrupts and notifies the OS when tasks are completed.
- **Error Detection & Handling**
 - Detects device errors and reports them in a standardized format.

Functions of Device Drivers

- **Initialization:** Configure device settings at startup.
- **Device Control:** Send commands to the device.
- **Data Transfer:** Move data to/from device and memory.
- **Status Monitoring:** Report current device status to OS.
- **Synchronization:** Manage concurrent access requests.

Types of Device Drivers

- **Character Device Drivers**

- Handle character-based I/O (one byte/character at a time).
- Examples: Keyboard, mouse, serial ports.

- **Block Device Drivers**

- Handle block-based data transfer (fixed-size blocks).
- Examples: Hard disk, SSD, CD-ROM.

- **Virtual Device Drivers**

- Do not correspond to physical devices; instead, emulate a device in software.
- Example: Virtual printers, virtual network adapters.

Device Driver Architecture in OS

- **Application Programs** → request I/O via system calls (open, read, write).
- **OS I/O Subsystem** → passes request to appropriate driver.
- **Device Driver** → translates request to device-specific commands.
- **Hardware Device** → performs the actual operation and sends status back.

Interrupt Handlers

- When an I/O device needs attention (e.g., data is ready to be read), it sends an interrupt signal to the CPU, which triggers the appropriate interrupt handler to perform the necessary actions.

Hardware:

- This is the physical layer, including the actual devices and their controllers, which the device drivers interact with directly.

Security Environment

- A **Security Environment** in OS defines the **framework of policies, mechanisms, and tools** that protect the system against threats.
- It provides a **safe execution environment** for processes, users, and applications by ensuring **confidentiality, integrity, and availability**.

1. Components of Security Environment

- The OS security environment consists of multiple layers:

❑ User-Level Security

- **Authentication:** Verifies identity of users (passwords, biometrics, tokens).
- **Authorization:** Defines what an authenticated user can access (files, processes, devices).
- **User Roles:** Admin, normal users, and guest accounts with different privileges.

❑ Process-Level Security

- **Isolation:** Processes are separated to prevent interference.
- **Memory Protection:** Prevents one process from accessing another process's memory.
- **Execution Modes:** User Mode vs. Kernel Mode ensures limited access.

❑ File and Data Security

- **File Permissions:** Read, Write, Execute (e.g., Unix rwx model).
- **Encryption:** Protects sensitive files and communications.
- **Access Control Lists (ACLs):** Define fine-grained access rights.

❑ Network Security

- **Firewalls:** Prevent unauthorized external access.
- **Intrusion Detection/Prevention:** Detect and block malicious activities.
- **Secure Protocols:** HTTPS, SSH, VPNs for secure communication.

❑ System Monitoring and Auditing

- **Audit Logs:** Record user and system activities.
- **Intrusion Detection Systems (IDS) and Antivirus.**
- **System Updates and Patches:** Prevent exploitation of vulnerabilities

2. Security Environment Goals

- **Confidentiality** – Prevent unauthorized access to data.
- **Integrity** – Ensure data accuracy and prevent tampering.
- **Availability** – Guarantee system resources are accessible to authorized users.
- **Accountability** – Record user actions for auditing.
- **Non-repudiation** – Ensure users cannot deny their actions.

3. Threats to Security Environment

- **Internal Threats:** Privilege abuse, careless users.
- **External Threats:** Hackers, malware, phishing.
- **System Threats:** Worms, viruses, Trojan horses.
- **Network Threats:** Denial-of-Service (DoS), spoofing, packet sniffing

4. Mechanisms to Maintain Security Environment

- **Authentication Mechanisms**
 - Passwords, biometrics, smart cards, multi-factor authentication.
- **Access Control Models**
 - DAC (Discretionary Access Control)
 - MAC (Mandatory Access Control)
 - RBAC (Role-Based Access Control)
- **Cryptography**
 - Data encryption (AES, RSA) and digital signatures.

Cont...

- **System Hardening**

- Disabling unused ports/services, applying patches, enforcing security policies.

- **Monitoring & Auditing**

- Logging, intrusion detection, and auditing for suspicious activities.

Security Design Principles

Security Design Principles

- The design of an OS to make it secure, reliable, and resistant to attacks.
- Its ensure that system resources are protected against **unauthorized access, misuse, and failures**.
- These principles were first formalized by **Saltzer and Schroeder**

Principle of Least Privilege

- Each process, program, or user should have **only the minimum access rights** needed to perform its task.
- Reduces damage in case of compromise.

Example: Normal users cannot install system software; only administrators can.

Principle of Fail-Safe Defaults

- Default state should **deny access**, and explicit permission must be given to allow access.
- Prevents accidental access to resources.

Example: File permissions in Linux default to restricted unless granted

Principle of Economy of Mechanism

- Security design should be kept **simple and small**.
- Easier to analyze, verify, and implement.

Example: Unix “rwx” permission model is simple and effective.

Principle of Complete Mediation

- Every access request should be checked for authorization.
- No caching or bypassing of security checks.

Example: Each file access in modern OS goes through the kernel’s access control checks

Principle of Open Design

- Security should **not depend on secrecy of design or implementation**, but on secrecy of **keys/passwords**.
- Promotes trust and allows peer review.

Example: OpenSSL is publicly available but secure because of cryptographic keys.

Principle of Separation of Privilege

- Access to critical resources should require **multiple conditions** or approvals.
- Reduces single point of failure.

Example: Two-factor authentication (password + OTP).

Principle of Least Common Mechanism

- Minimize sharing of resources among multiple users/processes.
- Prevents unintended information leakage.

Example: Separate user environments in multi-user OS.

Principle of Psychological Acceptability

- Security mechanisms should be **easy to use and understand**.
- If too complex, users may bypass or disable them.

Example: GUI-based security settings in Windows vs complex command-line settings

User Authentication

Authentication is the process of **verifying the identity of a user** before granting access to system resources.

- It ensures that only **legitimate users** can access files, applications, and system services.
- Plays a key role in **OS security environment** along with **authorization and access control**.

Goals of User Authentication

- **Verify Identity** – Confirm that the user is who they claim to be.
- **Prevent Unauthorized Access** – Block attackers or unauthorized users.
- **Accountability** – Record actions of authenticated users.
- **Enable Authorization** – Authentication is the first step before assigning permissions.

Methods of User Authentication

1. Password-Based Authentication

- Most common and simplest method.
- **Weakness:** Password guessing, brute force.
- **Countermeasures:** Strong pwds, hashing, pwd policies.

2. Multi-Factor Authentication (MFA)

- Uses **two or more authentication factors** for stronger security.
- **Example:** ATM access requires **CARD + PIN**.

Methods of User Authentication

3. Token-Based Authentication

- Uses hardware/software tokens that generate temporary codes (OTP).
- Example: RSA tokens, Google Authenticator.

4. Biometric Authentication

- Uses unique physical/behavioral traits.
- **Examples:** Fingerprint scanners, face unlock, retina scans.
- **Weakness:** Privacy issues, expensive, false acceptance/rejection rates.

5. Certificate-Based Authentication

- Uses **digital certificates** and public key cryptography.
- Common in secure networks, corporate systems.

6. Single Sign-On (SSO)

- User logs in **once** and gains access to multiple applications without re-authentication.
- **Example:** Google Account login across Gmail, Drive, YouTube.

Protection Mechanisms

- In OS, protection mechanisms ensure that I/O devices and data are accessed securely and correctly.
- Protection is required to **prevent unauthorized access, avoid data corruption, and provide controlled sharing** among processes.
- Since I/O devices are shared resources, improper or malicious use can crash the system or compromise security.

Objectives I/O Protection

- **Prevent unauthorized access** to devices.
- **Protect data integrity** during read/write operations.
- **Ensure controlled sharing** among processes.
- **Provide fault isolation** so that one faulty process does not affect others.
- **Enable safe execution** of user programs by restricting direct access to hardware

Protection Mechanisms

(a) Dual Mode Operation

- CPU operates in two modes:
 - **User Mode:** User programs run with limited privileges.
 - **Kernel (Supervisor) Mode:** OS runs with full privileges.

(b) Access Control

- Controls which users/processes can access which devices.
- Achieved using:
 - **Access Control Lists (ACLs)** → define permissions for each device.
 - **Capability Lists** → specify what rights a process has.
- Prevents accidental or malicious misuse of devices.

(c) Memory Protection in I/O

- Ensures that I/O devices cannot overwrite memory of other processes.
- Achieved using:
 - **Base and Limit Registers** → restricts memory access range.
 - **I/O Mapping** → isolates I/O address space from main memory.

(d) I/O Privileged Instructions

- Certain I/O-related instructions (e.g., IN, OUT) are **privileged**.
- Executed only in **kernel mode** to avoid misuse.
- Example: A user program cannot directly send commands to disk controller

(e) Device Access Control

- OS uses **Device Drivers** to control access.
- Drivers implement:
 - **Queuing** (to schedule multiple requests).
 - **Buffering** (to prevent data loss).
 - **Error handling** (to avoid system crashes).

(f) Interrupt Handling Protection

- Interrupts are used for device communication.
- OS ensures:
 - Only **authorized interrupt handlers** run in kernel mode.
- Malicious code cannot replace handlers

(g) File System Protection

- Since most I/O deals with files, protection is extended to file systems:
 - **Read/Write/Execute permissions.**
- **Owner, group, and public access control**

Protection Domain

- Protection domain defines the **set of resources (objects) and access rights** available to a process.
- It provides a **boundary of authority** for processes, ensuring they access only the devices/files they are permitted to.
- Especially important for **I/O systems** since devices (disk, printer, network, etc.) are shared among users and must be protected.

Concept of Protection Domain

Domain = (Objects + Rights)

- Each process executes within a **domain of protection**, which specifies:
 - **Objects:** Files, I/O devices, memory segments.
 - **Rights:** Operations allowed on those objects (read, write, execute, control).

Examples

- Printer domain → Process has **write** rights (send data to print), but no **read** rights.
- Disk domain → Process has **read/write** access to files but no right to issue a **format command**.
- Terminal domain → Process may **read input** and **write output**, but not **control device settings**

Types of Domains

- **User Domain**

- Limited privileges.
- Example: User can print a file but cannot directly control the printer hardware.

- **Kernel Domain**

- Full privileges.
- Can execute all I/O instructions and control devices directly.

Domain Switching

- A process may need to **switch domains** to perform certain operations (e.g., a user process making a system call to access a disk).
- Switching ensures:
 - Protection from **unauthorized access**.
 - Controlled access to **I/O devices** only through the OS.

Protection Matrix Representation

- Protection can be modeled using a **matrix**:
 - Rows → Domains (processes/users).
 - Columns → Objects (files, devices).
 - Entries → Rights.

Importance in I/O System

- Prevents one process from misusing or damaging devices.
- Ensures **safe sharing** of devices among multiple users.
- Isolates faulty processes so that they do not corrupt global I/O resources.

Access Control List (ACL)

- **ACL** is a protection mechanism used to specify which **users or processes can access a resource (object)** and what operations they are allowed to perform.
- Widely applied to **I/O systems** (disks, printers, terminals, files, etc.) to prevent unauthorized access.
- **ACL is associated with each object** (device, file, directory) that specifies.

Structure of ACL

Each entry in an ACL contains:

- **User/Process ID**
- **Rights/Permissions** (e.g., Read, Write, Execute, Control)

ACL in I/O System

- ACLs control **which processes can interact with I/O devices and how.**

Examples:

- Disk → Admin has **Read/Write**, users have **Read only**.
- Printer → Faculty and students can **Write (print jobs)**, only admin can **Control (cancel jobs, configure printer)**.
- Network device → Admin can **Control**, normal users can **Send/Receive**.

-

Advantages of ACL

- **Fine-grained control** → Different rights for different users.
- **Flexibility** → Supports multiple types of access (read, write, execute, control).
- **Security** → Prevents unauthorized or malicious use of I/O devices.
- **Transparency** → Easy to check permissions for each object.

Disadvantages of ACL

- **Complexity** → Large systems may have long ACLs (hard to manage).
- **Performance Overhead** → Checking ACLs may slow down access.
- **Revocation Difficulty** → Removing rights for many users can be tedious.

Development of Unix/Linux

- **Unix** and **Linux** are foundational operating systems in computing history.
- They influenced many modern OSes like BSD, macOS, Android, and even Windows subsystems.
- Known for **multi-user**, **multi-tasking**, **portability**, and **open-source philosophy**.

Origins of Unix

- **1969** – Developed at **AT&T Bell Labs** by **Ken Thompson, Dennis Ritchie**, and others.
- Written originally in **assembly language** for PDP-7.
- Later rewritten in **C language (1973)** → made it **portable** across hardware platforms.
- Introduced key concepts:
 - Hierarchical **file system**
 - **Pipes** and **filters**
 - **Shell** as a command interpreter
 - **Process control** and multitasking

Evolution of Unix

- **1970s – BSD Unix** (University of California, Berkeley):
 - Added **C shell (csh)**, networking (TCP/IP), virtual memory.
- **1980s – Commercial Unix:**
 - AT&T System V, SunOS, HP-UX, AIX.
- **1990s – Fragmentation:**
 - Many versions existed, leading to the **POSIX standard** for compatibility.

Linux Development Timeline

- **1991** – Linux 0.01 released (basic kernel, limited functionality).
- **1992** – Linux licensed under GPL → community contributions.
- **1994** – Linux 1.0 (first stable release).
- **2000s** – Popular for servers, supercomputers.
- **2008 onwards** – Android (Linux-based) dominates mobile OS market.
- **Today** – Linux powers >90% of supercomputers, most servers, IoT devices, and cloud systems.

Features of Unix/Linux

- **Unix:**
 - Stable, multi-user, security-focused.
 - Proprietary (in most forms).
- **Linux:**
 - Open-source, free, community-driven.
 - Highly customizable (distros like Ubuntu, Fedora, Red Hat).
 - Portable across hardware (PCs, servers, embedded devices).

Unix vs Linux (Comparison Table)

Feature	Unix	Linux
Origin	AT&T Bell Labs, 1969	Linus Torvalds, 1991
Licensing	Mostly proprietary	Open-source (GPL)
Portability	Limited, vendor-specific	High portability, wide hardware
Cost	Expensive (commercial)	Free & open-source
Usage	Servers, mainframes	Servers, desktops, mobiles, IoT

Role of Kernel in Unix/Linux

- The **Kernel** is the **core component** of Unix/Linux operating systems.
- Acts as a **bridge** between hardware and user-level applications.
- Runs in **privileged mode** (kernel mode) with full control over system resources.

Types of Kernel Architectures

- **Monolithic Kernel** (Unix, traditional Linux):
 - All core services (process, memory, file system, device drivers) run in kernel space.
 - High performance, but large code base.
- **Microkernel** (MINIX, Mach):
 - Minimal kernel functions (only essential services).
 - Other services run in **user space**.
 - More modular, but slower due to context switching.
- **Hybrid Kernel**:
 - Mix of both (e.g., modern macOS, Windows NT).

Introduction to Linux Commands

- Linux is a **command-line-based operating system** where users interact with the system using commands.
- Each command has a **specific function** such as creating files, managing directories, monitoring processes, or configuring the system.
- Commands are typed into a **shell/terminal** and usually follow this format:

File and Directory Management Command

- **pwd (Print Working Directory)**
 - Displays the current directory path.
 - Example: pwd
- **ls (List files and directories)**
 - Lists files and directories in the current folder.
 - Options:
 - ls -l → long format
 - ls -a → includes hidden files
- **cd (Change Directory)**
 - Changes the working directory.
 - Example:
 - cd Documents → go to Documents
 - cd .. → move one step back

Cont...

- **mkdir (Make Directory)**
 - Creates a new directory.
 - Example: mkdir myfolder
- **rmdir (Remove Directory)**
 - Removes an empty directory.
 - Example: rmdir myfolder
- **touch (Create File)**
 - Creates an empty file.
 - Example: touch file.txt
- **cat (Concatenate / View File)**
 - Displays file contents.
 - Example: cat file.txt

Cont...

- **cp (Copy File/Directory)**
 - Copies files or directories.
 - Example: `cp file.txt copy.txt`
- **mv (Move/Rename)**
 - Moves or renames a file.
 - Example: `mv file.txt newfile.txt`
- **rm (Remove File)**
 - Deletes a file.
 - Example: `rm file.txt`

System Information Commands

- **whoami** – Displays the current logged-in user.
- **uname** – Displays system information (uname -a for detailed info).
- **date** – Displays current date and time.
- **cal** – Shows calendar of the current month.
- **clear** – Clears the terminal screen.

File Viewing and Editing

- **nano filename** – Opens file in Nano text editor.
- **vi filename** – Opens file in Vi editor.
- **less filename** – Views large files page by page.

User and Permission Management

- **who** – Shows logged-in users.
- **id** – Displays user ID (UID) and group ID (GID).
- **chmod** – Changes file permissions. Example:
chmod 755 file.sh
- **chown** – Changes ownership of a file. Example:
chown user:group file.txt

Shell Programming

- **Shell** is a **command interpreter** in Linux/Unix that allows users to interact with the operating system.
- It accepts commands from the user and sends them to the kernel for execution.
- Examples of shells:
 - **Bash (Bourne Again Shell)** – most common
 - **sh (Bourne Shell)**
 - **csh (C Shell)**
 - **ksh (Korn Shell)**

What is Shell Programming?

- Writing a **series of commands** in a file (called a **shell script**) to automate tasks.
- Shell scripts can:
 - Execute multiple commands sequentially.
 - Handle decision-making (if/else).
 - Perform looping operations(while, do-while).
 - Take input and display output.

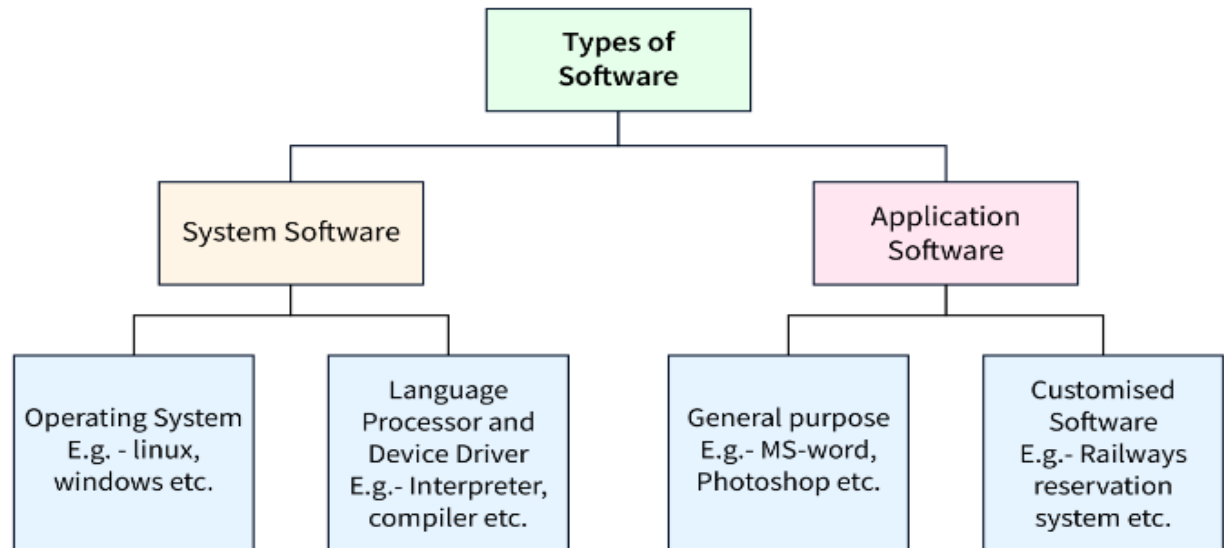
UNIT-4

Operating Systems & System Programming

(23ACA06)

Software and Software Hierarchy

- **Software** is a set of programs, procedures, and related documentation that directs the computer to perform specific tasks.
- Unlike hardware, software is intangible.
- It acts as an **interface between the user and hardware**



Types of Software

Software is classified into two main categories:

- **System Software**

- Provides the basic functionality for running the computer.
- Acts as a platform for application software.
- Examples: Operating System (Windows, Linux), Compiler, Assembler, Linker, Loader.

- **Application Software**

- Programs designed for end users to perform specific tasks.
- Examples: MS Word, Photoshop, Web browsers, Database applications.

System Software

- System software can be further divided into:
- **Operating System (OS)**
 - Manages hardware and software resources.
 - Examples: Windows, Linux, macOS.
- **Language Processors**
 - Convert high-level or assembly code into machine code.
 - Types:
 - **Assembler** – Converts assembly language to machine code.
 - **Compiler** – Translates high-level language into machine code (whole program).
 - **Interpreter** – Translates line by line.

Software Hierarchy

Software can be viewed as a **hierarchy** (layered structure):

Level 1: Hardware

Physical components of the computer system.

Example: CPU, Memory, I/O devices.

Level 2: System Software

Controls and manages hardware.

Includes Operating System, Language Translators, Utility Programs, Device Drivers.

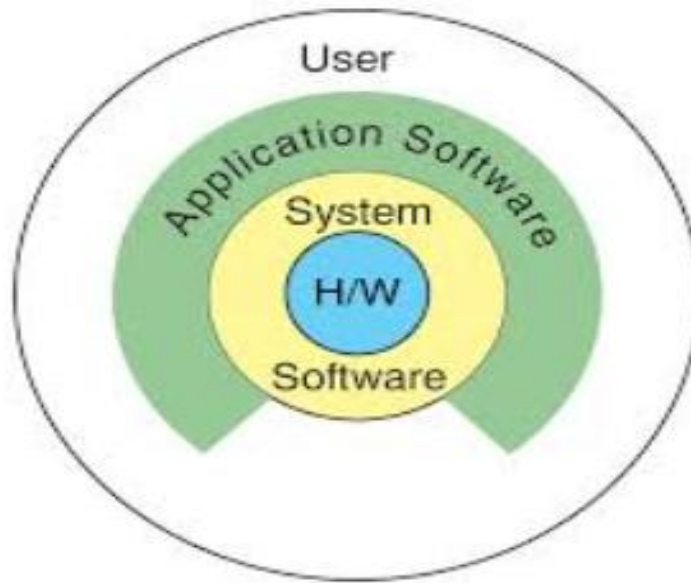
Level 3: Application Software

Built on top of system software.

Provides user-oriented functionalities.

Examples: Word processors, Games, Web browsers.

Cont...



Level 4: User

End users interact with applications to perform tasks.

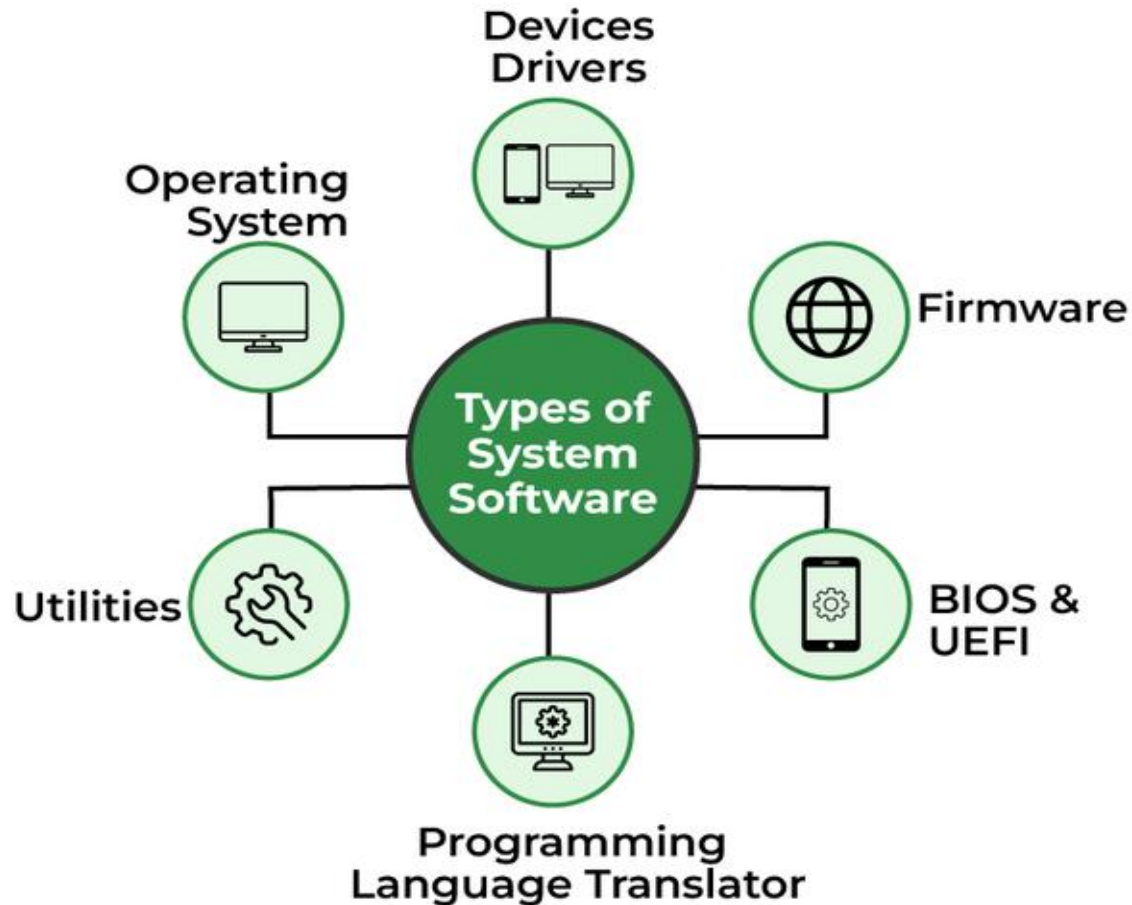
- **Utility Programs**
 - Perform system maintenance tasks.
 - Examples: Antivirus, Disk cleaners, File compression tools.
- **Device Drivers**
 - Enable communication between OS and hardware devices.

Systems Programming In System Software

Systems programming involves building the low-level, high-performance programs that **manage resources, provide fundamental services**, and create functional computing environment.

- **Purpose:** To create software platforms and services for other software to run on, focusing on efficient resource use.

Core Components of System programming



Role of systems programming

Systems programming provides the core components system software layer.

- **Operating Systems (OS):** The entire OS, such as Linux or Windows, is a product of systems programming.
- It manages all of the computer's resources, including the CPU, memory, and file storage, and schedules the processes of other software.
- **Device Drivers:** These are special programs that enable the OS to communicate and control specific hardware, such as a printer, graphics card, or network adapter.

Role of systems programming

- **Firmware:** Embedded software like a computer's BIOS or UEFI(Unified Extensible Firmware i/f) is a result of systems programming.
- It provides the low-level control **needed to initialize hardware** when the computer starts.
- **Language Processors:** Compilers, interpreters, and assemblers are system programs that **transform code from one language to another**, such as converting source code into machine-executable instructions.

Role of systems programming

- **Utility Programs:** System utilities perform maintenance and optimization tasks that ensure the smooth functioning of the computer.
- Examples include disk management tools, backup software, and antivirus programs.
- **Networking Software:** Systems programming builds the network protocols and utilities that enable communication between different systems and over the internet

The process of systems programming

- **Low-level hardware access:** The developer works closely with the hardware, writing code that directly interacts with the memory, CPU registers, and I/O devices.
- **Efficient resource management:** A primary goal is to achieve the most efficient use of system resources, often in environments with limited power or memory.

The process of systems programming

- **Creation of abstractions:** System programs provide API, which is a high-level interface for application developers to use.
- This hides the complexity of the underlying hardware.
- **Enforcement of system-wide rules:** The OS, created through systems programming, enforces system-wide rules for memory management, process scheduling, and security to prevent applications from interfering with one another

Machine Structure

- Machine Structure defines how the **computer hardware components** are organized and how **software interacts** with them.
- **Machine structure** helps system software (like compilers, assemblers, loaders, and operating systems) efficiently control and utilize hardware resources.

Levels of Machine Structure

Level	Name	Description
Level 0	Digital Logic Level	Consists of electronic circuits — gates, flip-flops, registers. Executes micro-operations.
Level 1	Microprogram Level	Implements the machine instructions using microinstructions stored in control memory.
Level 2	Instruction Set Architecture (ISA)	Machine language instructions that the processor understands (add, move, jump, etc.).
Level 3	Operating System Level	Provides system calls and manages resources like CPU, memory, and I/O.
Level 4	Assembly Language Level	A symbolic form of machine instructions that can be directly translated by an assembler.
Level 5	High-Level Language Level	Includes programming languages (C, Java, Python, etc.) that are translated into assembly/machine code by compilers or interpreters.

Components of Machine Structure

- **Central Processing Unit (CPU)**
 - Performs computations and controls all operations.
 - Contains **ALU (Arithmetic Logic Unit)**, **Control Unit**, and **Registers**.
- **Memory Unit**
 - Stores data and instructions.
 - Divided into **primary memory (RAM, cache)** and **secondary storage (hard disk)**.
- **Input/Output Units**
 - Facilitate communication between user and computer (keyboard, monitor, printer, etc.).
- **System Bus**
 - Connects CPU, memory, and I/O devices through **data**, **address**, and **control buses**.

For Example,

in an Intel x86 architecture:

- Machine instructions like **MOV**, **ADD**, and **JMP** belong to the **ISA level**.
- These are executed by hardware controlled by **microinstructions** at the **microprogram level**.
- High-level language code (**sum = a + b;**) is translated by the **compiler** to these machine instructions.
-

Interface

- Interface is shared boundary across which **two or more separate components of a computer** system exchange information, commands, or control signals.

Characteristics of Interfaces

- **Simplicity:** Easy to use and understand
- **Consistency:** Common rules for all components
- **Abstraction:** Hides internal implementation details
- **Reliability:** Ensures error-free communication
- **Efficiency:** Reduces overhead in data exchange

Types of Interfaces

Type of Interface	Description	Example
Hardware Interface	Communication between hardware components and control software.	Device drivers, I/O ports
Software Interface	Defines how different software modules communicate.	APIs, library calls
User Interface (UI)	Interaction between the user and the system.	Command-line interface (CLI), GUI
System Interface	Interaction between application programs and the operating system.	System calls (e.g., read(), write() in UNIX)

Components of Interfaces

System Software	Interface Role
Assembler	Interface between assembly language programs and machine code.
Compiler	Interface between high-level programs and assembly/machine code.
Linker	Interface between object modules and the final executable.
Loader	Interface between compiled code and memory (execution environment).
Operating System	Interface between user programs and hardware through system calls.

Address Space

- The set of valid memory addresses that a process or program can access.
- It provides a **logical view** of memory for each process, separating one program's memory from another.

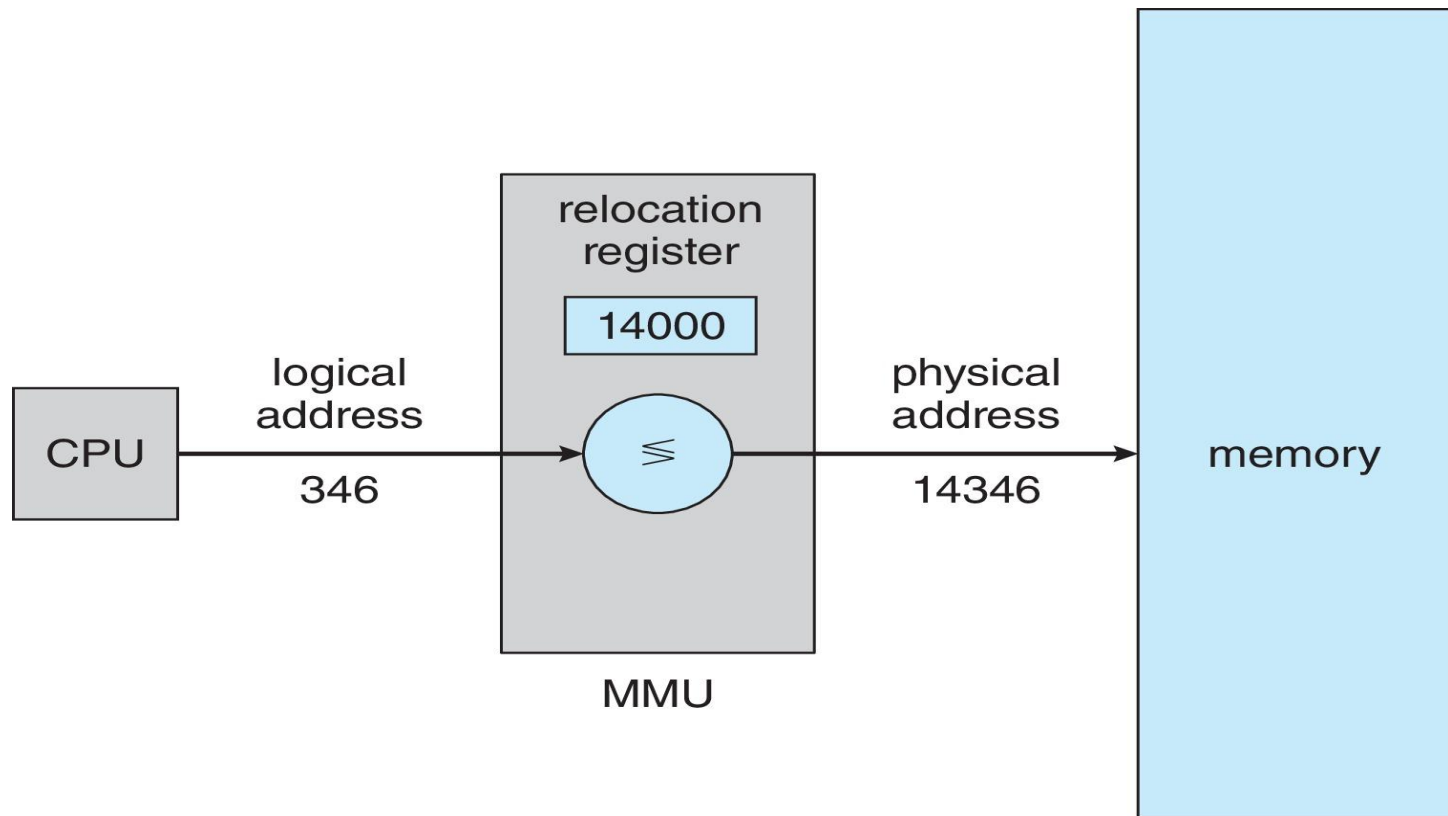
Types of Address Spaces

Type	Description
Logical Address Space	Addresses generated by the CPU (used by programs).
Physical Address Space	Actual addresses in the main memory (RAM).

- Note: The Memory Management Unit (MMU) converts logical addresses → physical addresses during execution

Address Space in System Software

- **Compiler:** Generates code with symbolic or relative addresses.
- **Assembler:** Converts symbolic names into memory addresses (relocatable code).
- **Linker:** Adjusts addresses when multiple modules are combined.
- **Loader:** Assigns final absolute memory addresses during program loading.
- **Operating System:** Manages address spaces of multiple processes for protection and multitasking.



Computer Language

- A computer language is a set of **symbols, rules, and syntax** used to write programs that instruct a computer to perform specific tasks.
- System software such as **assemblers, compilers, and interpreters** are designed to translate programs written in these languages into machine-executable code.

Types of Computer Languages

Level	Language Type	Description	Example
Level 1	Machine Language	The LLL consisting of binary codes (0s and 1s). Directly executed by hardware.	10110000 01100001
Level 2	Assembly Language	Symbolic representation of machine instructions using mnemonics. Requires an Assembler to convert to machine code.	MOV A, B
Level 3	High-Level Language (HLL)	Uses human-readable syntax; easy to learn and portable across systems. Needs a Compiler or Interpreter.	C, Java, Python
Level 4	Very High-Level / Problem-Oriented Language	Designed for specific applications like database or AI programming.	SQL, MATLAB, PROLOG
Level 5	Natural Language Interface	Allows programming or interaction using human language constructs.	Chatbots, AI-based interfaces

Evolution of Computer Languages

- **1st Generation:** Machine language (binary instructions).
- **2nd Generation:** Assembly language (mnemonics and symbolic code).
- **3rd Generation:** High-level languages (procedural, e.g., C, FORTRAN).
- **4th Generation:** Problem-oriented languages (e.g., SQL, MATLAB).
- **5th Generation:** Artificial intelligence and natural language programming (e.g., PROLOG, LISP).

Simple addition example in different language levels:

Language Level	Example Code
Machine Language	0001 0010 0011
Assembly Language	ADD R1, R2, R3
High-Level Language (C)	sum = a + b;

Advantages and Disadvantages

Level	Advantages	Disadvantages
Machine Language	Fast execution	Hard to code/debug
Assembly Language	Easier than binary, efficient	Hardware dependent
High-Level Language	Portable, readable	Needs translation, slower
Very High-Level Language	Faster development	Less control over hardware

System Software Development and Recent Trends

- System software is a collection of programs **designed to operate, control, and extend the processing capabilities** of the computer itself.
- It **serves as a bridge** between user applications and hardware.

Examples:

Operating System (OS), Compiler, Assembler, Linker and Loader. Device Drivers. And System Utilities

System Software Development Process

a) Requirement Analysis:

- Identify user and system requirements.
- Define system objectives and constraints.

b) Design Phase:

- Architecture design of system components.
- Interface and module definition.
- Use of **modular and layered design** for scalability and maintainability.

System Software Development Process

c) Implementation:

- Coding in low-level languages (like C or Assembly) for performance.
- Development of core modules: kernel, file system, process management, etc.

d) Testing:

- Unit testing of individual modules.
- Integration testing for inter-module communication.
- System testing for performance and stability.

e) Deployment & Maintenance:

- Installation of system software on target hardware.
- Regular updates and patch management

Key Components of System Software

- **Operating System (OS)** – Manages hardware and software resources.
- **Compiler & Interpreter** – Translate high-level code to machine code.
- **Assembler** – Converts assembly code to object code.
- **Linker & Loader** – Combine object files and load them into memory.
- **Device Drivers** – Interface between OS and hardware devices.
- **System Utilities** – Tools for file management, monitoring, etc.

Recent Trends

1. Cloud-Based Operating Systems

- OS functions are integrated with cloud platforms.
- Examples: Chrome OS, CloudReady.
- Features: online storage, lightweight design, centralized updates.

2. Mobile Operating Systems

- Designed for smartphones and tablets.
- Examples: Android, iOS.
- Features: touch interface, app sandboxing, power management.

3. Real-Time Operating Systems (RTOS)

- Used in embedded and IoT systems.
- Deterministic response time for tasks.
- Examples: VxWorks, QNX, FreeRTOS.

4. Virtualization and Containerization

- **Virtualization:** Running multiple OSs on a single machine using hypervisors (e.g., VMware, Hyper-V).
- **Containerization:** Lightweight OS virtualization using Docker, Kubernetes.

5. Security-Enhanced OS

- Built-in protection mechanisms (SELinux, Windows Defender).
- Emphasis on encryption, access control, and isolation.

6. Artificial Intelligence Integration

- OSs now use AI for resource prediction, adaptive scheduling, and threat detection.
- Examples: Windows Copilot, macOS AI Assistants.

7. Open Source and Modular OS

- Open-source OS development promotes transparency and community-driven innovation.

Examples: Linux, Android AOSP, FreeBSD.

8. Internet of Things (IoT) Operating Systems

- Lightweight and energy-efficient OSs for embedded devices.

Examples: RIOT OS, Zephyr, TinyOS.

9. Edge Computing and Distributed OS

- Manage computation closer to data sources (IoT edge devices).
- Focus on low-latency processing and data privacy

Programming Languages and Language Processing

Language Processor is a software system that **translates and processes programs written in programming language** into a form that can be executed by a computer.

Examples: Assembler, Compiler, Interpreter, Linker, Loader

Programming Languages is a formal language that specifies **set of instructions** that can be used to **produce various kinds of output**, usually to **control the behavior of a machine**.

Categories of Programming Languages:

Category	Description	Examples
Machine Language	Instructions in binary form, directly executed by hardware.	10100010, etc.
Assembly Language	Symbolic representation of machine code using mnemonics.	MOV, ADD, SUB
High-Level Language (HLL)	Human-readable, portable languages.	C, C++, Java, Python
Fourth-Generation Language (4GL)	More abstract, often used for database and AI programming.	SQL, MATLAB
Scripting Languages	Used for automation and web scripting.	Python, JavaScript, PHP

Language Processing System

- **Language processing system** is a collection of software tools that transform source code written in a **high-level language** into **executable machine code**.

Main Components:

- **Preprocessor**
 - Handles macros and directives before compilation.
 - Example: #include, #define in C.
- **Compiler**
 - Translates the entire source code into object code.
 - Performs optimization and error checking.

Language Processing System

- **Assembler**
 - Converts assembly language code into machine code.
- **Linker**
 - Combines multiple object files and libraries into a single executable.
- **Loader**
 - Loads the executable code into memory for execution.

Types of Language Processors

Processor Type	Function	Example
Assembler	Converts assembly language to machine code.	MASM, NASM
Compiler	Translates whole HLL program into machine code at once.	GCC, Turbo C
Interpreter	Translates and executes code line-by-line.	Python, JavaScript
Preprocessor	Processes macros and directives before compilation.	C Preprocessor
Linker & Loader	Combines and loads object modules.	ld (Linux Linker)

Example of Language Processing Flow

Source Program (C code):

```
int main() {  
    int a = 5, b = 10, c;  
    c = a + b;  
    printf("%d", c);  
    return 0;  
}
```

Processing Steps:

- **Preprocessor:** Expands #include and macros.
- **Compiler:** Converts to assembly code.
- **Assembler:** Generates object code.
- **Linker:** Links library functions (e.g., printf).
- **Loader:** Loads executable into memory and runs it.

Symbol Tables and Data Structures for Language Processing

- A **Symbol Table** is a data structure used by the **compiler or interpreter** to store and retrieve information about program entities (symbols).

Purpose:

- To record information about variables, functions, constants, objects, etc.
- To support various compiler phases such as **lexical analysis, syntax analysis, semantic analysis, and code generation.**

Information Stored in a Symbol Table

Attribute	Description
Name / Identifier	Symbol name (variable, function, etc.)
Type	Data type (int, float, char, etc.)
Scope	Local, global scope
Memory Location / Address	Location where the variable is stored
Value	Value assigned (if constant)
Parameter List	For functions and procedures
Size	Memory space required
Link Information	For referencing other symbols or declarations

Functions of a Symbol Table

- **Insertion** – Add a new name or identifier with its attributes.
- **Lookup / Search** – Find information about an existing symbol.
- **Update** – Modify attributes (e.g., change type or address).
- **Deletion** – Remove entries (e.g., when exiting a block scope).
- **Scope Management** – Handle nested blocks and scope visibility.

Implementation of Symbol Tables

1. Linear List (Sequential Search)

- Simple array or linked list.
- Each entry is checked sequentially.

2. Hash Table

- Uses a **hash function** to compute the address of a symbol.
- Fast insertion and lookup (average $O(1)$ time).
- **Collisions** handled using chaining or open addressing

Example – Symbol Table Representation

Source Code:

```
int a, b;  
float avg;  
a = 10;  
b = 20;  
avg = (a + b) / 2.0;
```


Symbol Table Example:

Name	Type	Scope	Address	Value
a	int	Global	1000	10
b	int	Global	1002	20
avg	float	Global	1004	15.0

Error Handling with Symbol Tables

- Symbol tables help detect and report errors such as:
- **Undeclared variable** (not found in symbol table)
- **Redeclaration** of a variable
- **Type mismatch** during assignment or operation
- **Scope conflicts** between local and global variables

Search and Allocation Data Structures

Search Data Structures

- Search data structures are designed to **store information** and **allow efficient retrieval** when needed.

In compilers and system software, they are mainly used in **symbol tables**, **literal tables**, and **macro tables**.

Objectives of Search Data Structures

- To **store** a large number of entries (symbols, constants, etc.) efficiently.
- To allow **fast lookup** of information (e.g., find address or attributes).
- To handle **insertions, deletions, and modifications** dynamically.
- To minimize memory usage and search time.

Common Search Data Structures

STRUCTURE	DESCRIPTION	SEARCH TIME COMPLEXITY
Linear List	Sequential search through an array or linked list. Simple but slow.	$O(n)$
Hash Table	Uses a hash function to compute the index for entries.	$O(1)$ average
Binary Search Tree (BST)	Data stored in sorted tree form, allows fast lookup.	$O(\log n)$
Balanced Trees (AVL / Red-Black)	BST that maintains height balance for faster operations.	$O(\log n)$
Trie (Prefix Tree)	Stores symbols character-by-character, useful in lexical analyzers.	$O(m)$ (m = length of

Allocation Data Structures

- **Allocation Data Structures** manage **memory allocation and deallocation** for programs during compilation, assembly, or execution.

They are,

- **Assemblers:** To allocate memory for variables, literals, and instructions.
- **Linkers and Loaders:** To assign absolute addresses to program segments.
- **Operating Systems:** For dynamic memory management and process scheduling

Objectives of Allocation Data Structures

- To **track free and occupied memory blocks** efficiently.
- To **allocate and release memory** dynamically as per program needs.
- To minimize **fragmentation** and maximize **memory utilization**.
- To maintain **address mapping** during relocation and loading

Types of Memory Allocation

Type	Description	Example Usage
Static Allocation	Memory is assigned at compile time.	Global variables in C
Stack Allocation	Memory managed using stack (LIFO).	Function calls, local variables
Heap Allocation	Memory allocated at runtime.	Dynamic memory using malloc()

Elements of Assembly Language Programming

Assembly language: is a low-level language that provides symbolic representation (**mnemonics**) of machine instructions.

Purpose:

Used to write programs that **directly interact with hardware components** and perform system-level tasks.

Assembler:

A program that translates **assembly language code into machine code** (object code) that the processor can execute

Components of Assembly Language

- An assembly program consists of several key elements:

a) Mnemonics

- Mnemonics are short, easy-to-remember symbolic codes for machine instructions.

Example:

- MOV – Move data
 - ADD – Add values
 - SUB – Subtract values
- JMP – Jump to another instruction

b) Operands

- Operands specify the data to be operated on.
- Types of operands:
 - **Immediate operand:** A constant value (e.g., MOV AX, 05H)
 - **Register operand:** Data stored in registers (e.g., MOV BX, AX)
 - **Memory operand:** Data stored in memory (e.g., MOV AX, [2000H])

c) Labels

- Labels identify memory addresses or program locations.
- Used in jumps and loops.

Example:

```
START: MOV AX, 05H
      ADD AX, 02H
      JMP START
```

d) Comments

- Comments improve readability and understanding of code.
- Denoted by ; in most assembly languages.

Example:

MOV AX, BX ; Move value of BX into AX

e) Directives (Pseudo-operations)

- Provide instructions to the assembler (not executed by CPU).

Examples:

- DATA SEGMENT – Start of data segment
- CODE SEGMENT – Start of code segment
- END – Marks the end of program
- DB – Define byte
- DW – Define word

f) Instructions

- Instructions specify operations for the CPU to perform.

Categories of Instructions:

- **Data Transfer Instructions** – MOV, XCHG, PUSH, POP
- **Arithmetic Instructions** – ADD, SUB, INC, DEC
- **Logical Instructions** – AND, OR, XOR, NOT
- **Branch Instructions** – JMP, CALL, RET, JZ, JNZ
- **Input/Output Instructions** – IN, OUT
- **String Instructions** – MOVS, CMPS, SCAS

Program Development Steps

Writing the Program:

Use mnemonics and symbolic names.

Assembling the Program:

Convert assembly code → object code using an assembler.

Linking:

Combine object files and libraries using a linker.

Loading:

Load the executable into memory.

Execution:

CPU executes instructions sequentially.

Design and Types of Assemblers

Assembler:

An assembler is a system software that translates an assembly language program into machine code (object code).

Purpose:

To convert symbolic instructions (mnemonics, labels, and directives) into binary instructions understood by the CPU.

Input: Assembly language source code (.ASM file)

Output: Object code or executable file (.OBJ / .EXE)

Program Relocation:

Adjusts address references so the program can execute correctly in any memory location.

Program Linking:

combining multiple modules into one executable.

Error Handling:

Detects syntax errors, undefined symbols, and reports them to the programmer.

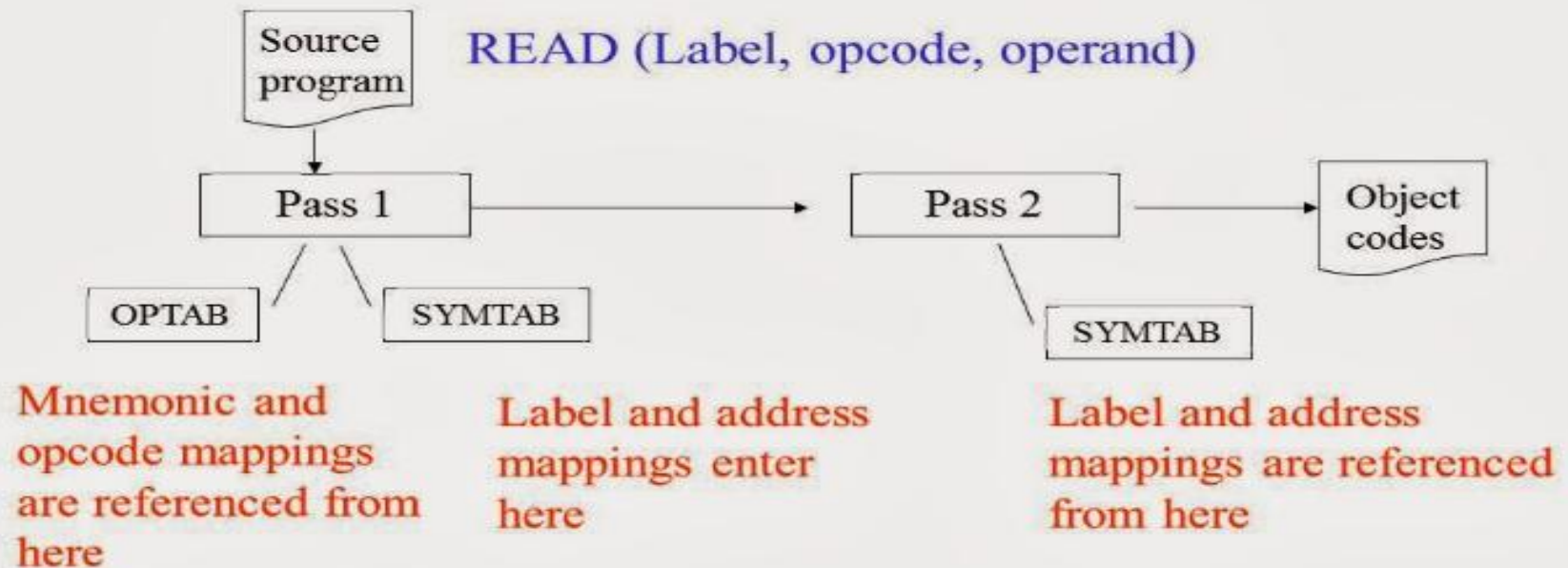
Macro Processing:

Expands macros into the corresponding sequence of instructions.

Design of Assembler

- The design of Assembler operation into two main passes **or** phases.

A Simple Two Pass Assembler Implementation



Types of Assemblers

- Assemblers are classified based on **number of passes, functions, and output handling.**

Based on Number of Passes

One-Pass Assembler

- Translates code in a single pass.
- Uses **forward reference resolution** with techniques like backpatching.
- Example: Simple microprocessor assemblers.

Two-Pass Assembler

- First pass builds symbol and literal tables.
- Second pass generates object code.
- Example: MASM (Microsoft Assembler).

1. One-Pass Assembler

- Performs translation in a single scan of the source program.
- Assigns addresses and generates object code simultaneously.

Challenges:

- Difficult to handle forward references (symbols used before definition).

Solutions:

- Uses backpatching: temporarily leaves blank fields for unresolved addresses and fills them later.

Advantages:

- Faster than two-pass assembler.
- Requires less memory.
- Disadvantages:
- Complex to design due to handling of forward references

2. Two-Pass Assembler

Pass 1 – Analysis Phase:

- Scans the source code and collects information.
- Tasks performed:
 - Assign addresses to all statements.
 - Build the symbol table.
 - Identify labels, directives, and constants.
 - Generate intermediate code.

Pass 2 – Synthesis Phase:

- Uses information from Pass 1 to generate machine code.
- Tasks performed:
 - Replace symbolic addresses with actual values.
 - Generate object code and listing.
 - Handle relocation and linking information.

Phases of Assembler Design

Phase	Description
1. Lexical Analysis	Identify tokens like mnemonics, operands, labels.
2. Syntax Analysis	Check correctness of instruction format.
3. Symbol Resolution	Build and update symbol table.
4. Opcode Translation	Convert mnemonics to machine opcodes. Eg. MOV A B
5. Address Binding	Assign memory addresses to symbols.
6. Object Code Generation	Produce final machine code.

Concept of Linking and Relocation

- When a program is written, it is often divided into several modules (files) for easier development and maintenance. Each module can be compiled separately, producing **object modules**.

However, before execution, all these modules must be combined and adjusted properly — this process involves **linking** and **relocation**.

Linking

- **Linking** is the process of combining multiple object modules and resolving all the references between them to create a single executable file.

Need for Linking:

- A large program is usually divided into smaller modules.
- One module may refer to functions or variables defined in another module.
- **Linker** resolves these inter-module references.

Functions of a Linker:

- **Combining Object Modules:**
Collects all object files and library routines into one executable program.
- **Symbol Resolution:**
Matches external references (used in one module) with their definitions (in another module).
- **Relocation:**
Adjusts addresses so that the program can be loaded anywhere in memory.
- **Library Linking:**
Links standard library routines (like printf, scanf, etc.) automatically if needed.

Types of Linking:

Static Linking:

- All modules and library routines are linked before execution.
- Produces a single large executable file.
- Faster execution, but more memory usage.

Example: Traditional .EXE files in MS-DOS.

Dynamic Linking:

- Linking occurs at runtime (during program execution).
- Uses shared libraries (DLLs).
- Saves memory and disk space.

Example: Windows uses .DLL files.

Relocation

Relocation is the process of modifying the address-dependent code and data in a program so that it can execute correctly from a different memory location.

Need for Relocation:

- The compiler assumes the program starts at address 0.
- But the actual memory location assigned during loading may be different.
- **The loader or linker adjusts** all address references accordingly.

Steps in Relocation:

- **Assign a Load Address:**

The starting address where the program will be loaded in memory.

- **Modify Address Constants:**

All address-sensitive instructions (like jumps or data accesses) are updated by adding the relocation factor.

Linking and Relocation in MS-DOS

Object Modules in MS-DOS:

- In MS-DOS, compilers (like Turbo C, MASM) generate **.OBJ** files.
- Each .OBJ file contains:
 - Code segment
 - Data segment
 - Symbol table
 - Relocation table

Relocation in MS-DOS:

- The linker calculates the **segment and offset addresses** for all variables and functions.
- It updates the relocation table entries.
- The loader (part of DOS) then loads the executable file into memory and further adjusts addresses if necessary.

Compare between Linking and Relocation

Concept	Purpose	Performed By	Example in MS-DOS
Linking	Combine object files & resolve symbols	Linker (LINK.EXE)	Linking multiple .OBJ files
Relocation	Adjust address references	Linker/Loader	Adjusting code for new memory address

Dynamic Linking

- **Dynamic Linking** is a process in which linking of programs and library routines is performed **during program execution (runtime)**, rather than before execution (**at compile time**).
- This means the **executable file contains references to shared library modules** that are loaded only when required.

Purpose of Dynamic Linking

- To **save memory** and **disk space**.
- To **allow program updates** without recompiling or relinking applications.
- To enable **sharing of code** (common library routines) among several running programs.

Working of Dynamic Linking

Step-by-Step Process:

- **Compilation Stage:**

- Each source file is compiled into an **object file (.OBJ)**.
- External library functions are not linked immediately.
- Instead, **placeholders or references** are added in the symbol table.

- **Loading Stage:**

- When the program is executed, the **dynamic linker/loader** checks for required shared libraries.
- The necessary **Dynamic Link Libraries (DLLs)** are loaded into memory (only once).

Working of Dynamic Linking

Step-by-Step Process:

- **Relocation at Runtime:**
 - The loader fixes the addresses of the referenced functions and variables.
 - These addresses point to the shared library in memory.
- **Execution Stage:**
 - The program executes using the library code that's already in memory.

Static vs Dynamic Linking

Feature	Static Linking	Dynamic Linking
When Linking Occurs	Before execution (compile/link time)	During execution (runtime)
Executable Size	Large	Small
Memory Usage	More (duplicate code in each process)	Less (shared among processes)
Update Libraries	Requires relinking	No relinking needed
Example	.EXE with built-in code	.EXE + .DLL files

Sequential and Direct Loading

Sequential Loading is a loading method where the loader reads and loads the program instructions and data **in the same order** as they appear in the executable file.

How its Working:

- The loader starts reading the program **from the beginning** and loads instructions one after another into consecutive memory locations.
- This is the simplest and oldest form of loading used in early operating systems (like MS-DOS, batch systems).

Characteristics:

- The program is stored **sequentially** in secondary memory (disk).
- The loader **reads linearly** from the start to the end of the file.
- The **execution starts** from the first instruction after loading.
- No need for complex address translation or relocation.

Advantages:

- Simple and easy to implement.
- Requires **less overhead** (no random access or relocation table).
- Suitable for **small programs** or **single-user systems**.

Direct Loading

- **Direct Loading** is a method where the loader can **access and load specific parts** of a program **directly** into memory, rather than loading sequentially.
- This allows random (non-sequential) access to program modules or data blocks.

Working Principle:

- The loader uses **address or location information** from the executable's header or relocation table.
- It **jumps directly** to specific addresses or file offsets and loads code/data segments as required.
- Modern executable formats (like .EXE or .DLL files) use **direct loading**.

Characteristics:

- The program is divided into **segments** (code, data, stack, etc.).
- The loader reads only **necessary segments** and places them in specified memory areas.
- The **header information** guides the loader where to load each segment.
- Allows **random access** within the executable file.

Advantages:

- **Fast loading** (skips unused parts).
- **Efficient memory use** (loads only needed modules).
- Enables **dynamic loading** and **demand paging**.
- Suitable for **large or modular programs**.

Comparison Table

Feature	Sequential Loading	Direct Loading
Access Method	Sequential (one after another)	Direct (random access)
Speed	Slower	Faster
Memory Use	Loads entire program	Loads required parts only
Complexity	Simple loader	Complex loader
Used In	Simple systems (e.g., MS-DOS .COM)	Modern OS (e.g., Windows .EXE, Linux ELF)
Relocation	Usually not required	Often required
Example	DOS .COM files	Windows .EXE, .DLL files

Absolute, Relocating, and Linking Loaders

- An **Absolute Loader** is the simplest type of loader that loads a program into a **fixed, predefined memory location** without any address modification.

Working:

- The assembler or compiler must know in advance where the program will be loaded.
- The object code produced already contains **absolute (real) addresses**.
- The loader simply reads the object code and places it into those exact memory addresses.

Functions:

- Reads the object program from secondary storage.
- Places the machine instructions into specified memory locations.
- Transfers control to the starting address for execution.

Advantages:

- Very simple and fast (no relocation or linking).
- Easy to implement.

Disadvantages:

- Program **must always load at the same address.**
- **Not suitable for multiprogramming** (memory conflict possible).
- Reassembling required if load address changes.

Example:

- In early MS-DOS .COM programs:
- The code is loaded starting at offset 0100h every time.
- No relocation or linking is performed.

Relocating Loader

- **A Relocating Loader** (or **Relative Loader**) is one that can load a program into **any part of memory** by **modifying address-dependent locations** in the program.

Working:

- The assembler generates **relocatable object code**, assuming the program starts at address 0.
- The loader determines the actual load address (e.g., 2000H).
- It adds a **relocation factor** to all address-sensitive instructions or data.

$$\text{Relocation Factor} = \text{Actual Load Address} - \text{Assumed Start Address}$$

Functions:

- Reads the object program and relocation information.
- Calculates the relocation factor.
- Modifies all address-sensitive instructions using the relocation factor.
- Loads the relocated code into memory and starts execution.

Advantages:

- Can load the same program at **different memory locations**.
- Supports **multiprogramming** and **dynamic memory allocation**.
- Efficient use of memory.

Linking Loader

- **A Linking Loader** performs both **linking and relocation** while loading the program into memory.
- It combines multiple **object modules** and **library routines**, resolves external references, relocates addresses, and then loads the final program for execution.

Working:

- **Reads** all object modules (e.g., MAIN.OBJ, UTIL.OBJ).
- **Performs linking** – resolves all external references.
- **Performs relocation** – adjusts all addresses based on load location.
- **Loads** the linked and relocated program into memory.
- **Starts** execution at the specified start address.

Functions of a Linking Loader:

- **Program Linking:**
Combines multiple object files into one loadable program.
- **Relocation:**
Adjusts addresses so that the program can be placed anywhere in memory.
- **Loading:**
Places the final machine code into memory.
- **Program Execution:**
Transfers control to the start address.

Advantages:

- Supports **modular programming** (separate compilation).
- Allows use of **libraries** and **shared routines**.
- Facilitates **relocation** and **reuse** of code.

Disadvantages:

- **Complex** implementation.
- Requires more **time and memory** during loading.

Comparison of Linkers and Loaders

Feature	Linker	Loader
Function	Combines object modules and resolves external references	Loads executable file into memory for execution
Input	Object files (.OBJ)	Executable file (.EXE)
Output	Executable file (.EXE)	Program loaded in memory
Time of Operation	Before program execution	At the time of execution
Performs Relocation	Adjusts addresses during linking	May adjust addresses again during loading
Performs Linking	Yes, static linking	Sometimes (in dynamic linking)
Speed	Performed once → slower initially	Fast, executed every run
Example Tool (MS-DOS)	LINK.EXE	Part of the DOS operating system
Main Responsibility	Prepare the program for loading	Prepare the program for execution
Output Location	Disk (creates .EXE file)	Main Memory
Dependency	Requires object code from compiler	Requires linked executable from linker

UNIT-5

Operating Systems & System Programming

(23ACA06)

Programming Language Grammars and Classification

- In **compiler design**, understanding programming language grammars is critical for **scanning (lexical analysis)** and **parsing (syntax analysis)**.
- **Scanning**: Converts raw source code into **tokens**.
- **Parsing**: Checks token sequences against the **grammar rules** to ensure syntactic correctness.
- **Grammars** define the **syntax rules** of a language and help in constructing **lexical analyzers** and **parsers**.

Lexical Analysis (Scanning):

- Uses **regular grammar** to identify **tokens** like identifiers, keywords, numbers, operators.
- **Token examples:** id, number, if, while, +, -
- **Finite automata** are used to recognize regular patterns.

Syntax Analysis (Parsing):

- Uses **context-free grammar (CFG)** to check **valid arrangement of tokens**.
- Builds **parse trees** to represent program structure.
- Detects **syntax errors** and ensures compliance with language rules.

Components of Programming Language Grammar

A formal grammar G is defined as:

$$G = (N, T, P, S)$$

Component	Description	Example
N	Non-terminal symbols	<expression>, <statement>
T	Terminal symbols	id, +, *, if, ;
P	Production rules	<expression> $\rightarrow$ <expression> + <term>
S	Start symbol	<program>

Classification of Grammars in Scanning and Parsing

Type	Grammar	Used In
Type 3 (Regular Grammar)	Can be represented by regular expressions	Scanning / Lexical Analysis
Type 2 (Context-Free Grammar - CFG)	Non-terminals → terminals & non-terminals	Parsing / Syntax Analysis
Type 1 (Context-Sensitive Grammar)	Non-terminals → strings depending on context	Rarely used in practical compilers
Type 0 (Unrestricted Grammar)	No restrictions	Theoretical computation models

Scanning (Lexical Analysis)

Objective: Convert source code into tokens.

Input: Source code string

Process: Matches sequences of characters with **regular expressions** (patterns for identifiers, literals, operators)

Output: Stream of tokens for the parser

Example:

Source code: `x = a + 5;`

Lexeme	Token
x	id
=	assignment
a	id
+	plus
5	number
;	semicolon

Parsing (Syntax Analysis)

- **Objective:** Ensure token sequences conform to grammar rules (CFG).
- **Input:** Token stream from scanner
- **Process:** Builds **parse tree** using **parsing techniques**
 - **Top-Down Parsing:** Starts from start symbol and predicts tokens
 - **Bottom-Up Parsing:** Builds tree from tokens towards start symbol
- **Output:** Parse tree or syntax error messages

Connection Between Grammar Type and Compiler Phases

Phase	Grammar Type	Purpose
Lexical Analysis	Regular Grammar	Tokenization using regex & DFA
Syntax Analysis	Context-Free Grammar (CFG)	Build parse trees, check structure
Semantic Analysis	Context-Sensitive Rules	Type checking, scope resolution

Ambiguity in Grammatical Specification

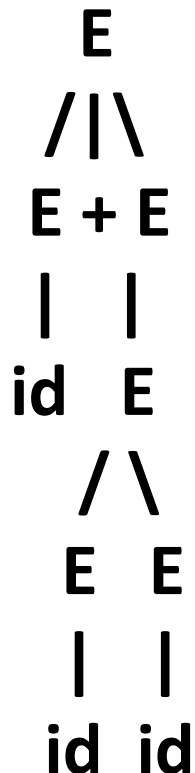
- A grammar G is **ambiguous** if there exists **at least one string 'w'** in the language $L(G)$ that has **two or more distinct parse trees** (or derivations).

Formally:

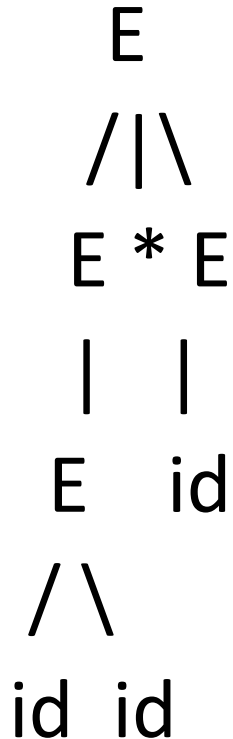
- $\exists w \in L(G): w \text{ has } \geq 2 \text{ parse trees according to } G$

Example of Ambiguous Grammar

- Consider the arithmetic expression grammar: $E \rightarrow E + E \mid E * E \mid (E) \mid \text{id}$
- String:** $\text{id} + \text{id} * \text{id}$
- Parse Tree 1:** $+$ evaluated before $*$



- **Parse Tree 2:** * evaluated before +



Problem: Same expression can be interpreted differently → ambiguity.

Causes of Ambiguity

- **Operator Precedence Not Defined**
 - Example: + and * in arithmetic expressions.
- **Multiple Parsing Rules**
 - More than one production can generate the same string.
- **Optional Symbols / Recursion**
 - Left recursion and optional rules may lead to multiple derivations.

Problems Caused by Ambiguity

- **Parser Confusion:** The parser may not know which tree to build.
- **Multiple Interpretations:** Programs may have **different meanings**, causing errors in execution.
- **Complicates Compiler Design:** Ambiguity needs resolution to generate consistent code.

Detecting Ambiguity

- Construct **parse trees** for the same string.
- If more than one valid parse tree exists → grammar is ambiguous.
- Some grammars are **intrinsically ambiguous** (no equivalent unambiguous grammar exists).

Resolving Ambiguity

1. Redefining Grammar

Modify production rules to **remove multiple derivations**.

Example: Resolve arithmetic expression ambiguity:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid \text{id}$$

* has higher precedence than +

This grammar is **unambiguous** for expressions like $\text{id} + \text{id} * \text{id}$.

Resolving Ambiguity

2. Use Operator Precedence and Associativity

Specify:

Precedence: Which operator is applied first

Associativity: Left-to-right or right-to-left evaluation

3. Parser Directives

Some parser generators (e.g., Yacc/Bison) allow **disambiguation rules:**

%left, %right, %nonassoc for operators

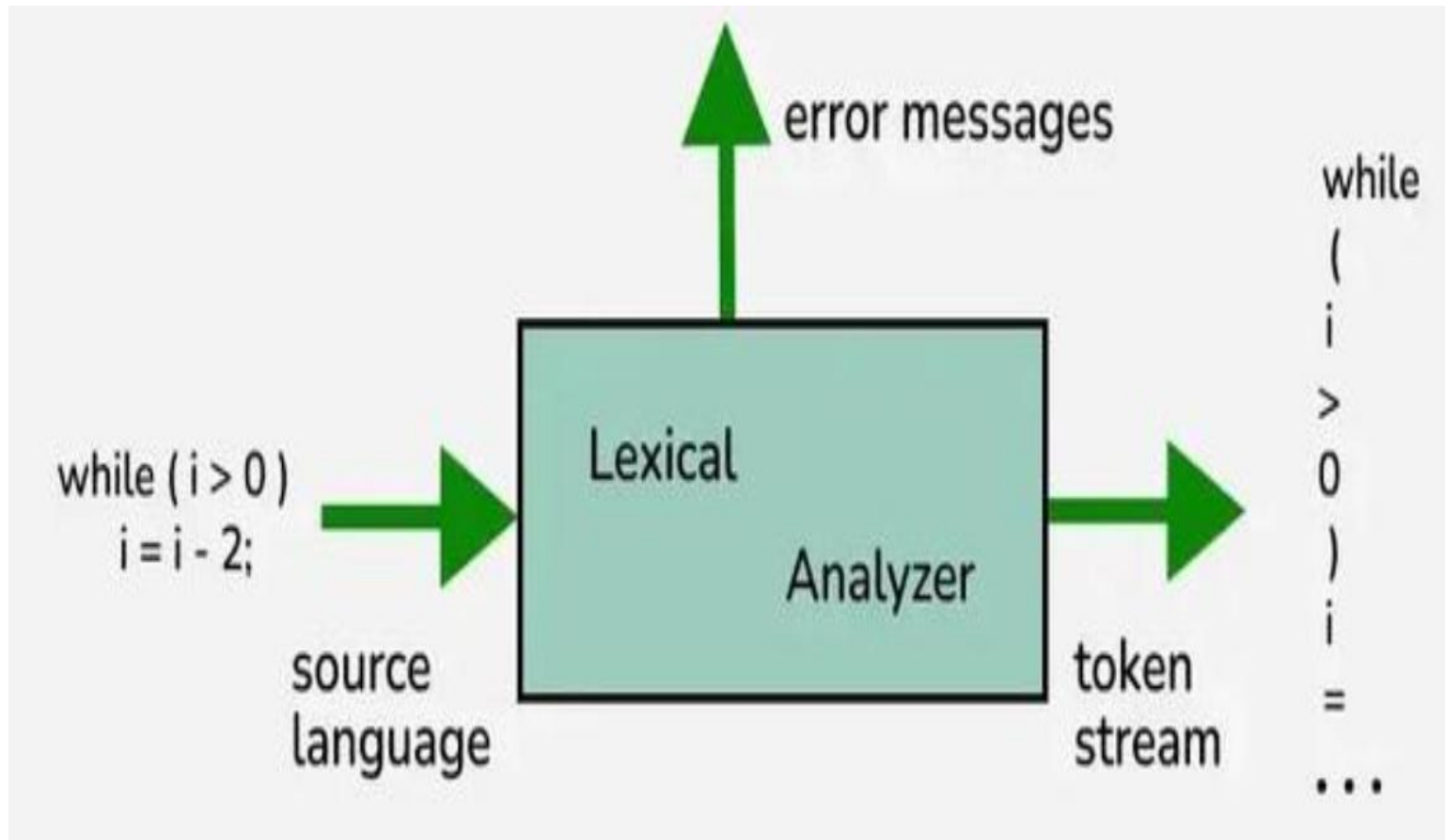
Summary Table

Aspect	Description
Ambiguity	A grammar is ambiguous if a string has multiple parse trees
Cause	Undefined operator precedence, multiple production rules
Problem	Parser confusion, multiple interpretations, compiler errors
Solution	Redefine grammar, specify precedence/associativity, use parser directives

Scanning

- **Scanning(Lexical Analysis.)** is the **first phase** of compilation in **system programming**.
- **Purpose:** scanning is to read the **source program** (sequence of characters) and convert into sequence of **tokens**.
- **Scanning (Lexical Analysis)** is the process of **reading** the source code **character by character** and **grouping** them into meaningful sequences called **lexemes**, which are then converted into **tokens**.

Scanning Diagram



Role of the Scanner

- Acts as a **bridge** between the **source code** and the **parser**.
- Simplifies the parser's job by identifying **valid words or symbols (tokens)** from **raw** input text.
- Example: `int a = b + 10;`

Important Terms

Term	Description	Example
Lexeme	Actual character sequence from source code	int, a, 10
Token	Symbolic name for a lexeme	KEYWORD, IDENTIFIER, NUMBER
Pattern	Rule that defines how to form lexemes	e.g., Identifier → [A-Za-z][A-Za-z0-9]*

Functions of the Scanner

- **Read input characters**
 - Reads the program one character at a time.
- **Remove whitespace and comments**
 - Skips irrelevant data to make parsing easier.
- **Recognize tokens**
 - Groups characters into keywords, identifiers, numbers, symbols, etc.
- **Report lexical errors**
 - Detects invalid characters or illegal identifiers.
- **Communicate with the symbol table**
 - Stores identifiers and related information for later stages.

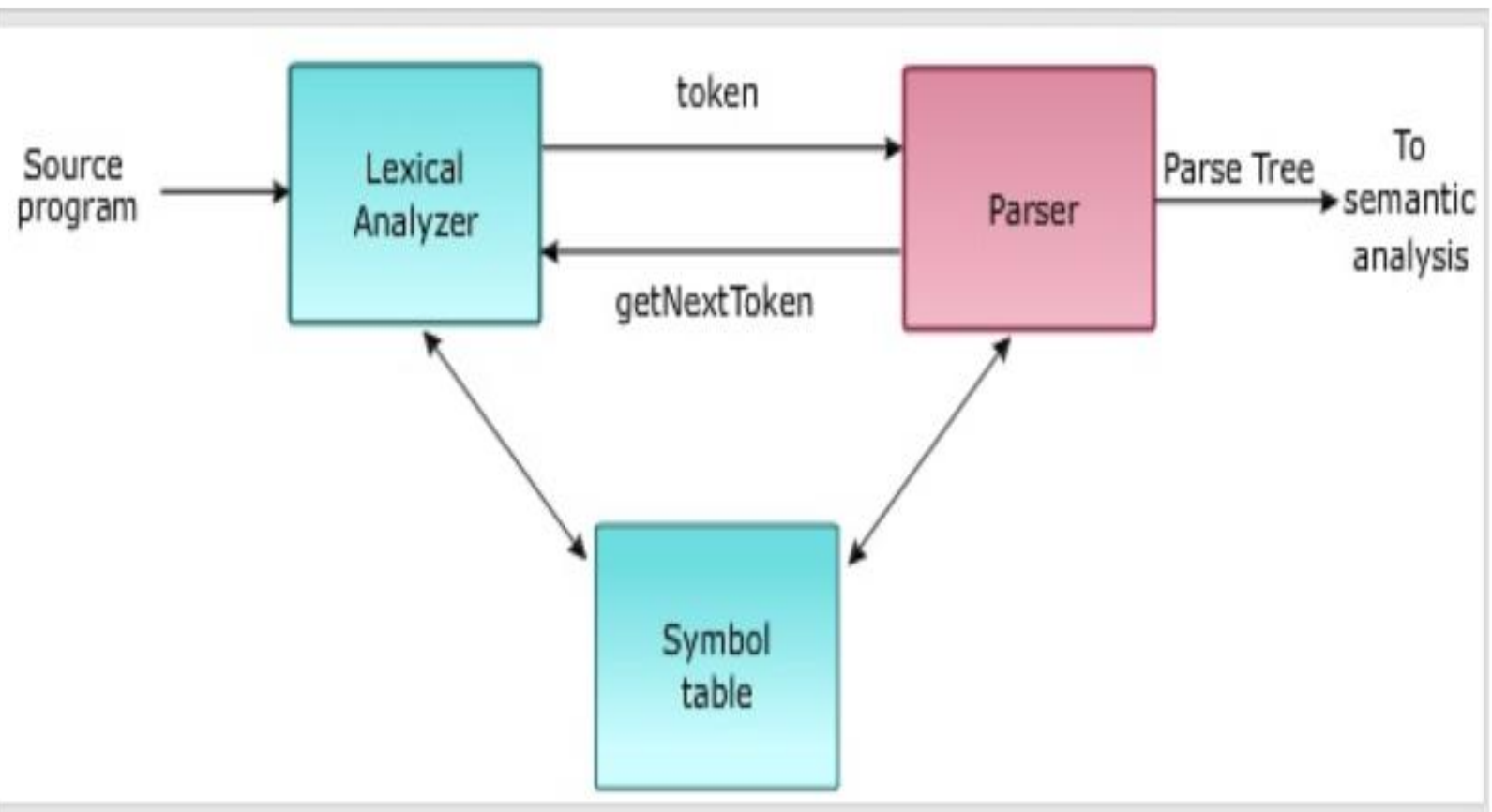
Token Types

Token Type	Example
Keywords	if, else, int, while
Identifiers	sum, count, total
Constants	10, 3.14, 'A'
Operators	+, -, *, /, =
Punctuators / Delimiters	;, ,, (,)

Parsing

- **Parsing** is the second phase of compilation in System Programming.
- The **parser** receives a stream of **tokens** from the **scanner** and checks whether the **sequence of tokens follows the syntax rules** of the programming language.
- **Parsing (Syntax Analysis)** is the process of analyzing a sequence of tokens to determine **its grammatical structure** according to the rules of a **context-free grammar (CFG)**.

Parsing Diagram



Role of Parser

- Takes tokens from the **scanner**.
- Applies **grammar rules** to construct a **parse tree**.
- Detects **syntax errors** in the **source program** and **Provides structural information** to the next compiler phase (**semantic analysis**).

Example source code:

```
a = b + c * d;
```

Parse Tree

- A **Parse Tree** is a **hierarchical tree structure** that shows how a string (or program) is derived from the **start symbol** of a grammar using the **production rules** of that grammar.
- A **Parse Tree** is a graphical representation of how a parser analyzes and derives a source code's structure according to grammar rules.

Key Points of Parse Tree

- Each **internal node** in the tree represents a **non-terminal symbol**.
- Each **leaf node** represents a **terminal symbol** (tokens such as identifiers, operators, or keywords).
- The **root node** represents the **start symbol** of the grammar.
- The tree shows the **derivation steps** of the input string.

Grammar in Parsing

A **grammar** defines the syntactic structure of a programming language using production rules.

$$G = (V, T, P, S)$$

Where:

- $V \rightarrow$ set of non-terminal symbols
- $T \rightarrow$ set of terminal symbols (tokens)
- $P \rightarrow$ set of production rules
- $S \rightarrow$ start symbol

Types of Parsing

Type	Description	Example
Top-Down Parsing	Begins from the start symbol and tries to derive the input string	Recursive Descent, LL(1)
Bottom-Up Parsing	Begins with input and tries to reduce it to the start symbol	Shift-Reduce, LR(0), SLR(1)

1. Top-Down Parsing

- Tries to **predict** which production rule to use.
- Works **from root to leaves** of the parse tree.
- Example methods:
 - **Recursive Descent Parser**
 - **Predictive (LL(1)) Parser**
- **Example:**

For grammar $E \rightarrow E + T \mid T$,
parser starts with E and tries to derive the
input id + id.

Bottom-Up Parsing

- Works **from leaves (input tokens) to root** (start symbol).
- Repeatedly **reduces** input string using grammar rules.

Example methods:

- **Shift-Reduce Parser**
- **LR Parser** (LR(0), SLR(1), LALR(1))

Difference Between Scanning and Parsing

Aspect	Scanning	Parsing
Phase	First phase	Second phase
Input	Source program (characters)	Tokens
Output	Tokens	Parse tree
Checks	Lexical structure	Syntax structure
Error Type	Lexical errors	Syntax errors

Compilers and Interpreters

- A **Compiler** is a system software that **translates a high-level source program** (like C, Java) into **machine code (object code)** that the computer can execute directly.
- The entire program is translated **before execution**.
- Errors are reported **after the compilation** process is complete.

Example:

C compiler, C++ compiler, Java compiler (javac)

Interpreters

- An **Interpreter** is a system program that **translates and executes** a source program **line-by-line or statement-by-statement**.
- Each instruction is **translated and executed immediately**.
- Errors are reported **one at a time**, stopping execution when found.

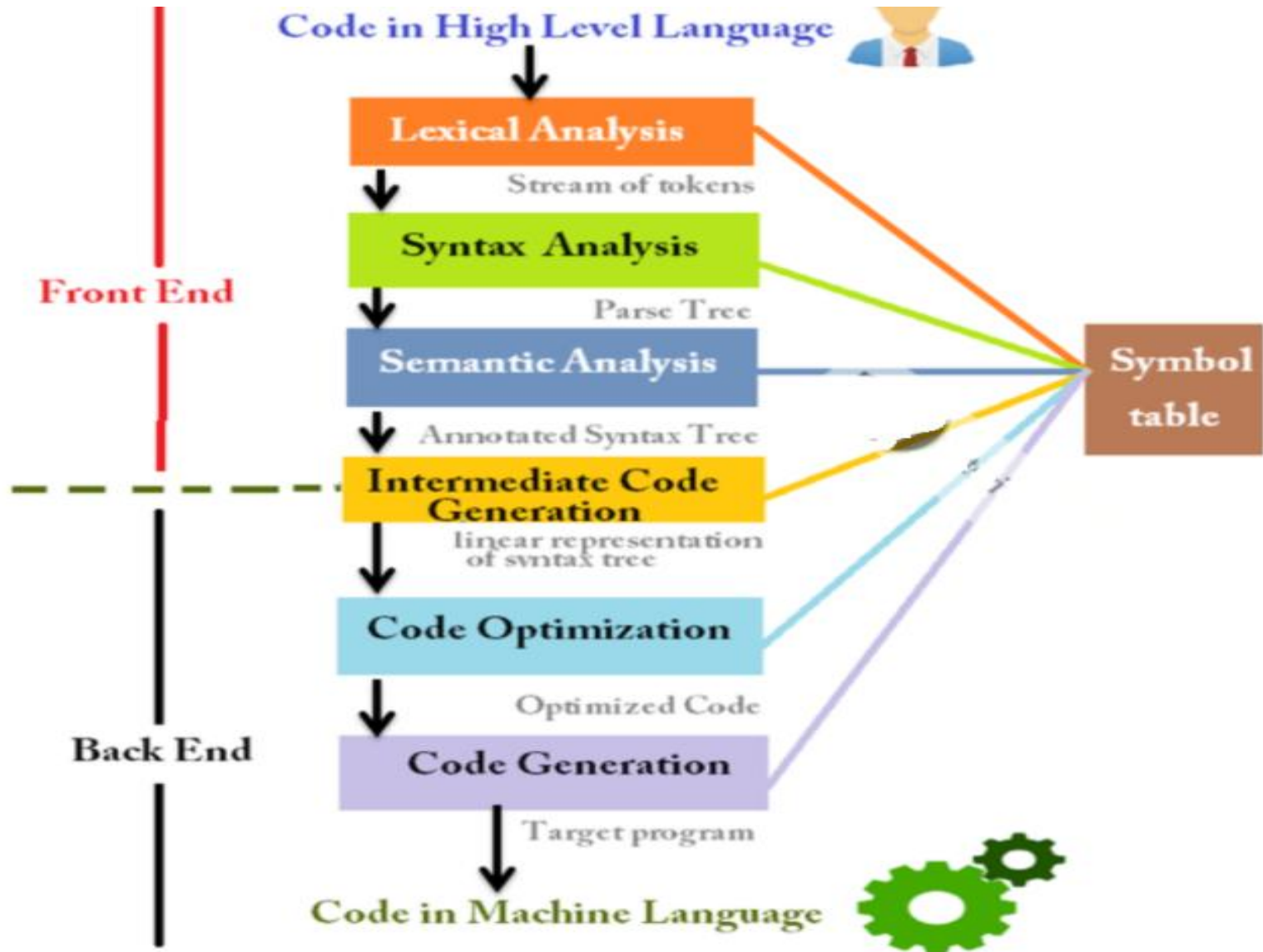
Example:

Python interpreter, JavaScript engine, PHP interpreter

Compilation Process

- The compilation process converts a high-level language program into machine-level code through several **phases**. Each phase performs a specific task in translating and optimizing the program.

Phases of Compiler



Phases of Compiler

1. Lexical Analysis (Scanning)

Breaks the source code into **tokens** (keywords, identifiers, literals, operators).

Tool used: **Lexical Analyzer (Lexer)**.

Output: **Token stream**.

2. Syntax Analysis (Parsing)

Checks if the sequence of tokens follows the grammar rules of the language.

Builds a **Parse Tree** or **Syntax Tree**.

Tool used: **Parser**.

Phases of Compilation:

3. Semantic Analysis

Ensures the **meaning** of the program is correct.

Checks **type compatibility**, **variable declarations**, and **scope rules**.

Output: **Annotated Syntax Tree**.

4. Intermediate Code Generation

Produces a **machine-independent intermediate code**.

Easier for optimization and translation.

Example format: Three-address code.

Phases of Compilation:

5. Code Optimization

Improves efficiency of intermediate code without changing its meaning.

Reduces **execution time** and **memory usage**.

6. Code Generation

Converts optimized intermediate code into **machine code** (target code).

Output: **Object code**.

7. Code Linking and Loading

Links different object files and **loads** the final executable into memory.

Semantic Gap

- The **Semantic Gap** refers to the **difference between the high-level human understanding of a problem** and the **low-level operations** that the computer hardware performs.
- It measures how far apart the **source language (user-oriented)** and **machine language (hardware-oriented)** are in **abstraction level**.

Example:

Writing **sum = a + b** in **C** is **simple** for a human.

The computer must perform:

- LOAD a → Register1
 - LOAD b → Register2
 - ADD Register1, Register2 → Register3
 - STORE Register3 → sum(C)
- This gap in understanding and execution is the **semantic gap**.

Binding and Scope Rules

- **Binding** is the process of **associating** an attribute with an entity such as a variable, data type, or function in a program.
- Binding of a variable to its **data type**:
 int x;
 → binds variable **x** to type **int**
- Binding of a variable to its **memory location** (during execution)
- Binding of a function name to its **definition**

Types of Binding

Type	Description	Example
Static Binding (Early Binding)	Binding that occurs before program execution, usually during compilation	Data type of a variable, function overloading resolution
Dynamic Binding (Late Binding)	Binding that occurs during execution (runtime)	Function overriding in object-oriented languages like Java or Python

Phases of Binding in Compilation

Binding Type	Time of Occurrence
Language Definition Time	Keywords, syntax, semantics defined
Compile Time	Variable types, procedure calls
Link Time	External references resolved
Load Time	Addresses assigned
Run Time	Dynamic bindings (values, objects) assigned

Scope Rules

- **Scope** of a variable is the **region of a program** where that variable is **visible and can be used**.
- **Scope** determines **which variable name refers to which binding** in nested or overlapping context

Types of Scope

- 1. Static Scope (Lexical Scope)
- 2. Dynamic Scope

1. Static Scope (Lexical Scope)

- Determined **at compile time** based on **program structure**.
- The compiler uses **textual (lexical) structure** of the code to resolve variable references.
- Most modern languages (C, Java, Python) use **static scope**.

Example:

```
x = 10;
def func():
    x = 20;
    print(x)
func()    # Output: 20
print(x)  # Output: 20
```

2. Dynamic Scope

- Determined **at runtime**, based on the **calling sequence** of functions.
- A variable is searched in the **current function** and then **in the calling functions**.
- Used in older languages (like early Lisp versions).

Example:

- Function A defines $x = 5$
- Function B prints x
- If **B** is called by **A** $\rightarrow$ **B** uses **A's x** (dynamic lookup)

Memory Allocation

Memory allocation is the process of **reserving portions of computer memory** for storing **program instructions and data** during program execution.

- In simple terms, when a program runs, it needs memory for:
 - Program **code (instructions)**
 - **Variables** and **data structures**
 - **Function calls** (stack)
 - **Dynamic data** (heap)
- The **compiler, linker, loader, and operating system** work together to manage how memory is allocated.

Types of Memory Allocation

1. Static Memory Allocation:

- In **static allocation**, memory for all variables is **allocated during compile time**.
- The size and location of each variable are fixed before program execution begins.

Example:

```
int a = 10;  
float b = 5.5;  
char name[20];
```

All variables above are assigned fixed memory locations at compile time.

Characteristics:

- Memory is allocated **once** and **cannot change** during execution.
- Allocation is handled by the **compiler**.
- Faster execution because addresses are known beforehand.
- Less flexible.

Advantages:

- Simple to implement.
- No runtime overhead.
- Faster access.

Disadvantages:

- Memory is **wasted** if not fully used.
- The program cannot handle **variable-size data** (e.g., dynamic arrays).

2. Dynamic Memory Allocation

- In **dynamic allocation**, memory is allocated **at runtime** based on program needs.
- This allows programs to request memory **while executing**.

Characteristics:

- Done during **program execution**.
- Managed using **heap memory**.
- Requires **pointers** to access dynamically allocated memory.
- Programmer must **manually release** memory when it's no longer needed.

Example:

- `int *ptr;`
- `ptr = (int *) malloc(sizeof(int) * 5); // Allocates memory for 5 integers.`

Dynamic Allocation Functions in C:

Function	Purpose
malloc()	Allocates memory block
calloc()	Allocates and initializes to zero
realloc()	Reallocates or resizes memory
free()	Frees allocated memory

Advantages:

- Efficient use of memory.
- Flexible — memory can grow or shrink as needed.
- Suitable for variable-size data structures (like linked lists, trees).

Disadvantages:

- Slightly **slower** due to runtime management.
- May cause **memory leaks** if not properly freed.
- **Fragmentation** may occur in the heap.

Memory Allocation Areas in a Program

Segment	Purpose
Code Segment (Text Segment)	Stores compiled instructions of the program
Data Segment	Stores global and static variables
Heap	Used for dynamic memory allocation
Stack	Used for function calls and local variables
Registers	Store temporary data and CPU-level operations

Storage Classes and Memory Allocation (in C)

Storage Class	Scope	Lifetime	Memory Area	Example
AUTO	Local	Function call	Stack	auto int x;
REGISTER	Local	Function call	CPU Register	register int x;
STATIC	Local/Global	Entire program	Data Segment	static int x;
EXTERN	Global	Entire program	Data Segment	extern int x;

Code Optimization

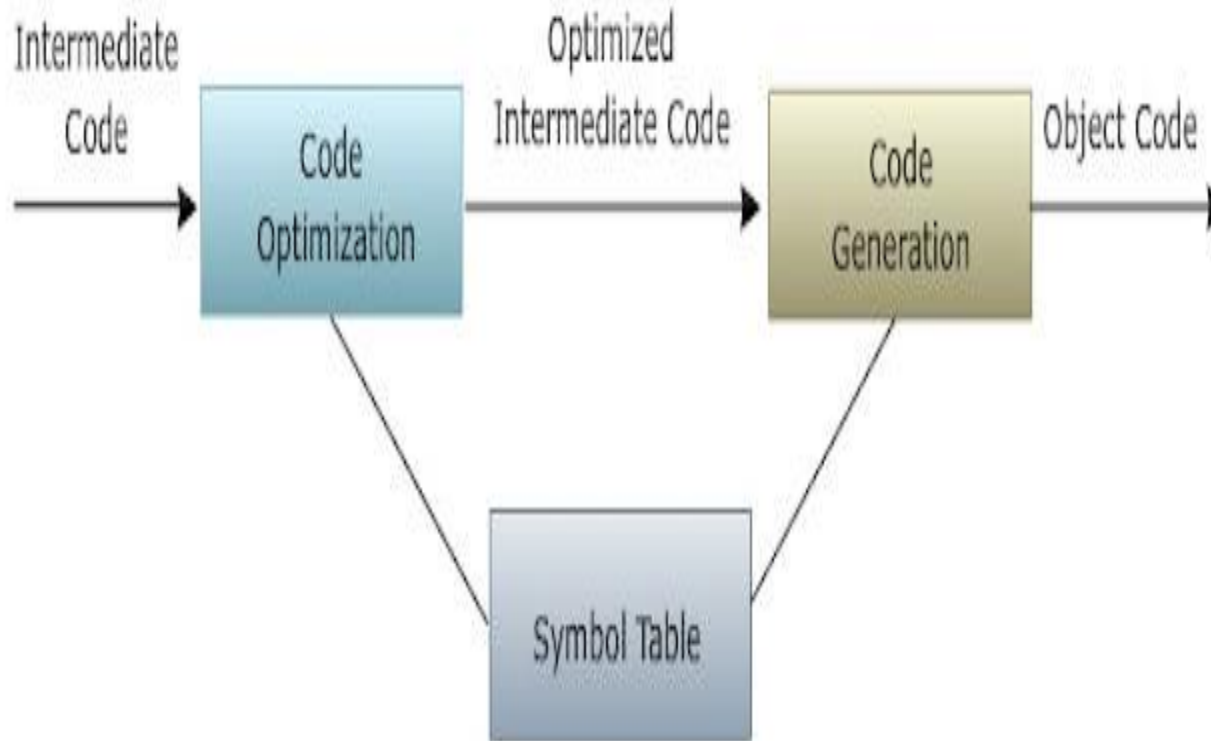
- **Code Optimization** is the process of **improving the intermediate or final code** of a program so that it **runs faster, uses less memory, or performs more efficiently** — **without changing its output or meaning.**
- In **phases of the compiler**, typically performed **after intermediate code generation and before final code generation.**

Objective of Code Optimization

The main goals of code optimization are:

- **Improve Execution Speed** – Make the program run faster.
- **Reduce Memory Usage** – Use fewer variables and storage locations.
- **Reduce Power Consumption** – Important for embedded systems and mobile devices.
- **Improve Resource Utilization** – Optimize CPU and cache performance.
- **Preserve Correctness** – Ensure optimized code gives the **same output** as the original code.

Work of Code Optimization



Types of Code Optimization

1. Machine-Independent Optimization
2. Machine-Dependent Optimization

1. Machine-Independent Optimization

- Performed on **intermediate code**.
- Independent of target machine architecture.
- Focuses on reducing **redundancy** and **improving logic**.

Common Techniques:

Technique	Description	Example
Constant Folding	Evaluate constant expressions at compile time.	$x = 4 * 2 \rightarrow x = 8$
Constant Propagation	Replace known constant variables with their values.	$a = 5; b = a + 3; \rightarrow b = 8;$
Dead Code Elimination	Remove code that never affects program output.	<code>if (false) { ... } removed</code>
Common Subexpression Elimination	Avoid recomputing the same expression multiple times.	$x = a + b; y = a + b + c; \rightarrow$ reuse $a + b$
Loop Invariant Code Motion	Move calculations outside loops if they don't change within the loop.	<code>for(i=0;i<n;i++) y=x*5; →</code> move $x*5$ outside
Strength Reduction	Replace expensive operations with cheaper ones.	$x = i * 2; \rightarrow x = i + i;$
Copy Propagation	Replace variables that only copy values from others.	$a = b; c = a + 1; \rightarrow c = b + 1;$
Algebraic Simplification	Simplify expressions using algebraic rules.	$x = x + 0 \rightarrow x$

2. Machine-Dependent Optimization

- Performed during **target code generation**.
- Depends on the **architecture** of the machine (e.g., register usage, instruction set).
- Improves execution speed by efficiently using **CPU registers** and **machine instructions**

Common Techniques:

Technique	Description
Register Allocation	Assign frequently used variables to CPU registers instead of memory.
Instruction Scheduling	Arrange instructions to minimize CPU idle time or pipeline stalls.
Loop Optimization	Unroll or optimize loops for speed.
Inline Expansion	Replace small function calls with the function body itself.
Peephole Optimization	Examine small sets of instructions (“peepholes”) and replace inefficient sequences with better ones.

Example:

Before Optimization:

MOV R1, A

MOV R2, A

After Optimization:

MOV R2, R1

Advantages of Code Optimization

Advantage	Description
Faster Execution	Eliminates redundant instructions.
Reduced Memory Use	Removes unused variables and code.
Smaller Code Size	Compact and efficient code.
Improved Performance	Better use of CPU and cache.
Energy Efficiency	Important for mobile and embedded systems.

Compilation of Expressions and Control Structures

- During the **compilation process**, one of the main tasks of the **code generation phase** is to **translate expressions and control structures** (like loops and conditionals) into **intermediate code** or **machine code** that the computer can execute.
- This process ensures that high-level language constructs are converted into **efficient low-level instructions**.

1. Compilation of Expressions

- Expressions are the **basic building blocks** of any program.
They include **arithmetic**, **relational**, and **logical** operations.

Expression Types

- **Arithmetic Expressions** –
involve operators like +, -, *, /
- **Relational Expressions** –
involve operators like <, >, <=, ==
- **Logical Expressions** –
involve operators like &&, ||, !

Steps in Compilation:

- **Parsing Phase:**
 - Converts expression into a **parse tree**:
- **Intermediate Code Generation:**
 - The compiler converts the parse tree into **three-address code (TAC)**.
- **Optimization (optional):**
 - Removes redundant computations or simplifies code.
- **Code Generation:**
 - Translates intermediate code into **assembly or machine code**

2. Compilation of Control Structures

- Control structures define **the order in which statements are executed**.
The compiler must translate them into **branching and looping instructions**.

If–Else Statement

- High-level code:**

```
if (a < b)
    x = 1;
else
    x = 2;
```

Intermediate Code:

```
if a < b goto L1
goto L2
L1: x = 1
goto L3
L2: x = 2
L3:
```

Flow of Control

Structure	Control Flow Type	Mechanism Used
Sequential	Straight-line execution	None
Selection	Conditional branching	if, else, switch
Iteration	Repeated execution	for, while, do-while
Jump	Unconditional	goto, break, continue

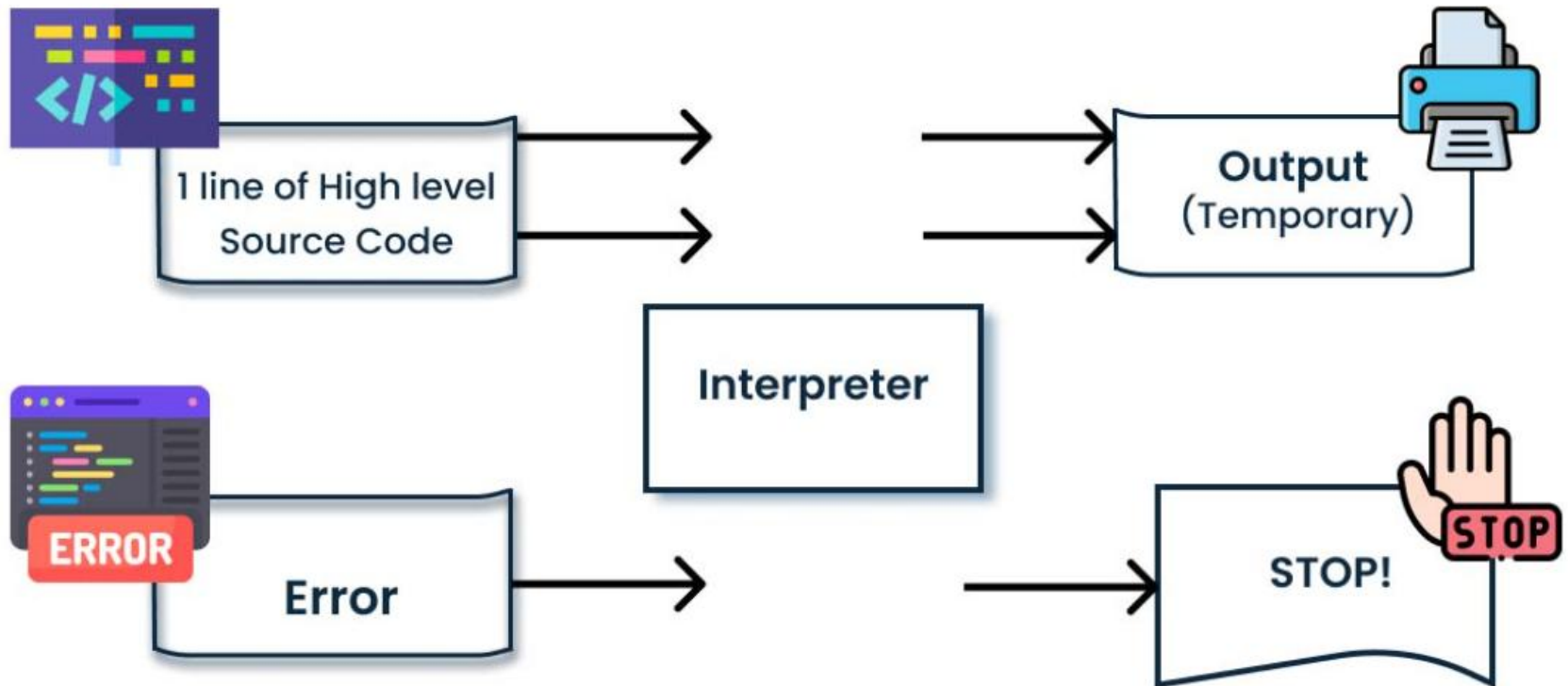
Overview of Interpreters and Debuggers

- **Interpreter** is a system program that executes the source program **line by line** translating **HLL** instructions into machine code **at runtime**.

Function:

- Reads a high-level language statement.
- Converts it to intermediate or machine code.
- Executes it immediately before moving to the next line.

Process of Interpreter



Key Features

- No separate object code is generated.
- Execution is slower than compiled programs.
- Useful for testing and debugging small programs.

Examples

- **Python Interpreter (CPython)**
- **JavaScript Engine (V8)**
- **Ruby Interpreter**
- **MATLAB Interpreter**

Advantages

- Easier debugging (error shown immediately).
- Ideal for scripting and interactive environments.
- Platform-independent execution.

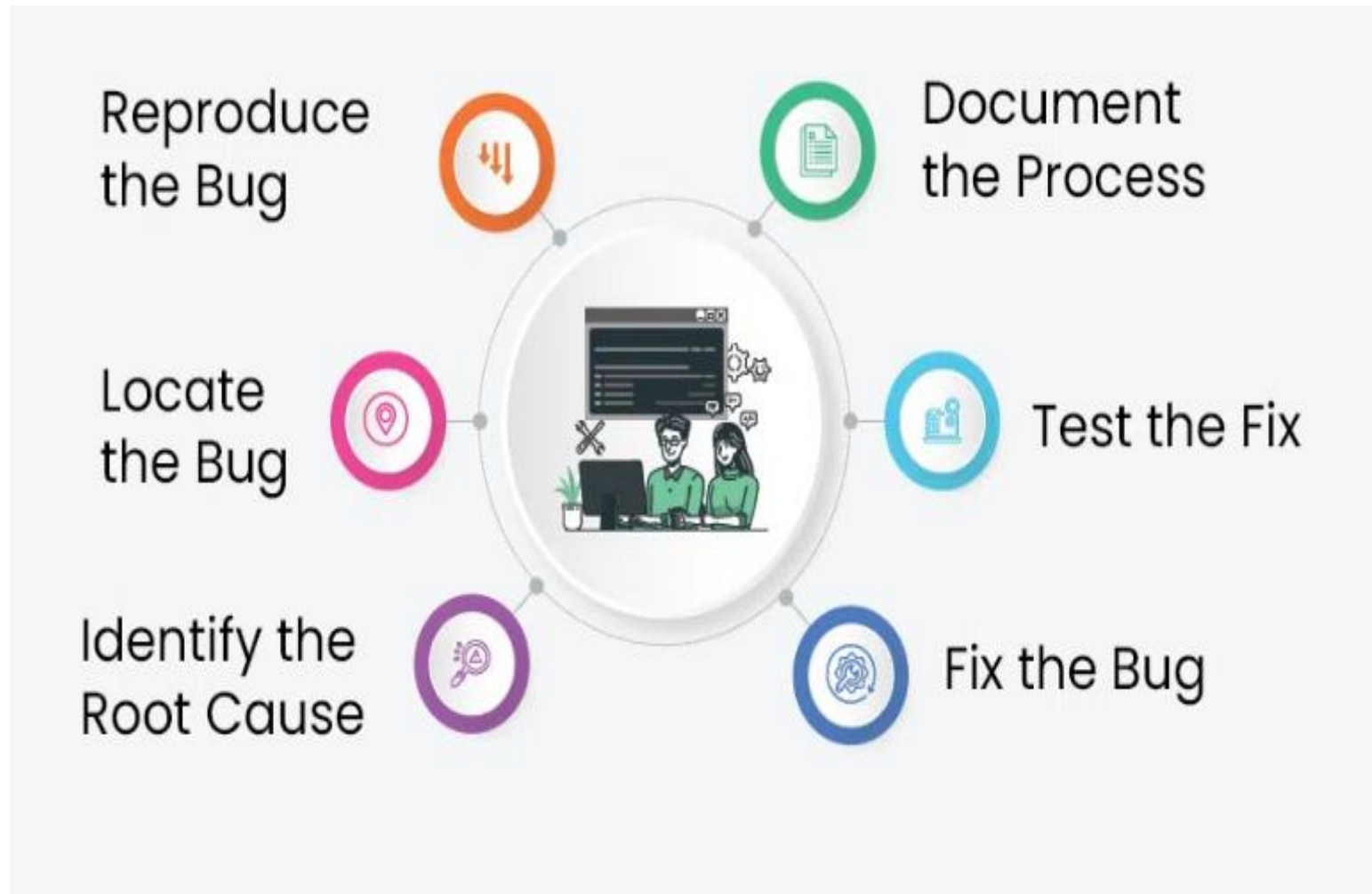
Debugger

- A **Debugger** is a system program used to **test and debug** other programs by allowing the programmer to **control execution** and **examine program state**.

Function

- Helps identify **logical, runtime, or syntax errors**.
- Provides tools to **pause, inspect, modify, and resume** execution.

Process of Debugger



Key Features

- **Breakpoints:** Stop execution at specific lines.
- **Step Execution:** Execute code line by line.
- **Watch Variables:** Monitor variable values.
- **Memory Dump:** View memory and register contents.
- **Call Stack Inspection:** Trace function calls.

Examples

- **GDB (GNU Debugger)** – for C/C++
- **PDB (Python Debugger)**
- **Visual Studio Debugger**
- **LLDB (LLVM Debugger)**

Advantages

- Detects runtime and logic errors effectively.
- Allows tracing of program flow.
- Helps in optimizing code and performance.

C Development Tools

- C development on an Operating System (like Linux, macOS, or Windows) involves using **command-line tools and shell utilities** for **editing, compiling, linking, executing, and debugging** C programs.
- These tools form the **software development environment** that supports program creation and execution.

Essential C Development Tools

Tool Category	Purpose	Common Commands / Tools
Text Editor	Writing source code	vi, vim, nano, gedit, notepad
Compiler	Converts C source code to object code	gcc, clang, cc
Assembler	Converts assembly code to machine code	as
Linker	Combines object files and libraries into executable	ld
Loader	Loads executable into memory for execution	./a.out, ./program
Debugger	Helps to test and debug programs	gdb, ddd, valgrind
Make Utility	Automates compilation using Makefiles	make
Shell	Provides command interface for all tools	bash, sh, zsh, csh

C Development Workflow Using Shell Commands

Step 1: Write the Source Code

Use a text editor to create a C source file.

\$ vi hello.c

```
#include <stdio.h>
int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Step 2: Compile the Source Code

- Use a compiler (like GCC) to compile and generate an executable file.
- `$ gcc hello.c -o hello`
- `hello.c` → Source file
- `-o hello` → Output executable name

Step 3: Execute the Program

- `$ ./hello`

Output:

- Hello, World!

Important Shell Commands for C Development

Command	Purpose
ls	List files in directory
cat, more, less	View source code contents
cp, mv, rm	Copy, move, or remove files
chmod	Change file permissions (e.g., to make executable)
echo	Display messages during scripting
man gcc, man gdb	View manual pages for tools
history	View previously executed commands

Machine-Level Representation of Data and Programs

- at the **machine level**,
all data and programs are represented in **binary form (0s and 1s)**.
- Computers can only understand and execute instructions expressed in **machine language**, which corresponds directly to the **hardware architecture** (processor design, instruction set, and memory organization).

Representation of Data

a) Binary Representation

The **binary system** uses two symbols: **0** and **1**.

Each binary digit is called a **bit**.

A group of 8 bits forms a **byte**.

Example:

A → ASCII code → 01000001

b) Numeric Data Representation

Type	Description	Example
Integer	Represented using binary numbers (fixed number of bits).	00001010 = 10
Signed Integer	Uses Two's Complement for negative numbers.	11110110 = -10
Floating Point	Represented using IEEE 754 format (sign, exponent, mantissa).	010000010010000000 0000000000000000 = 10.0
BCD (Binary Coded Decimal)	Each decimal digit is represented by 4 bits.	59 → 0101 1001

c) Character Data Representation

Encoding Scheme	Description	Example
ASCII	7-bit code for English letters and symbols.	A → 65
EBCDIC	8-bit code used in IBM mainframes.	A → 193
Unicode	Supports global characters (16-bit or 32-bit).	A → U+0041

d) Boolean Data

- Represented by single bits:
 - ❑ 0 → False
 - ❑ 1 → True
- Used in logical operations and control structures.

e) Pointer and Address Representation

- A pointer stores the memory address of a variable or instruction.

Example:

- Variable x stored at address 0x2000
- Pointer p = &x → value of p is 0x2000

3. Representation of Programs

a) Source Program

Written in a high-level language (like C, C++, Python).

Human-readable instructions.

Example:

```
int sum = a + b;
```

b) Assembly Program

Translated from source code into assembly language (mnemonics).

Example:

```
MOV R1, a
```

```
ADD R1, b
```

```
MOV sum, R1
```

c) Machine Code

Binary form of the program executed by the CPU.

Example (for a simple ADD operation):

10001011 00000001

Each instruction corresponds to an opcode
(operation code) and operands.

d) Memory Representation of Programs

- When a program runs, it is divided into several segments:

Segment	Purpose
Code Segment (Text Segment)	Stores machine instructions
Data Segment	Stores global and static variables
Stack Segment	Used for function calls and local variables
Heap Segment	Used for dynamic memory allocation

e) Instruction Format

- A **machine instruction** is represented in binary and consists of:

Field	Description
Opcode	Operation code (what to do)
Operands	Data or memory addresses
Mode bits	Specify addressing mode

File Input/Output

File I/O (Input/Output) refers to the process of:

Input → Reading data from files

Output → Writing data to files

In **system-level programming**, file I/O is managed through **kernel-level system calls**, such as:

open(), read(), write(), close(), lseek()

These calls interact directly with the **file descriptors** maintained by the operating system.

System-Level File I/O System Calls

System Call	Purpose	Example
open()	Opens or creates a file	<code>int fd = open("data.txt", O_RDONLY);</code>
read()	Reads data from an open file	<code>read(fd, buffer, size);</code>
write()	Writes data to an open file	<code>write(fd, buffer, size);</code>
lseek()	Moves file pointer (for random access)	<code>lseek(fd, 0, SEEK_SET);</code>
close()	Closes the file	<code>close(fd);</code>

File Descriptors

- When a file is opened, the OS assigns a **file descriptor (fd)** — a small integer that represents an open file.

Descriptor	Name	Description
0	stdin	Standard Input
1	stdout	Standard Output
2	stderr	Standard Error

Concurrency in File I/O

- **Concurrency** means multiple processes or threads executing simultaneously and possibly accessing the same file or resource.
- If not handled properly, concurrent access can lead to:
- **Race conditions**
- **Data corruption**
- **Inconsistent output**

Synchronization Techniques

(a) File Locking

- Locks prevent multiple processes from modifying the same file region at once.

(b) Thread Synchronization

- In multi-threaded programs, **mutexes** are used to ensure that only one thread performs file I/O at a time.

Process Creation and Control (fork, exec)

In **System-Level Programming**, a **process** is a program in execution — it includes the program code, data, registers, and system resources.

- The **Operating System (OS)** provides system calls to:
 - **Create** new processes
 - **Control** process execution
 - **Replace** process images

Two important system calls in UNIX/Linux are:

- **fork()** → creates a new process
- **exec()** → replaces the process image with a new program
- Together, they form the **foundation of process management** in UNIX-like operating systems.

What is a Process?

A **process** consists of:

- **Program Code (Text Segment),**
- **Data Segment**
- **Stack**
- **Heap**
- **Program Counter (PC)**
- **Set of Registers**

Each process has a unique **Process ID (PID)** assigned by the OS.

Process Creation using fork()

- fork() is a **system call** used to **create a new process** by duplicating the calling (parent) process.
- The newly created process is called the **child process**.
- The original process is called the **parent process**.
- Both processes execute the same program **from the next instruction after fork()**.

Replacing Process Image using exec()

- The exec() family of system calls **replaces the current process image** with a new program.
- After exec():
- The current program is **completely replaced**.
- Execution continues from the **new program's main()**.
- The **PID remains the same**, but the program and memory are replaced.

Combining fork() and exec()

- A common use is to **create a new process** using fork() and then **replace the child's program** using exec().

Process Control System Calls

System Call	Purpose
fork()	Creates a new process
exec()	Replaces process with a new program
getpid()	Returns current process ID
getppid()	Returns parent process ID
wait()	Parent waits until child finishes
exit()	Terminates a process
kill()	Sends a signal to terminate a process

Pipes

- A **pipe** is a **unidirectional communication channel** used for **data transfer between related processes** (such as parent and child).
- It acts as a **temporary buffer** in the kernel that connects the **output of one process** to the **input of another**.

Characteristics of Pipes

Feature	Description
Direction	Unidirectional (one-way data flow)
Scope	Used between related processes (usually parent ↔ child)
Data Flow	First-In-First-Out (FIFO)
Medium	Uses kernel memory buffer
Synchronization	Reading process waits if no data; writing process waits if buffer full

Types of Pipes

Type	Description	Example Use
Unnamed Pipe	Used between related processes (parent and child).	pipe() system call
Named Pipe (FIFO)	Used between unrelated processes; has a name in the file system.	mkfifo() or mknod()

Unnamed Pipes

System Call Syntax

- `int pipe(int fd[2]);`
- `fd[0]` → file descriptor for **reading**
- `fd[1]` → file descriptor for **writing**
- Returns **0 on success, -1 on error**

Named Pipes (FIFOs)

- Named pipes allow communication between **unrelated processes** using a **name in the file system**.
- **Creating a Named Pipe**
- `$ mkfifo mypipe`
- This creates a **special file** named mypipe.

Advantages of Pipes

- Simple to use for **process communication**
- Data transfer is **synchronized automatically** (blocking reads/writes)
- Efficient because it uses **kernel buffer space**
- Ideal for **parent–child data sharing**

Signals

- A **signal** is an **asynchronous notification** sent to a process or a specific thread to inform it that an event has occurred.
- Signals can be sent **by the kernel** or **by another process**.
- When a signal is received, the process can:
 - **Handle** the signal (execute a handler function), or
 - **Ignore** the signal, or
 - **Terminate** (if default action is to kill the process)

Characteristics of Signals

Feature	Description
Asynchronous	Delivered at unpredictable times
Lightweight	Minimal data, just notification
Kernel or User Generated	Can originate from hardware exceptions, user input, or other processes
Interrupt Execution	Can pause process execution temporarily
Handled or Ignored	A process can decide how to react

Common Signal Examples

Signal	Name	Description	Default Action
1	SIGHUP	Hangup detected (e.g., terminal disconnect)	Terminate
2	SIGINT	Interrupt from keyboard (Ctrl + C)	Terminate
3	SIGQUIT	Quit from keyboard (Ctrl + \)	Core dump
9	SIGKILL	Kill signal (cannot be caught or ignored)	Terminate
11	SIGSEGV	Segmentation fault	Core dump
14	SIGALRM	Alarm clock signal	Terminate
15	SIGTERM	Termination signal	Terminate
17	SIGCHLD	Child process stopped or terminated	Ignore
18	SIGCONT	Continue if stopped	Continue
19	SIGSTOP	Stop process (cannot be	Stop

Default Actions

- Every signal has a **default action** associated with it:

Action	Description
Terminate	Ends the process
Core Dump	Terminates and dumps memory for debugging
Ignore	Signal is ignored
Stop/Continue	Suspends or resumes process

Basic Threading

- A **thread** is a **lightweight sub-process** that represents an independent flow of control within a program.
- Threads are often used to perform **multiple tasks concurrently** within a single process, such as:
 - Handling multiple client connections
 - Performing background computations
 - Managing user interface responsiveness

Process vs. Thread

Feature	Process	Thread
Definition	Independent program with its own memory	Lightweight unit of execution within a process
Memory Space	Has separate memory	Shares memory with other threads in the same process
Creation Overhead	High	Low
Communication	Through IPC (pipes, sockets, etc.)	Through shared memory (faster)
Crash Impact	Crash does not affect other processes	Crash may affect all threads in the same process
Speed	Slower to create and switch	Faster context switching

Thread States

- A thread typically moves through the following **lifecycle states**:

State	Description
New	Thread created but not yet started
Runnable	Ready to run or currently running
Blocked/Waiting	Waiting for an event or resource
Terminated	Thread has completed execution

Thread Safety

- A function is **thread-safe** if it can be called simultaneously by multiple threads **without causing conflicts**.
To ensure safety:
- Use **mutexes** or **semaphores**
- Avoid **global variables** or **shared writable data**
- Use **thread-local storage** if necessary